







中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生数学建模竞赛

学　　校	
参赛队号	
队员姓名	1.
	2.
	3.

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题　目：　基于航段能耗与视距中继耦合模型的山区洪涝无人机运输通信协同调度研究

摘　要：

山区洪涝灾害同时切断道路、供电与通信，被困居民点的饮用水、药品和食品只能依靠空中投送。本文以广西横州镇龙乡的标准测试场景为对象，以30米数字高程模型刻画航段净空与视线遮挡，依次建立单点往返最大安全载荷模型、异构多架次协同调度模型、运输与中继联合调度模型和任务分区资源核算模型，研究载荷能力、配送及时性、通信保障与资源配置之间的制约关系。

针对问题一，首先沿两节点直线以5米步长采样数字高程模型，取沿线最高地面高程抬升50米得到计划巡航海拔，据此分解爬升、巡航与下降三段；由于题目未给出水平能耗与爬升附加能耗的闭式，本文把水平能耗按等效航程折算、把爬升附加能耗按整机势能除以爬升能耗效率折算，得到架次能耗关于载荷严格单调的表达；然后以返航安全余量为判据二分求解，得到**最大安全载荷**，A型在全部15个服务区均为25 kg，C型在最远的S008降至58.90 kg，较其额定载货质量下降26.4%；最后把货箱组批写成带机型选择的二维装箱问题，用**约束规划**按架次数、总能耗、累计作业时间的字典序精确求解，得到**18个架次**、总运输能耗**59.13 kWh**、累计作业时间**32776.2 s**，并另用**子集动态规划**独立重解，两种算法逐服务区结果完全一致。灵敏度分析表明返航余量不超过0.20时组批方案不变，达到0.40时最远的两个服务区无解。

针对问题二，首先在问题一的基础上放开单点往返限制，把实体无人机、共享电池、两阶段充电周转和物资时限一并纳入；然后以架次序列作为解的唯一表示，把派机、配电池与起飞时刻交给一个确定性可行性仿真器还原，从而使任意解自动满足资源约束，建立**异构无人机多点多架次协同调度模型**；最后用**自适应大邻域搜索与模拟退火混合算法**求解，并与贪心构造、纯模拟退火、纯大邻域搜索三种方法各独立运行6次对比，混合算法得到的最好解目标值最低；五组权重共18次运行按基准权重重评后取最优，得到**22个架次**、运输能耗**63.86 kWh**、全部任务完成时间**7009.7 s**，80个货箱的首批截止与期望送达要求全部满足，电池峰值占用等于库存上限，是本问的紧约束。为验证解的稳健性，另以已有的37组解为初始种群做遗传算法搜索，未能找到更优方案。

针对问题三，首先按双向链路预算计算三类链路门限，直连122 dB、接入116 dB、回传126 dB，对应视距作用距离12.51、6.27、19.83 km，遮挡后分别降至3.96、1.98、6.27 km；逐点视线穿地判定发现15个服务区中仅3个可与固定网关直连，链路可用性几乎完全由地形决定；然后在研究区上以400米格网、离地100至300米枚举悬停候选，先用回传视距筛选再按**航段整段覆盖的集合覆盖**选出**3个悬停点**，建立**运输与中继联合调度模型**；最后把中继班次表与运输架次序列放进同一个解一并搜索，得到**28个运输架次**加**5个中继架**

次，联合任务完成时间10919.4 s，总能耗85.87 kWh，独立核验的9188个采样点全部处于保障之下，按时长计中继承担35.3%；与问题二相比完成时间延长55.8%，这是山区连续通信保障的代价。

针对问题四，首先由同架次共访关系做**并查集合并**，得到**5个不可再分的任务原子块**，其中西部与北部两簇各自完整、东部簇碎成三块；然后按**区间重叠计数**核算各组独立执行所需的四类资源峰值，建立**任务分区资源核算模型**；最后枚举全部合法分区并按缺口、规模、失衡排序，得到分为2组时共需**31台套**、缺口3台套，分为3组时共需**37台套**、缺口8台套，两种分法都是一个大组加若干小组，组间工作量失衡指标由1.866升到2.570，说明原子块粒度过细时分组既增冗余又拉大失衡，缺口源于资源不得跨组调配切断了时间上的错峰复用，补缺重点落在库存最紧的B型机与中继机上。

关键词： 无人机调度、数字高程模型、链路预算、通信中继、约束规划、大邻域搜索

目 录

一、 问题重述 4
1.1 问题背景 4
1.2 问题重述 5
1.3 解题总体思路 6
二、 模型的假设 7
三、 符号说明 7
四、 问题一模型的建立、分析与求解 8
4.1 问题分析 8
4.2 航段几何与地形净空 9
4.3 飞行时间与航段能耗口径 10
4.4 最大安全载荷模型的建立 11
4.5 货箱组批模型的建立 12
4.6 模型的求解 13
4.7 结果分析 13
4.8 返航安全余量的灵敏度分析 16
4.9 模型的检验 16
4.10 本章小结 17
五、 问题二模型的建立、分析与求解 17
5.1 问题分析 17
5.2 能源周转的刻画 18
5.3 可行性仿真器 18
5.4 协同调度模型的建立 20
5.5 模型的求解 21
5.6 结果分析与资源可行性检验 23
5.7 本章小结 28
六、 问题三模型的建立、分析与求解 29
6.1 问题分析 29
6.2 通信链路预算与遮挡判定 29
6.3 中继悬停点选址模型 31
6.4 运输与中继联合调度模型的建立 33
6.5 模型的求解 34
6.6 结果分析与保障率检验 35
6.7 本章小结 39
七、 问题四模型的建立、分析与求解 40
7.1 问题分析 40
7.2 任务原子块的识别 40
7.3 分区资源核算模型的建立 41
7.4 分区方案的比较与资源缺口 42
7.5 模型的检验 45
7.6 本章小结 45
八、 模型的评价 46
8.1 模型的优点 46
8.2 模型的缺点 46
8.3 模型的改进 46
8.4 模型的推广 47
参考文献 47
附录 48

一、 问题重述

1.1 问题背景

山区洪涝突发性强、影响范围分散，会在同一时段切断道路、供电与通信。连续强降雨引发的山洪、滑坡与塌方使部分居民点在短时间内与外界失去稳定的地面联系，而饮用水、应急食品、医疗物资和生活卫生用品必须在灾后早期送达。传统车辆难以进入地形起伏大、路面损毁严重的区域，无人机受地面道路条件影响小，因而成为应急运输的重要补充手段。

2026 年 7 月，受台风带来的持续强降雨影响，广西横州市镇龙乡遭遇严重洪涝，洪水、山体落石和道路塌方从多个方向阻断通行，核心受灾区域一度断路、断电、断网，全乡约 8000 人受困 [1]。救援期间食品、饮用水和药品依靠徒步或空中投送进入受灾区域，大型无人机还承担了电力抢修材料的运输任务，临时通信设备与应急供电设备被用于逐步恢复灾区联络。这一现实场景说明，应急投送不只是把货箱送达目标点：运输无人机在爬升、巡航、下降与物资交接的全过程中，都需要保持指挥、遥测和交接确认链路；当固定通信设施受损或山体遮挡使地面网关无法覆盖完整任务区间时，还需要空中中继无人机在适当的位置和时段提供临时保障。

本文研究的标准测试场景以镇龙乡及周边山区为地理背景，包含 1 个临时调度中心 O01、15 个服务区和 80 个不可拆货箱，配置三种运输机型共 8 架实体无人机、配套共享电池组、2 架中继无人机及其可更换能源组件，并给出 30 米分辨率的数字高程模型 [2] 与无线链路参数。研究区的地形与节点分布如图 1 所示，该图在 ArcMap 中制作，底图是按高程分层设色、再叠加半透明山体阴影的地形，其上依次是水体、蓝色的水系、深褐色的道路、黄色的村镇点位，以及 15 个服务区与调度中心 O01。图中一条主河沿南北方向穿过研究区，O01 位于这条河谷内，道路沿河谷与支谷延伸，而服务区大多位于支谷末端、与主谷之间隔着山脊。这正是灾后地面交通一旦中断便难以恢复的地理成因，也预示着视线遮挡将成为通信保障的主要障碍。

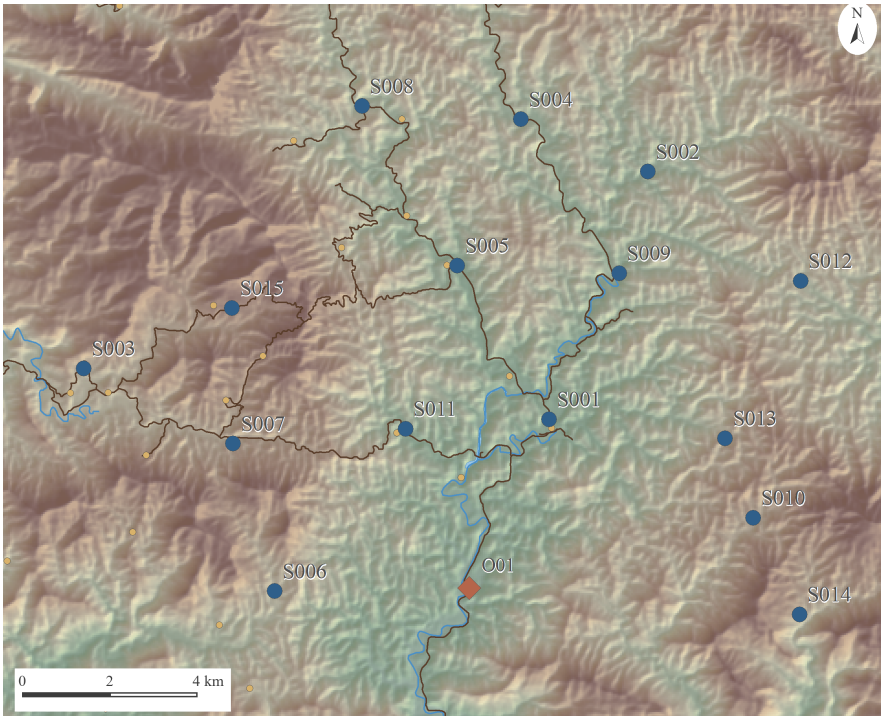


图 1 研究区地形与节点分布

赛题的高程数据以经纬度给出，而航段距离、视线判定与可视域分析都要在米制平面上进行。本文先用 ArcGIS 把高程数据投影到 UTM 第 49 带、按 30 米重采样，再把投影结果载入 ENVI 5.6 核对，如图 2 所示。右下的元数据栏显示投影为 UTM 第 49 带北半球、像元 30 米、共 1427 列 1360 行，与后续计算所用的参数一致；主窗口按 2% 线性拉伸显示，亮处是高山、暗处是谷地，高程范围 42 至 1129 米。中部一条南北向的主河谷把研究区一分为二，两侧支谷伸向山体，服务区大多位于这些支谷的末端。

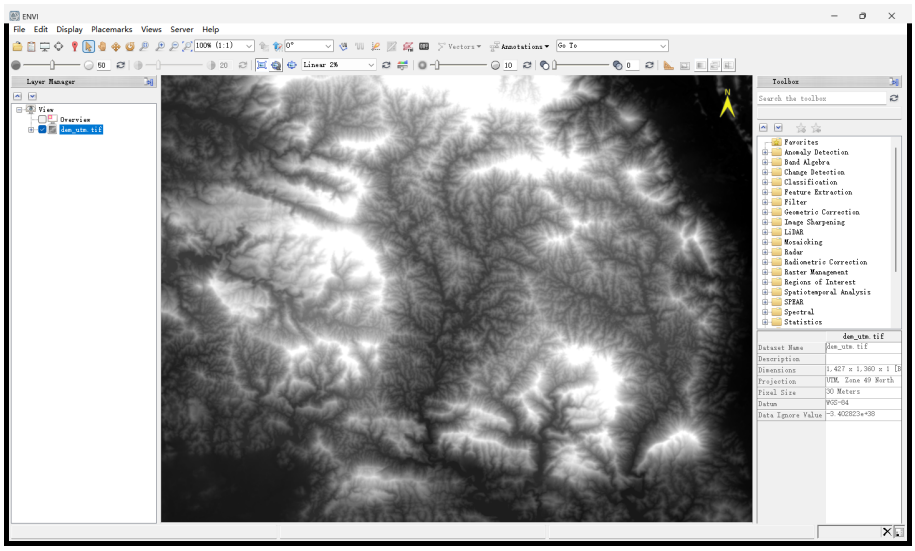


图 2　数字高程模型的投影核对

1.2　问题重述

围绕运输组织、通信匹配与资源调配之间的协同关系，题目提出四个层层递进的问题。

问题一：单点往返运输能力与货箱组批方案。在不考虑实体无人机和共享电池调度的条件下，研究单服务区直接往返的运输能力与货箱组批。每个架次采用 $O01 \to S_i \to O01$ 的直接往返形式且仅服务一个服务区，同一服务区可由多个架次分批服务，不得跨服务区组批。需要计算三种机型在各服务区执行单点往返任务时的最大安全载荷；在货箱不可拆分、每个货箱只能安排一次并满足载质量、装载体积和返航安全能量余量的条件下确定各服务区的货箱组批方案；在完成全部货箱交付的基础上综合考虑往返架次数、总运输能耗和累计作业时间进行优化，并说明各指标之间的权衡关系或优先关系；同时讨论返航安全余量变化对不同机型最大安全载荷和组批结果的影响。

问题二：异构无人机多点多架次运输调度。在不考虑通信保障的条件下研究异构运输无人机的多点多架次调度。每个运输架次可访问一个或多个服务区，并在完成物资交付后返回 O01。需要在货箱不可拆分、载质量与装载体积、返航安全余量、无人机与共享电池数量、充电周转及物资时限等约束下，联合确定货箱组批、服务区访问顺序、运输机型、具体执行无人机、共享电池及各架次开始时刻。其中医疗物资应满足期望送达时间要求，首批保障货箱应满足首批截止时间要求，其他物资的期望送达时间用于衡量配送及时性。需要综合考虑配送及时性、全部任务完成时间、运输能耗和架次数等指标进行优化并说明权衡关系，给出运输路线、架次安排、逐箱送达时刻以及无人机和电池的资源使用情况，并检验方案的资源可行性。

问题三：通信约束下的运输与中继联合调度方案。在考虑通信保障的条件下研究运输无人机与中继无人机的联合调度。运输无人机在爬升、巡航、下降及物资投送阶段均应保持连续通信；与固定网关直连不可用时，可由一架空中中继无人机提供通信保障，不允许

中继之间多跳转发。需要在货箱时限、载荷与能量、连续通信及资源可用性等约束下，联合确定货箱组批与访问顺序、运输无人机及共享电池安排、各架次开始时刻，以及中继无人机的悬停位置、飞行海拔、服务时段和能源组件分配。中继悬停位置应位于所给数字高程模型覆盖范围内，悬停离地高度不超过规定上限。需要综合考虑配送及时性、联合任务完成时间、运输与中继总能耗以及两类无人机的使用架次等指标进行优化，并说明货箱交付、资源使用和通信保障情况。

问题四：救援任务分区与资源配置优化方案。以问题三得到的联合调度方案为基础，分别将 15 个服务区划分为 2 个和 3 个任务组。分区方案由参赛者确定，每个服务区必须且只能属于一个任务组，每个任务组至少包含一个服务区。分区过程中保持问题三已经确定的货箱组批、服务区访问顺序、运输与中继任务安排及通信保障关系不变；若同一运输架次同时涉及多个服务区，则这些服务区应划入同一任务组。各任务组仅承担本组服务区对应的运输与通信保障任务，并分别核算独立执行所需的运输无人机、共享电池、中继无人机和中继能源组件数量，任务执行期间各类资源不得跨组调配。需要从资源配置规模、资源冗余、组间工作量均衡以及与现有库存之间的资源缺口等方面比较两种分区方式的特点，并对所需资源超过现有库存的情形给出缺口及其原因。

1.3 解题总体思路

四个问题层层递进，前一问的结论是后一问的输入，全文的推理脉络如图 3 所示。图中第一行是四问共用的基础层：先把附件数据与 30 米数字高程模型整理成统一的节点表，再沿每条航段采样取最高点确定巡航海拔，据此把航段分解为爬升、巡航、下降三段，并确定等效航程意义下的能耗口径与三类通信链路的门限。其后每一行对应一个问题，行末的框给出该问的核心结果。问题一在单点往返的限制下求出各机型的载荷上限并完成组批；问题二解除单点往返限制，把实体无人机、共享电池与充电周转纳入，用仿真器还原资源分配、用混合启发式搜索架次序列；问题三引入连续通信这一硬约束，先判定视线遮挡、再做整段集合覆盖选址、最后把中继班次与运输架次放进同一个解联合搜索；问题四不改动调度，在问题三的方案上识别原子块并核算分区后的资源需求。四问共用同一套航段几何与能耗口径，保证结果可以相互衔接。

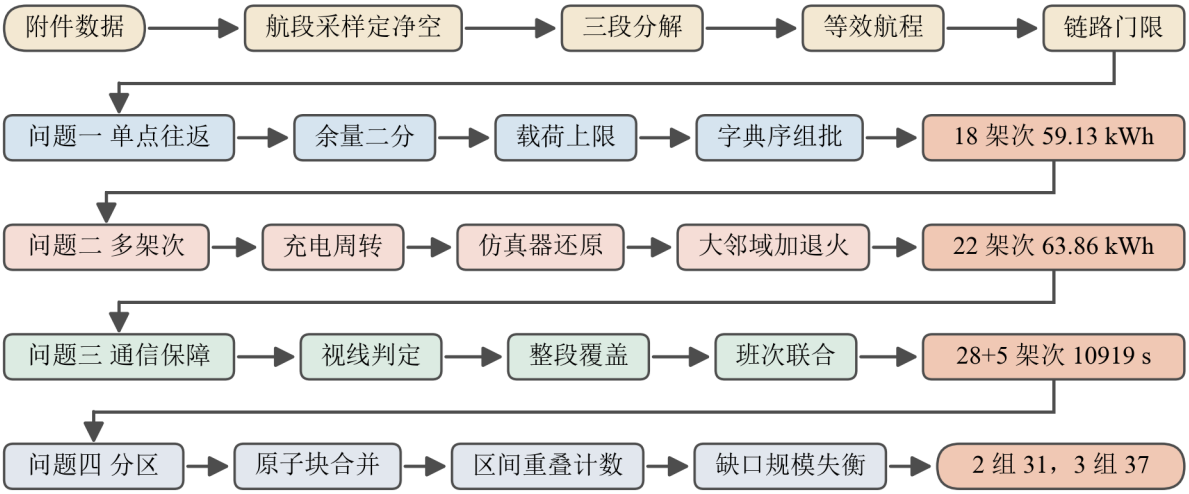


图 3 全文解题思路

二、 模型的假设

为明确建模前提，本文对无人机运输与通信协同问题作出以下假设。

（1）认为任意两个任务节点之间沿水平直线飞行，计划巡航海拔取该直线所经数字高程模型像元的最高地面高程以上 50 米，不考虑绕飞与动态避障。

（2）认为爬升、巡航、下降三段依次进行且均以各自的标准速度匀速完成：爬升与下降为垂直运动，不产生水平位移；段间转换的加减速时间计入所在航段，不单独建模。

（3）认为水平飞行的单位航程能耗由等效航程决定，即整块电池的可用能量恰好支持飞完当前载荷下的等效航程；爬升附加能耗由整机势能增量除以爬升能耗效率得到；下降不单独计入能耗。

（4）认为同一机型的共享电池可在该机型的不同实体无人机之间自由调度，不同机型之间不可混用；电池初始荷电状态为 100%，再次投入任务前必须充至 100%。

（5）认为货箱装卸与交接在服务区上空 30 米的作业高度完成，交接期间无人机保持悬停，其能耗计入所在架次的地面作业时间而不额外计入飞行能耗。

（6）认为无线传播满足自由空间损耗规律，视线被地形遮挡时附加固定的遮挡损耗，不考虑降雨衰减、多径效应与同频干扰，也不考虑地球曲率对视线的影响。

（7）认为中继无人机到达悬停位置并完成建链后即可持续提供保障，悬停期间位置与高度保持不变；同一时刻每架运输无人机只由固定网关或一架中继无人机提供保障，不允许中继之间多跳转发。

（8）认为救援期间气象条件稳定，不考虑风速风向对飞行时间与能耗的影响，也不考虑无人机故障与临时任务插入。

三、 符号说明

序号	符号	单位	含义
1	g	—	运输无人机机型，取 A、B、C
2	S_i	—	第 i 个服务区，$i = 1, 2, \ldots, 15$
3	q	kg	当前有效载荷
4	Q_g	kg	机型 g 的最大载货质量
5	V_g	m^3	机型 g 的可用装载体积
6	m_g	kg	机型 g 的含电池空载总质量
7	$L_g(q)$	m	机型 g 携带载荷 q 时的等效航程
8	$L_g^0,\ L_g^F$	m	机型 g 的空载与满载标准航程
9	E_g^{use}	kWh	机型 g 单组电池的可用能量
10	ρ_g	—	机型 g 的返航安全余量比例
11	$\eta_g^{\uparrow}$	—	机型 g 的爬升能耗效率
12	$v_g^{\uparrow}, v_g^{c}, v_g^{\downarrow}$	m/s	机型 g 的爬升、巡航、下降速度
13	$Z(\cdot)$	m	数字高程模型给出的地面高程
14	z_{ij}^{c}	m	航段 $i \to j$ 的计划巡航海拔
15	Δz	m	计划巡航海拔相对沿线最高地面高程的抬升净空
16	d_{ij}	m	航段 $i \to j$ 的水平巡航距离
17	$D_{ij}(t)$	km	通信端点 i 与 j 之间的三维直线距离
18	$h_{ij}^{+},\ h_{ij}^{-}$	m	航段 $i \to j$ 的爬升与下降高度
19	t_{gij}	s	机型 g 在航段 $i \to j$ 的飞行时间
20	$E_{gij}(q)$	kWh	机型 g 载荷 q 时航段 $i \to j$ 的能耗

序号	符号	单位	含义
21	$\Phi_{g,i}(q)$	kWh	机型 g 往返服务区 S_i 携带载荷 q 的架次能耗
22	$q^{*}_{g,i}$	kg	机型 g 往返服务区 S_i 的最大安全载荷
23	m_b	kg	货箱 b 的质量
24	v_b	m^3	货箱 b 的体积
25	w_b	—	货箱 b 的应急优先系数
26	$\tau_b^F,\ \tau_b^E$	s	货箱 b 的首批截止时间与期望送达时间
27	T_b	s	货箱 b 的实际送达时刻
28	T_p	s	架次 p 的总时长
29	$s_p,\ r_p$	s	架次 p 的开始时刻与返回时刻
30	σ_p	—	架次 p 结束时电池的荷电状态
31	T_{full}	s	电池或能源组件的等效完全充电时间
32	$t_{\text{chg}}(\sigma)$	s	荷电状态 σ 充至满电所需时间
33	P^{b}_{sens}	dBm	接收端 b 的接收灵敏度
34	M^{b}	dB	接收端 b 的衰落裕量
35	P^{b}_{th}	dBm	接收端 b 的有效接收门限
36	$L_{\text{FSPL},ij}(t)$	dB	端点 i 与 j 之间的自由空间传播损耗
37	$L_{\text{path},ij}(t)$	dB	考虑地形遮挡后的总传播损耗
38	L_{obs}	dB	地形遮挡附加损耗
39	$L^{a\leftrightarrow b}_{\max}$	dB	主体 a 与 b 之间的双向链路门限
40	$b_{ij}(t)$	—	地形遮挡变量，遮挡时取 1
41	$A_{ij}(t)$	—	链路可用性，可用时取 1
42	$\mathcal{H}$	—	中继悬停候选点集合
43	Δ_k	s	中继班次 k 的服务时长
44	C_{mk}	s	全部任务完成时间或联合任务完成时间
45	$\Lambda,\ \Gamma$	—	软性延误与硬性违约惩罚
46	$\mathcal{G}_k$	—	第 k 个任务组所含的服务区集合
47	$t^{\text{prep}}, t^{\text{link}}, t^{\text{turn}}$	s	中继的工位固定准备、建链与架次周转时间

其余符号在文中首次出现处说明。题目附件给出的机型参数与链路参数列于表 20 与表 21，全文计算一律取这两张表的值。

四、 问题一模型的建立、分析与求解

4.1 问题分析

问题一要求在不考虑实体无人机与共享电池调度的条件下，给出三种机型在各服务区执行单点往返任务时的最大安全载荷，并在此基础上完成 80 个不可拆货箱的组批。

题目所给的计算规则表明，本问的难点不在优化本身，而在把题目描述的物理过程准确地翻译成可计算的量。题目规定任意两节点之间取水平直线航段，计划巡航海拔取沿线数字高程模型像元的最高地面高程再抬升 50 米，因此必须先沿直线密集采样高程才能确定巡航海拔，进而分解出爬升高度、水平距离与下降高度三段。调度中心的作业高度取其地面海拔，服务区的作业高度取地面海拔以上 30 米，两端作业高度不同使去程与回程的爬升下降并不对称。

题目给出了等效航程随载荷按 3/2 次幂衰减的表达式，并规定航段能耗由水平巡航能耗与爬升附加能耗相加、下降不单独计入，但没有给出这两项的闭式，其口径必须由本文自行确定。由于多旋翼的需用功率主要由总重决定，同一机型在相同巡航速度下单位航程

的能耗与总重正相关，而等效航程正是整块电池在该载荷下可支持的水平航程，本文把水平能耗写成按等效航程折算的形式，把爬升附加能耗写成整机势能增量除以爬升能耗效率的形式。这一口径与多旋翼功率随总重单调上升的结论一致 [3][4]，也与题目对中继无人机爬升附加能耗按相同关系计算、质量取计划起飞总质量的规定相容。由此得到的架次能耗关于载荷严格单调，最大安全载荷可由返航安全余量条件二分求出。

给定各机型在各服务区的最大安全载荷后，组批是一个带机型选择的二维装箱问题：质量受最大安全载荷约束，体积受装载容积约束，两者都可能成为瓶颈。由于单个服务区的货箱数最多 15 个，问题规模允许精确求解，本文用约束规划 [5] 按架次数、总能耗、累计作业时间的字典序依次最小化，并通过改变返航安全余量比例观察载荷与组批结果的变化。

问题一的求解思路如图 4 所示。第一行完成从数据到能耗表达的准备，核心是证明往返能耗随载荷单调；第二行在此基础上先二分出载荷上限，再把组批写成带机型选择的装箱模型精确求解，最后考察返航余量的影响。

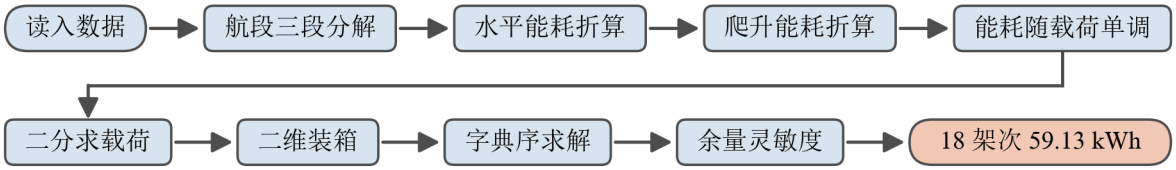


图 4　问题一的求解思路

4.2　航段几何与地形净空

节点由经纬度与地面海拔给定。为在米制下计算距离，先把经纬度投影到 UTM 第 49 带平面，记节点 i 的平面坐标为 (x_i, y_i)，则航段 $i \to j$ 的水平巡航距离为

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}. \tag{1}$$

沿该直线以 $\delta = 5$ 米的步长等距取 $N_{ij} = \lceil d_{ij}/\delta \rceil$ 个采样点，在数字高程模型上以最近邻方式读取地面高程，计划巡航海拔取沿线最高地面高程再抬升净空 $\Delta z = 50$ 米，即

$$z^{c}_{ij} = \max_{0 \le n \le N_{ij}} Z\!\left(P_i + \frac{n}{N_{ij}}(P_j - P_i)\right) + \Delta z, \tag{2}$$

其中 P_i 表示节点 i 的平面位置。记调度中心 O01 的作业高度为其地面海拔 z_0，服务区 S_i 的作业高度为其地面海拔以上 30 米，即 $z_i = Z_i + 30$，则爬升与下降高度为

$$h^{+}_{ij} = \max\{z^{c}_{ij} - z_i,\, 0\}, \qquad h^{-}_{ij} = \max\{z^{c}_{ij} - z_j,\, 0\}. \tag{3}$$

题目规定三段是先后进行的：无人机先在起点上空垂直爬升至巡航海拔，再沿水平直线巡航，到达终点上空后垂直下降至作业高度。爬升与下降不产生水平位移，式 (5) 中三项分别只依赖各自的高度差或水平距离，与这一顺序一致。巡航海拔的确定与三段分解如图 5 所示，图中取最长的 O01 至 S008 航段。图 5(a) 是俯视，航段是连接起降坪与村落的一条直线，线上的短刻度是采样点，放大圈里相邻两点的间距就是步长 δ，红点是沿线地面最高的采样点。图 5(b) 是同一航段的侧视剖面，每根竖线对应一个采样点的地面高程，最高的一根标成红色，巡航海拔 z^{c} 就由它抬升 50 米得到；无人机在左端垂直爬升 h^{+}，沿 z^{c} 水平飞行，到右端垂直下降 h^{-} 至离地 30 米的作业高度。图中巡航海拔只由沿线唯一的最高点决定，其余采样点上方都留有远大于 50 米的净空，这是直线航段假设带来的能耗冗余，也是后文讨论模型改进时的着眼点。

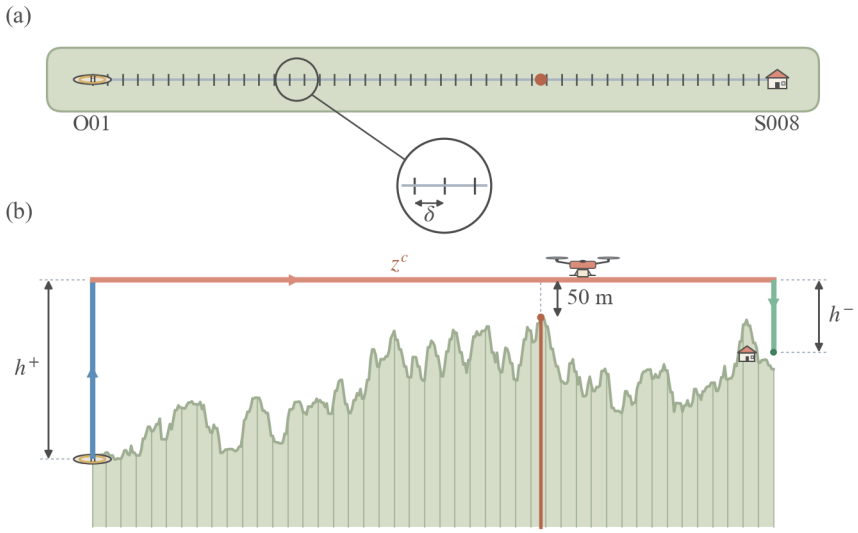


图 5　航段采样与三段分解：(a) 俯视采样点；(b) 侧视巡航剖面

4.3　飞行时间与航段能耗口径

机型 g 携带载荷 q 时的等效航程按题目给定的关系计算

$$L_g(q) = L_g^0 - \left(L_g^0 - L_g^F\right)\left(\frac{q}{Q_g}\right)^{3/2}, \qquad 0 \le q \le Q_g. \tag{4}$$

航段飞行时间由三段速度决定

$$t_{gij} = \frac{h_{ij}^{+}}{v_g^{\uparrow}} + \frac{d_{ij}}{v_g^{c}} + \frac{h_{ij}^{-}}{v_g^{\downarrow}}. \tag{5}$$

如前所述，本文取水平能耗按等效航程折算

$$E_{gij}^{\mathrm{hor}}(q) = E_g^{\mathrm{use}} \cdot \frac{d_{ij}}{L_g(q)}, \tag{6}$$

爬升附加能耗取整机势能增量除以爬升能耗效率并换算为千瓦时

$$E_{gij}^{\mathrm{up}}(q) = \frac{(m_g + q)\, g_0\, h_{ij}^{+}}{\eta_g^{\uparrow}} \cdot \frac{1}{3.6 \times 10^{6}}, \tag{7}$$

其中 $g_0 = 9.81\ \mathrm{m/s^2}$。于是航段总能耗为

$$E_{gij}(q) = E_{gij}^{\mathrm{hor}}(q) + E_{gij}^{\mathrm{up}}(q). \tag{8}$$

式 (6) 与式 (7) 均随载荷单调递增，这一性质是后文用二分法求最大安全载荷的前提。每个架次 p 须满足返航安全余量条件

$$E_p^{T} = \sum_{(i,j)\in p} E_{gij}(q_{pij}) \le (1 - \rho_g)\, E_g^{\mathrm{use}}, \tag{9}$$

其中 q_{pij} 是架次 p 在航段 $i \to j$ 上的剩余载荷。架次结束时电池的荷电状态为 $\sigma_p = 1 - E_p^{T}/E_g^{\mathrm{use}}$。

一个架次的总时长从工位准备开始计起，包含固定准备时间、按箱数计的装载时间、各航段飞行时间，以及在每个服务区的接收点基础交接时间与按箱数计的增加交接时间。记架次 p 装载 n_p 个货箱、在服务区 S_i 投送 n_{pi} 个货箱，则

$$T_p = t_g^{\text{prep}} + t_g^{\text{load}} n_p + \sum_{(i,j)\in p} t_{gij} + \sum_{i\in p} \left(t_g^{\text{hand}} + t_g^{\text{box}} n_{pi} \right). \tag{10}$$

上述各式在后续三问中反复使用，不再重复推导。

4.4 最大安全载荷模型的建立

单点往返架次由去程与回程两段构成。去程携带全部货箱，从 O01 的地面海拔爬升至巡航海拔，水平飞行 d_{0i} 后下降至服务区上空 30 米；回程空载，从 30 米作业高度重新爬升，水平飞回后下降至 O01 地面。两段的巡航海拔相同，但爬升下降高度互换，且去程带载而回程空载，因此架次能耗并非去程的两倍。记机型 g 往返服务区 S_i 携带载荷 q 的架次能耗为

$$\Phi_{g,i}(q) = E_{g,0i}(q) + E_{g,i0}(0), \tag{11}$$

其中第一项按式 (8) 以载荷 q 计算，第二项以空载计算。

由式 (4) 知 $L_g(q)$ 关于 q 严格递减，故式 (6) 关于 q 严格递增；式 (7) 关于 q 线性递增。因此 $\Phi_{g,i}(q)$ 在 $[0, Q_g]$ 上严格单调递增。最大安全载荷定义为在满足返航安全余量的前提下单个往返架次可携带的最大质量，即

$$q_{g,i}^{*} = \max \left\{ q \in [0, Q_g] \;\middle|\; \Phi_{g,i}(q) \le (1-\rho_g)\, E_g^{\text{use}} \right\}. \tag{12}$$

综上所述，最终得到**单点往返最大安全载荷模型**为

$$\begin{cases} z_{0i}^{c} = \max\limits_{n} Z\left(P_0 + \frac{n}{N}(P_i - P_0)\right) + 50, \\ L_g(q) = L_g^0 - \left(L_g^0 - L_g^F\right)\left(q/Q_g\right)^{3/2}, \\ \Phi_{g,i}(q) = E_g^{\text{use}} \left(\dfrac{d_{0i}}{L_g(q)} + \dfrac{d_{i0}}{L_g(0)} \right) + \dfrac{g_0 \left[(m_g + q) h_{0i}^{+} + m_g h_{i0}^{+} \right]}{3.6 \times 10^6\, \eta_g^{\uparrow}}, \\ q_{g,i}^{*} = \max \left\{ q \in [0, Q_g] \;\middle|\; \Phi_{g,i}(q) \le (1-\rho_g) E_g^{\text{use}} \right\}. \end{cases} \tag{13}$$

由于 $\Phi_{g,i}$ 严格单调，若 $\Phi_{g,i}(Q_g)$ 不超过能量预算则 $q_{g,i}^{*} = Q_g$，否则在 $[0, Q_g]$ 上二分求解，精度取 10^{-3} kg。

这一判据可以看成把一块电池的可用电量切成三段，如图 6 所示。

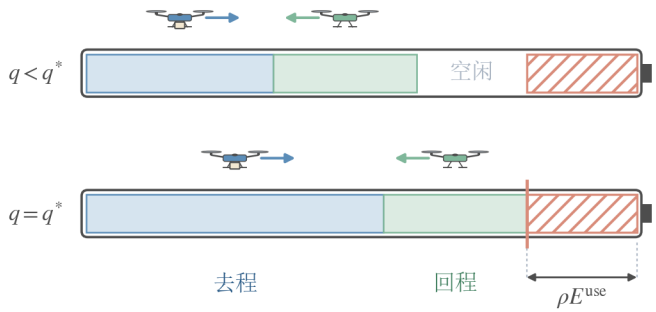


图 6　返航余量约束下最大安全载荷的判据

左段是带载去程的耗电，中段是空载回程的耗电，右端带斜纹的一段是必须留作返航余量的 ρE^{use}。上排载荷小于上限，去程与回程之和尚未触及余量，中间留有空隙；随着载荷增加，去程段按等效航程的 $3/2$ 次幂关系变长，而回程段因空载不变，直到下排两段之和恰好达到余量的边界，此时的载荷就是 q^{*}。去程段的增长速度决定了二分的收敛结果，也解释了最远服务区载荷上限降幅最大的原因。

4.5 货箱组批模型的建立

设服务区 S_i 的货箱集合为 $\mathcal{B}_i$，允许为每种机型开设至多 $|\mathcal{B}_i|$ 个架次。

组批的对象与约束如图 7 所示。左侧是一个服务区待运的货箱，四种颜色分别代表饮用水、医疗物资、应急食品与生活卫生用品，大小不一；右侧是三种机型的挂舱，A 型最小、C 型最大，舱侧的竖线表示每个舱同时受载质量上限 Q_g 与装载体积上限 V_g 的约束，而在远距离服务区，质量上限还要进一步收紧为最大安全载荷。货箱只能整箱装入某一架次，组批就是决定每个货箱进哪一型机的哪一个舱，使架次数、能耗与作业时间依次最小。

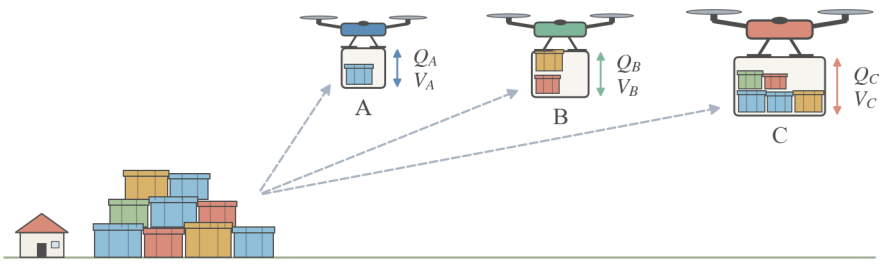


图 7　单点往返组批的装箱关系

决策变量的建立。待定的是每个货箱的归属架次与每个架次所用机型。引入 $x_{bgk} \in \{0,1\}$ 表示货箱 b 装入机型 g 的第 k 个架次，$u_{gk} \in \{0,1\}$ 表示该架次被启用。

约束条件的建立。每个货箱不可拆分且只能安排一次，故对每个 b 恰有一个 x_{bgk} 取 1；每个架次的总质量不得超过该机型在该服务区的最大安全载荷，总体积不得超过可用装载体积；架次未启用时不得装箱。

目标函数的建立。题目要求综合架次数、总运输能耗与累计作业时间三项。三者量纲不同且重要程度有明确的先后：架次数直接决定装备占用与出动规模，是首要指标；在架次数相同的前提下再比较能耗；能耗相同时比较作业时间。因此采用字典序优化。

综上所述，最终得到**单点往返货箱组批模型**为

$$
\begin{aligned}
&\text{lex-min} \quad \left(\sum_{g,k} u_{gk},\ \sum_{g,k} u_{gk}\,\Phi_{g,i}(M_{gk}),\ \sum_{g,k} T_{gk} \right) \\
&\text{s.t.} \quad \begin{cases}
\displaystyle\sum_{g,k} x_{bgk} = 1, & \forall b \in \mathcal{B}_i, \\
M_{gk} = \displaystyle\sum_{b} m_b x_{bgk} \le q^{*}_{g,i}\, u_{gk}, & \forall g,k, \\
\displaystyle\sum_{b} v_b x_{bgk} \le V_g\, u_{gk}, & \forall g,k, \\
T_{gk} = \left(t_g^{\text{prep}} + t_g^{\text{hand}} + t_{g,0i} + t_{g,i0}\right) u_{gk} + \left(t_g^{\text{load}} + t_g^{\text{box}}\right) \displaystyle\sum_{b} x_{bgk}, & \forall g,k, \\
x_{bgk},\ u_{gk} \in \{0,1\}.
\end{cases}
\end{aligned}
\tag{14}
$$

目标中的能耗与时间都乘以启用变量 u_{gk}，未启用的架次不计入。由于货箱质量均为整数，M_{gk} 取值于整数集合，架次能耗可事先制成以质量为索引的整数表，在约束规划中用元素约束直接查表，避免在模型内重复求值。

4.6 模型的求解

求解采用约束规划精确算法。该问题每个服务区最多 15 个货箱，状态空间有限，精确求解既能给出最优值又能为后续三问的启发式方法提供基准。求解步骤如下。

Step1：读入节点坐标与数字高程模型，按式 (2) 与式 (3) 预计算各服务区往返航段的巡航海拔、水平距离与爬升下降高度。

Step2：对每种机型每个服务区，按式 (12) 二分求最大安全载荷 $q^{*}_{g,i}$。

Step3：以质量为索引建立架次能耗整数表，超过 $q^{*}_{g,i}$ 的条目置为大数以排除不可行解。

Step4：对每个服务区建立模型 (14)，加入架次对称破缺约束，即同机型的架次按启用状态与载荷非增排列，以压缩等价解带来的搜索开销。

Step5：最小化架次数得 N^{*}，固定 $\sum u_{gk} = N^{*}$。

Step6：在此基础上最小化总能耗得 E^{*}，固定后再最小化累计作业时间。

Step7：汇总 15 个服务区的解，输出逐架次组批、总质量、总体积、往返时间、架次能耗与返航荷电状态。

4.7 结果分析

4.7.1 最大安全载荷

三种机型在各服务区的最大安全载荷如表 1 所示。A 型在全部 15 个服务区均可满载 25 kg，说明其 4.5 kWh 的电池相对 25 kg 的最大载货质量留有充裕余量；B 型仅在最远的 S008 降至 28.80 kg，其余服务区均可满载；C 型在 S002、S003、S004、S008、S012 五个远距离服务区分别降至 68.86、68.20、63.70、58.90、68.12 kg，最大降幅出现在水平距离 8.08 km 的 S008，较其 80 kg 的额定载货质量下降 26.4%。

表 1　三种机型在各服务区的最大安全载荷，单位 kg

服务区	A 型	B 型	C 型	服务区	A 型	B 型	C 型
S001	25.00	30.00	80.00	S009	25.00	30.00	80.00
S002	25.00	30.00	68.86	S010	25.00	30.00	80.00
S003	25.00	30.00	68.20	S011	25.00	30.00	80.00
S004	25.00	30.00	63.70	S012	25.00	30.00	68.12
S005	25.00	30.00	80.00	S013	25.00	30.00	80.00
S006	25.00	30.00	80.00	S014	25.00	30.00	80.00
S007	25.00	30.00	80.00	S015	25.00	30.00	80.00
S008	25.00	28.80	58.90				

三种机型的载荷能力随航距的变化如图 8(a) 所示，纵轴是最大安全载荷与额定载货质量之比。A 型的曲线全程维持在 1.0，B 型只在最远的 S008 降至 0.96；C 型的曲线在 6 km 之内保持满载，其后急剧下降到 0.74。造成这一差异的是满载标准航程：C 型的满载标准航程仅 12000 m，不到其空载航程 26000 m 的一半，而 A 型与 B 型的这一比值分别为 0.80 与 0.57。按式 (4)，满载与空载航程之差越大，等效航程随载荷衰减得越快，能耗也就越早触及返航余量约束。因此大载重机型虽然单次运力强，但其有效运力对航距更为敏感。

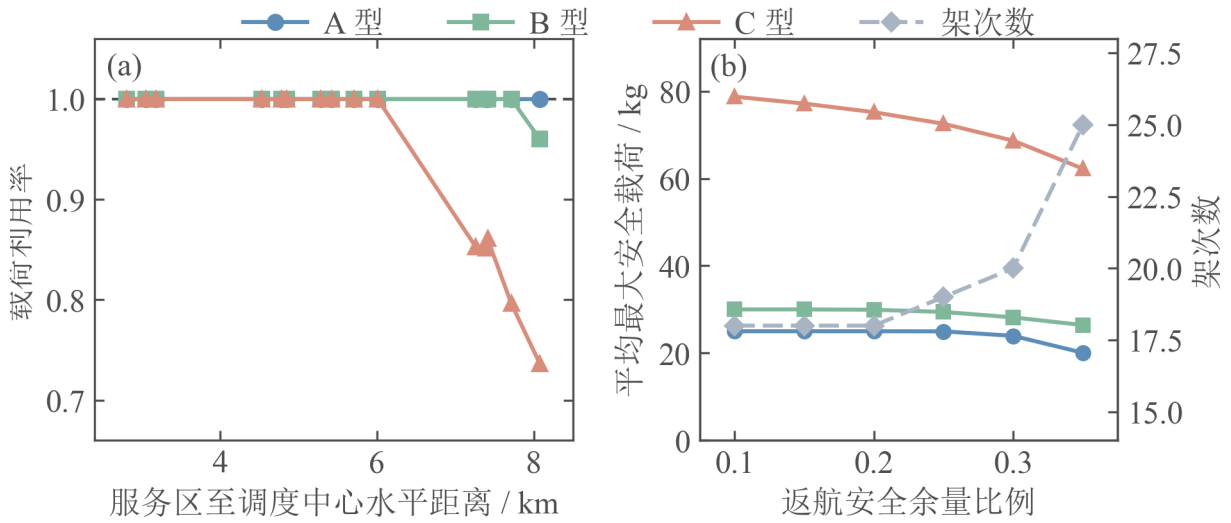


图 8　问题一的载荷能力：(a) 载荷利用率；(b) 余量灵敏度

4.7.2　货箱组批方案

字典序最优的组批方案共 18 个架次，总运输能耗 59.13 kWh，累计作业时间 32776.2 s。S001 需求量最大，15 个货箱合计 154 kg，由两个 C 型架次分批送达；S002 与 S003 各 8 个货箱，由一个 B 型架次与一个 C 型架次配合完成；其余 12 个服务区均由单个架次送达。逐架次的组批方案按提交模板的列结构列于表 2，货箱编号已省去与所在服务区重复的前缀。

表 2　问题一的组批方案，对应提交模板 Q1_ 单点组批

架次编号	服务区	机型	货箱编号列表	质量 (kg)	体积 (m^3)	往返 (s)	能耗 (kWh)	SOC(%)
Q1-001	S001	C	MED-01, MED-02, WAT-04, WAT-05, WAT-06, WAT-08, FOD-01, FOD-03	78	0.188	1626.2	2.9958	62.55
Q1-002	S001	C	WAT-01, WAT-02, WAT-03, WAT-07, FOD-02, HYG-01, HYG-02	76	0.206	1560.2	2.9176	63.53
Q1-003	S002	B	MED-01, WAT-03, FOD-02	25	0.067	1816.7	2.7134	32.16
Q1-004	S002	C	WAT-01, WAT-02, WAT-04, FOD-01, HYG-01	56	0.144	2041.8	5.7212	28.48
Q1-005	S003	B	MED-01, WAT-03, FOD-02	25	0.067	2080.5	2.7800	30.50
Q1-006	S003	C	WAT-01, WAT-02, WAT-04, FOD-01, HYG-01	56	0.144	2370.1	5.7651	27.94
Q1-007	S004	C	MED-01, WAT-01, WAT-02, WAT-03, FOD-01, HYG-01	59	0.156	2301.1	6.1530	23.09
Q1-008	S005	C	MED-01, WAT-01, WAT-02, WAT-03, FOD-01, HYG-01	59	0.156	1877.2	4.2222	47.22

架次编号	服务区	机型	货箱编号列表	质量 (kg)	体积 (m^3)	往返 (s)	能耗 (kWh)	SOC(%)
Q1-009	S006	C	MED-01, WAT-01, WAT-02, WAT-03, FOD-01, HYG-01	59	0.156	1561.6	2.5980	67.52
Q1-010	S007	C	MED-01, WAT-01, WAT-02, FOD-01, HYG-01	45	0.129	1907.6	3.4109	57.36
Q1-011	S008	C	MED-01, WAT-01, WAT-02, FOD-01, HYG-01	45	0.129	2199.7	5.8361	27.05
Q1-012	S009	B	MED-01, WAT-01, FOD-01	25	0.067	1558.4	2.0962	47.59
Q1-013	S010	B	MED-01, WAT-01, FOD-01	25	0.067	1555.4	1.8232	54.42
Q1-014	S011	B	MED-01, WAT-01, FOD-01	25	0.067	1188.4	1.0740	73.15
Q1-015	S012	B	MED-01, WAT-01, FOD-01	25	0.067	1922.7	2.7524	31.19
Q1-016	S013	B	MED-01, WAT-01, FOD-01	25	0.067	1510.5	1.8248	54.38
Q1-017	S014	B	MED-01, WAT-01, FOD-01	25	0.067	1869.8	2.1404	46.49
Q1-018	S015	B	MED-01, WAT-01, FOD-01	25	0.067	1828.3	2.3075	42.31

在字典序方案中 A 型无人机一次未被使用。对比三种机型的参数可见原因：B 型的最大载货质量 30 kg 高于 A 型的 25 kg，巡航速度 15 m/s 高于 A 型的 12 m/s，可用装载体积 0.073 m^3 也高于 A 型的 0.06 m^3，而两者的电池可用能量相差不大。因此在单点往返场景下，B 型在载重、速度、容积三个维度上均不劣于 A 型，A 型被完全压制。这一结论在问题二中会发生变化，因为多点多架次调度还要考虑实体机数量带来的并行能力，A 型的 4 架机届时成为稀缺资源。

4.7.3 三项指标的权衡关系

为考察指标之间的权衡，分别以总能耗最小和累计作业时间最小为单一目标重新求解，结果列于表 3。仅最小化能耗时总能耗降至 59.03 kWh，较字典序方案减少 0.10 kWh，但架次数增加到 19，作业时间增加 1663.9 s。这是因为把一个接近满载的架次拆成两个轻载架次可以利用等效航程在轻载段衰减平缓的特性，降低单位货物的能耗，代价是多付出一次工位准备与交接的固定开销。仅最小化作业时间得到的方案架次数与作业时间均与字典序方案相同，能耗多出 0.01 kWh，说明架次数与作业时间同向，固定开销在作业时间中占主导。

表 3 问题一三种优化目标下的结果对比

优化目标	架次数	总能耗/kWh	累计作业时间/s	机型分布
字典序，架次优先	18	59.13	32776.2	B 型 9，C 型 9
仅最小化总能耗	19	59.03	34440.1	B 型 11，C 型 8
仅最小化作业时间	18	59.15	32776.2	B 型 9，C 型 9

由此可以给出三项指标的优先关系：架次数与累计作业时间同向，应优先压缩；总能

耗与前两者冲突，幅度仅 0.17%。在应急场景下，多出动一个架次意味着多占用一架实体机与一组电池，其机会成本远高于 0.10 kWh 的电量，因此本文采用架次数优先、能耗次之、时间再次的字典序。问题一的最终答案为 18 个架次，总运输能耗 59.13 kWh，累计作业时间 32776.2 s。

4.8 返航安全余量的灵敏度分析

返航安全余量比例 ρ 是题目给定的安全参数，其取值直接决定可用能量的上限。把 ρ 从 0.10 提高到 0.45 重新求解载荷与组批，结果列于表 4，平均载荷与架次数的变化趋势见图 8(b)。

结果呈现三段特征。ρ 不超过 0.20 时组批方案完全不变：此时 A 型不受影响，B 型只在 S008 降到 28.80 kg，C 型虽在远距离服务区下降，但既有分组的装载量均未触及新的载荷上限。ρ 从 0.25 升至 0.35 时架次数由 19 增至 25、总能耗由 61.07 kWh 增至 75.61 kWh，增幅分别达 31.6% 与 23.8%，说明此区间内余量的提高被直接转化为出动规模的扩张。ρ 达到 0.40 时 S004 与 S008 无解：这两个服务区距离最远，而饮用水的单箱质量为 14 kg，任何一种机型都无法在满足余量的前提下完成一次往返。

表 4　返航安全余量变化对最大安全载荷与组批结果的影响

ρ	架次数	总能耗/kWh	作业时间/s	A 型均值/kg	B 型均值/kg	C 型均值/kg
0.10	18	59.13	32776.2	25.00	30.00	78.75
0.15	18	59.13	32776.2	25.00	30.00	77.16
0.20	18	59.13	32776.2	25.00	29.92	75.19
0.25	19	61.07	34565.8	24.97	29.42	72.54
0.30	20	67.23	36586.3	23.94	28.17	68.67
0.35	25	75.61	45383.6	20.07	26.42	62.26
0.40	无解	—	—	16.67	23.33	52.84

题目给定的 0.20 恰好位于方案不变区间的末端，兼顾安全与效率。若实际作业要求更高的余量，则必须为最远的两个服务区另行安排中途补给点或更换航程更长的机型，单纯提高余量会使运力急剧退化。

4.9 模型的检验

问题一的组批可以做三项独立检验。第一项是架次数的最优性。对每个服务区，所需架次不少于货箱总质量除以三种机型在该处最大安全载荷的最大值，也不少于货箱总体积除以三种机型装载体积的最大值，两者取大并向上取整即为该服务区的架次下界。15 个服务区的下界逐一与求解结果相同，合计 18，证明 18 个架次达到下界，架次数这一首要指标是全局最优的。

第二项是逐架次复核。用独立于求解器的程序对 18 个架次逐一复算：80 个货箱各出现且仅出现一次；每个架次的总质量不超过该机型在该服务区的最大安全载荷，总体积不超过装载体积，违反数为零；返航荷电状态最低 23.09%，全部高于 20% 的返航电量下限。

第三项是换一种算法重解整个问题。约束规划把装箱写成整数变量交给求解器分支定界，这里改用子集动态规划：把每个架次装哪些箱、用哪种机型枚举成货箱子集，对每个机型预先筛出质量与体积都允许的子集并算好能耗与时长，再用 dp 按货箱子集递推，状态记录装完这一组货箱所需的最少架次数、最小能耗与最短时间，按字典序比较。单个服务区最多 15 个货箱，枚举量在 3^{15} 量级，可以直接求到全局最优。两种算法逐服务区给出的

架次数、能耗与时间完全一致，合计都是 18 个架次、59.1317 kWh、32776.2 s。两条独立的求解路径得到同一结果，证明第一问的解是该口径下的全局最优解。求解结果通过全部检验。

4.10 本章小结

本章针对单点往返情形建立了最大安全载荷模型与货箱组批模型。前者把题目给出的等效航程关系与自定的能耗口径结合，证明了架次能耗关于载荷严格单调，从而把载荷上限的求解化为一维二分；后者把组批写成带机型选择的二维装箱问题，用约束规划按字典序精确求解。计算表明三种机型中大载重的 C 型对航距最敏感，而 A 型在单点往返场景下被 B 型完全压制。最终方案为 18 个架次、59.13 kWh、32776.2 s，且在返航余量不超过 0.20 时保持稳定。本章确立的航段几何、时间与能耗口径将被后续三问直接沿用。

五、 问题二模型的建立、分析与求解

5.1 问题分析

问题二在问题一的基础上放开了单点往返的限制，同时把实体装备与物资时限一并纳入，要求联合确定货箱组批、访问顺序、机型、执行无人机、共享电池与各架次开始时刻。

与问题一相比，本问引入了三处新的耦合关系，把路径问题与排程问题绑在一起。

其一是载荷沿途递减。架次从 O01 出发时携带全部货箱，每到一个服务区卸下一部分，后续航段的载荷随之减小。由式 (6) 知单位航程能耗随载荷下降，因此先访问需求量大的服务区可以使重载段更短，但这又可能与时限要求冲突，因为需求量大的服务区未必是时限最紧的。

其二是爬升的重复付出。题目规定每次投送后都要从 30 米作业高度重新爬升至下一航段的巡航海拔，而服务区海拔在 154 至 444.5 m 之间、沿线山体更高，使得每增加一个停靠点就要多付一次完整的爬升能耗。这与地面车辆路径问题中增加一个停靠点只延长一小段路程的情形有本质区别，是本题中多点架次不能无限串联的根本原因。

其三是装备的时间占用。实体无人机在架次期间被独占，共享电池在架次结束后还要按两阶段等效充电模型充电才能再次投入。A、B、C 三型的等效完全充电时间分别为 1800、2400、3000 s，而单个架次的时长多在 1000 至 2500 s 之间，因此电池而非机体往往是并行能力的瓶颈。

在建模策略上，若把派机、配电池与起飞时刻都写成独立的决策变量，约束数量将急剧膨胀，且大量约束只是在表达一个本可由规则确定的调度过程。本文改为只把架次序列作为解，其余交给一个确定性的可行性仿真器还原，这样既保证了任意解都满足资源约束，又把搜索空间压缩到了组批与顺序本身。求解上，该问题是带时间窗与资源周转的异构车辆路径问题 [6]，属 NP 难，80 个货箱的规模下精确方法难以求解，因此采用自适应大邻域搜索与模拟退火的混合算法，并与多种方法对比以说明选择的合理性。

问题二的求解思路如图 9 所示。

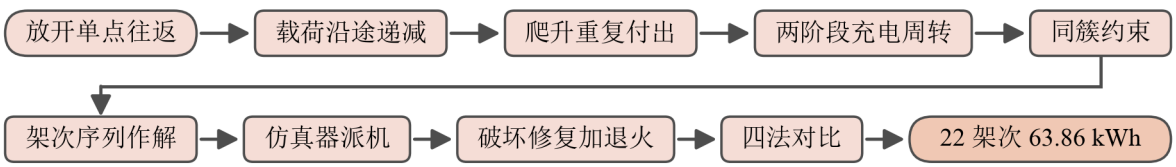


图 9 问题二的求解思路

第一行梳理放开单点往返后新增的三处耦合与为后续分组预留的同簇约束，第二行是求解链条：以架次序列为解，交给仿真器还原资源分配，再用破坏修复加退火接受的混合搜索寻优，并与其他三种方法对比。

5.2　能源周转的刻画

共享电池与中继能源组件均按两阶段等效充电模型周转。从荷电状态 σ 充至满电所需时间为

$$t_{\mathrm{chg}}(\sigma)=\begin{cases} T_{\mathrm{full}}\left[0.65\dfrac{0.90-\sigma}{0.90}+0.35\right], & 0\le\sigma<0.90,\\[2mm] T_{\mathrm{full}}\cdot 0.35\dfrac{1-\sigma}{0.10}, & 0.90\le\sigma\le 1. \end{cases} \tag{15}$$

该式刻画的是锂电池恒流恒压充电的时间分配：前 90% 电量在恒流阶段较快完成，最后 10% 在恒压阶段电流逐渐衰减因而耗时偏长 [7]。式 (15) 有一个值得注意的性质：即便架次只消耗了很少的电量，把电池充满仍需 $0.35T_{\mathrm{full}}$ 的时间。以 C 型为例，$T_{\mathrm{full}}=3000$ s，这意味着任何一次出动之后该电池至少要闲置 1050 s。因此在本题中浅放深充并不经济，把电池用得更充分反而有利于缩短总周转时间，这一点在后文的结果中可以得到印证。同一能源资源的任务占用与充电时段不得重叠，不同资源可并行充电。

电池的周转过程如图 10 所示。架次返回后换下的电池带着荷电状态 σ 进入充电柜，先经恒流段较快地充到 90%，再经恒压段慢慢充满，之后才能装上下一架次。图中三块电池的横向间距按式 (15) 的时间比例排布，恒压段只补 10% 的电量，却固定占去 $0.35T^{\mathrm{full}}$ 的时间，在这张图上与恒流段相差不大。这就是浅放深充并不经济的直观原因：放得越浅，恒流段越短，而恒压段一分都省不下来。

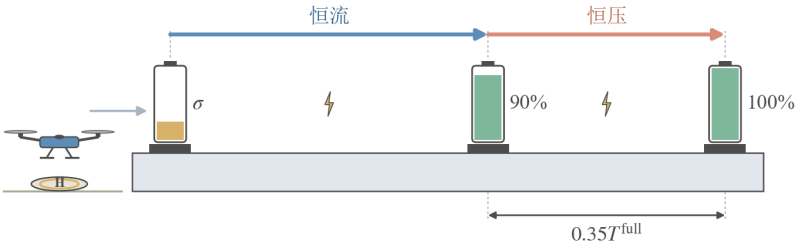


图 10　共享电池的两阶段充电周转

5.3　可行性仿真器

设解为有序的架次序列 $\Pi=\langle\pi_1,\pi_2,\ldots,\pi_P\rangle$，其中 $\pi_p=(g_p,\Sigma_p)$，g_p 为机型，$\Sigma_p=\langle(i_1,\mathcal{B}_{p1}),\ldots\rangle$ 为访问序列及各点投送的货箱集合。派机、配电池与起飞时刻由如下规则还原：架次 π_p 派给机型 g_p 中最早空闲的实体机 u、配给该机型中最早充满的电池 e，开始时刻取二者可用时刻的较晚者

$$s_p=\max\left\{a_u,\ c_e\right\}, \tag{16}$$

其中 a_u 为无人机 u 的可用时刻，c_e 为电池 e 的充满时刻。返回时刻 $r_p=s_p+T_p$，随后更新 $a_u\leftarrow r_p$，$c_e\leftarrow r_p+t_{\mathrm{chg}}(\sigma_p)$。

这一还原规则保证了任意架次序列都自动满足实体机与电池的数量约束和不重叠约束，把带资源约束的调度问题转化为对序列的搜索。需要说明的是，最早空闲优先的派发

规则并不保证对给定序列是最优的，但由于搜索算子会反复改变序列顺序，规则的次优性可以被搜索过程吸收。

仿真器的派发过程如图 11 所示。左侧是按顺序排队的架次卡片，每张卡片只写明机型与所载货箱；中间上方是机库，8 架实体机按 A、B、C 三型分行停放，每架机记着自己的可用时刻 a_u；中间下方是电池架，满电与充电中的电池混在一起，每块电池记着充满时刻 c_e。轮到某张卡片时，仿真器从对应机型中取最早空闲的机与最早充满的电池，两个时刻取较晚者即为起飞时刻 s_p。整个过程不需要任何人工排班，搜索算法只管调整卡片的内容与顺序。

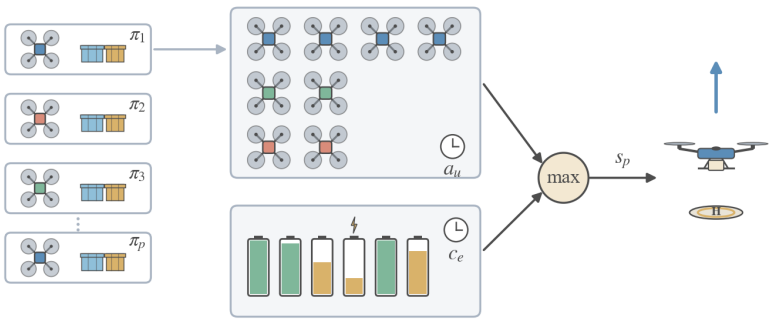


图 11　可行性仿真器的派机与配电池规则

多点架次的载荷递减与爬升重复如图 12 所示。

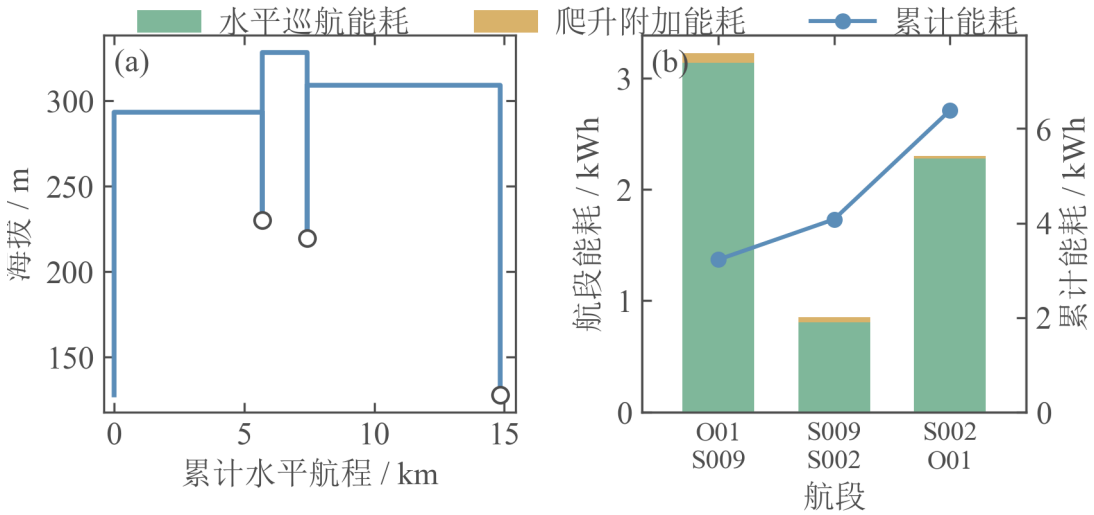


图 12　多点架次的剖面与能耗：(a) 海拔剖面；(b) 能耗构成

图中取最终方案里的两点架次 Q2-006 为例，它由 C 型机携带 70 kg 货箱，先到 S009 卸下 11 kg，再到 S002 卸完，然后返回 O01。图 12(a) 是沿访问顺序的海拔剖面，横轴只累计水平巡航距离，每到一个服务区先垂直下降到 30 米作业高度投送，再从同一位置垂直爬升至下一航段的巡航海拔，剖面因此呈现下探再回升的台阶；三段的爬升高度分别为 166、98、89 m，其中后两次都是投送之后重新付出的。图 12(b) 把各航段的能耗拆成水平巡航与爬升附加两部分：首段满载飞行 5.7 km，能耗 3.23 kWh 最高；第二段只有 1.7 km，载荷降到 59 kg，能耗 0.85 kWh；返程空载却飞 7.4 km，能耗回升到 2.30 kWh，整个架次耗电

6.38 kWh，返航荷电状态 20.2%，恰好满足返航余量要求。爬升附加能耗在三段中占 1.0% 至 5.6%，单看不大，但它随停靠点数量逐次累加，而且会压缩本已紧张的返航余量，这正是多点架次不能无限串联的原因，也解释了最终方案中多点架次最多只串联两个服务区。

为了在真实地形上核对这一航迹，本文把该架次的三维航迹写成带高程的折线，与数字高程模型一起载入 ArcScene 10.8.1，以 2.5 倍垂直夸张显示，结果如图 13 所示。视线方向与航迹大致垂直，可以清楚看到三处竖直的爬升与下降：左侧 O01 上空一段垂直爬升，中间 S009 上空先垂直下降投送、再原地垂直爬升，右侧 S002 上空同样如此；三段水平巡航分别位于各自航段的巡航海拔上，去程与返程两条水平线高度不同，因为二者经过的最高点不同。整条航迹全程高于地表，没有斜向穿越山体的航段。

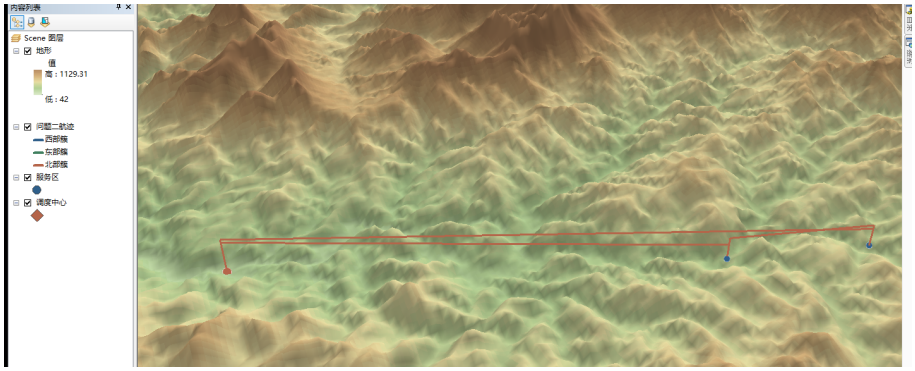


图 13　两点架次的三维航迹

5.4　协同调度模型的建立

决策变量的建立。待定的是货箱到架次的分配、每个架次的服务区访问顺序、所用机型、执行该架次的实体无人机、所配共享电池以及架次开始时刻。如前所述，这些量由架次序列 Π 连同式 (16) 的还原规则唯一确定，因此 Π 是唯一的自由变量。

约束条件的建立。货箱不可拆分且每箱只安排一次；每个架次的总质量不超过机型最大载货质量、总体积不超过可用装载体积；每个架次的总能耗满足式 (9) 的返航安全余量；首批保障货箱须在首批截止时间之前送达；医疗物资须在期望送达时间之前送达。

此外本文增设一条同簇约束：按平面坐标用 K 均值法把 15 个服务区聚成三个地理簇，西部簇为 S003、S006、S007、S011、S015，北部簇为 S002、S004、S005、S008、S009，东部簇为 S001、S010、S012、S013、S014，要求一个架次只能组合同簇的服务区。记服务区 i 所属的簇为 $\kappa(i)$，约束写作 $\kappa(i)=\kappa(j),\ \forall i,j\in\pi_p$。增设这条约束有三点考虑：一是跨簇串联意味着翻越簇间山脊，绕行与重复爬升都大，本来就很少出现在好解里；二是问题三的中继按地理区域值守，同簇架次只依赖一个悬停点，排程更简单；三是问题四要求同架次的服务区必须同组，同簇约束保证了可分组的结构稳定存在。这条约束的代价很小：不加限制时基准权重下 6 次运行的最好目标值为 106.55，加限制后五组权重共 18 次运行的最好目标值为 106.89，只差 0.3%。不加限制时各次运行得到的原子块数在 1 到 7 之间波动，有一次甚至把 15 个服务区全部并成一块，问题四将无从分组。

目标函数的建立。题目要求综合配送及时性、全部任务完成时间、运输能耗与架次数。记货箱 b 的送达时刻为 T_b，定义软性延误与硬性违约分别为

$$\Lambda=\sum_{b} w_b\,\frac{\left[T_b-\tau_b^{E}\right]^{+}}{1000},\tag{17}$$

$$\Gamma=\sum_{b\in\mathcal{F}}\left(w_b\,\frac{\left[T_b-\tau_b^{F}\right]^{+}}{1000}+\mathbb{I}\{T_b>\tau_b^{F}\}\right)+\sum_{b\in\mathcal{M}}\left(w_b\,\frac{\left[T_b-\tau_b^{E}\right]^{+}}{1000}+\mathbb{I}\{T_b>\tau_b^{E}\}\right),\tag{18}$$

其中 $\mathcal{F}$ 为首批保障货箱集合，$\mathcal{M}$ 为医疗物资集合，$[\cdot]^{+}$ 表示取正部。延误按应急优先系数 w_b 加权，体现不同物资的紧迫程度差异；硬性违约除按时长加权外还附加一个单位罚分，使得任何一次违约都比大量软性延误更不可接受。全部任务完成时间为 $C_{\mathrm{mk}}=\max_p r_p$。

综上所述，最终得到**异构无人机多点多架次协同调度模型**为

$$\min_{\Pi}\quad J=\lambda_\Gamma\Gamma+\lambda_\Lambda\Lambda+\lambda_C\frac{C_{\mathrm{mk}}}{1000}+\lambda_E\sum_p E_p^{T}+\lambda_N P$$

$$\text{s.t.}\quad\begin{cases}\bigcup_p\bigcup_i\mathcal{B}_{pi}=\mathcal{B},\quad \mathcal{B}_{pi}\cap\mathcal{B}_{p'i'}=\varnothing,\ (p,i)\neq(p',i'),\\ \sum_i\sum_{b\in\mathcal{B}_{pi}}m_b\le Q_{g_p},\qquad \sum_i\sum_{b\in\mathcal{B}_{pi}}v_b\le V_{g_p}, & \forall p,\\ E_p^{T}=\sum_{(i,j)\in\pi_p}E_{g_p ij}(q_{pij})\le\left(1-\rho_{g_p}\right)E_{g_p}^{\mathrm{use}}, & \forall p,\\ s_p=\max\{a_{u(p)},\,c_{e(p)}\},\quad r_p=s_p+T_p, & \forall p,\\ c_{e(p)}\leftarrow r_p+t_{\mathrm{chg}}\left(1-E_p^{T}/E_{g_p}^{\mathrm{use}}\right), & \forall p,\\ T_b\le\tau_b^{F}\ (b\in\mathcal{F}),\qquad T_b\le\tau_b^{E}\ (b\in\mathcal{M}),\\ \kappa(i)=\kappa(j), & \forall p,\ \forall i,j\in\pi_p.\end{cases}\tag{19}$$

权重取 $\lambda_\Gamma=50$、$\lambda_\Lambda=5$、$\lambda_C=3$、$\lambda_E=1$、$\lambda_N=1$，使硬性违约的惩罚远高于其余指标，从而优先保证时限。调整权重即可得到偏重不同指标的方案，用于后文的权衡分析。

5.5 模型的求解

本文采用自适应大邻域搜索与模拟退火的混合算法。大邻域搜索 [8][9] 通过成批移除再重新插入跳出局部最优，适合组批类问题；模拟退火 [10] 的接受准则则避免搜索过早收敛。求解步骤如下。

Step1：构造初始解。按服务区分组，组内货箱按首批截止时间、期望送达时间、优先系数排序，顺序装入当前架次，装不下则开新架次，机型随机选取。

Step2：以概率 0.3 执行局部扰动，包括交换两个架次的先后、改变某架次的机型、打乱某架次的访问顺序。

Step3：否则执行破坏与修复。破坏算子按等概率从随机移除、延误导向移除、整架次移除、同区移除四种中选择，移除 3 至 12 个货箱。延误导向移除优先取出当前延误最严重的货箱，同区移除取出同一服务区的全部货箱，两者分别针对时限与组批两类改进方向。

Step4：修复采用带噪声的贪心插入。对每个被移除的货箱枚举全部可行插入位置，包括并入已有停靠点、在同簇架次中新开停靠点、单独开设新架次，违反载重、体积、返航余量或同簇约束的位置直接跳过，在其余位置中取目标函数最小者；插入代价上叠加 5% 的随机扰动，避免每次修复都得到相同结构。

Step5：按 Metropolis 准则接受新解，温度按迭代数线性退火，初温取初始解目标值的 2%。

Step6：重复 Step2 至 Step5 共 40000 次，记录历史最优解。

Step7：对最优解用独立的核验程序复算，逐项检查每箱恰交付一次、载质量与体积、返航余量、资源不重叠与时限满足。

上述步骤的流程如图 14 所示。第一个判断框按概率在局部扰动与破坏修复之间选择算子，两路汇合后统一交给仿真器还原并评价；第二个判断框是退火接受准则，接受则更新当前解，不接受则直接进入计数；第三个判断框控制迭代次数，未满则沿左侧回边回到算子选择，满了便输出并核验。

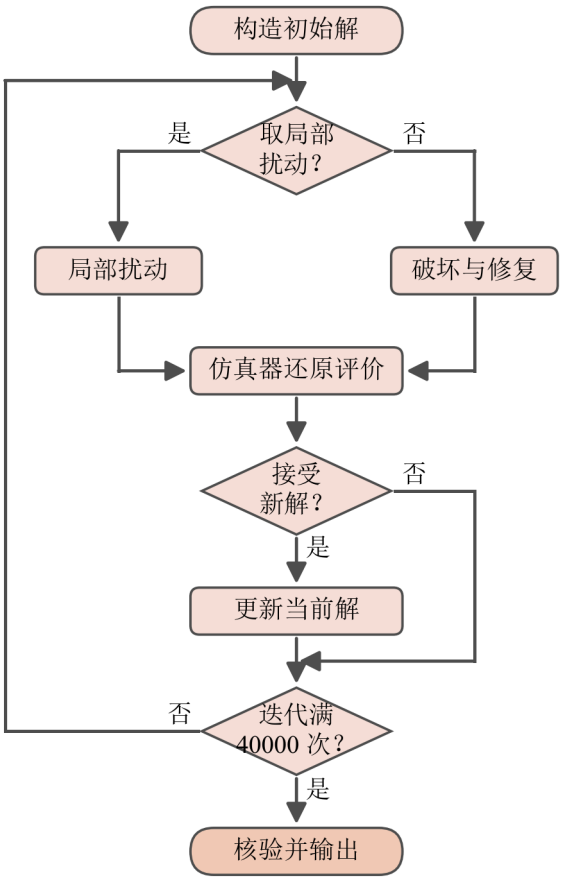


图 14　问题二混合算法的流程

破坏与修复是本算法改动组批结构的唯一途径，其作用如图 15 所示。

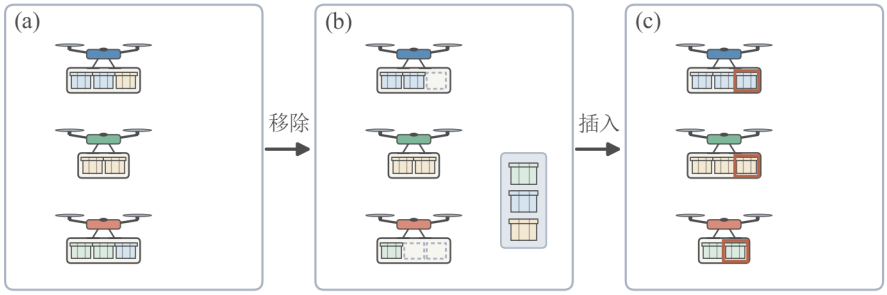


图 15　破坏与修复算子的作用：(a) 当前解；(b) 移除；(c) 插入

图中三架无人机代表当前解中的三个架次，货箱颜色代表所属服务区。图 15(a) 中蓝色架次混装了一个其他服务区的货箱，红色架次也是混装；图 15(b) 由破坏算子把三个货箱取出，舱内留下虚线空位，被取出的货箱暂存在右侧；图 15(c) 由修复算子逐个寻找代价最小的插入位置，三个货箱分别回到同色货箱所在的架次，加粗框为新插入的货箱。修复后

每个架次只服务一个服务区，访问点减少，重复爬升随之消失。局部扰动只能交换顺序或改机型，做不到这种跨架次的货箱迁移，这正是后文对比中两者差距的来源。

5.5.1 多算法对比

为说明所选算法的合理性，在同一评价函数与同一迭代次数 40000 下对比四种方法：分区贪心构造即 Step1 的初始解；仅局部扰动的模拟退火去掉了破坏与修复算子；只接受改进的大邻域搜索去掉了 Metropolis 接受准则；第四种为本文的混合算法。每种方法各用 6 个随机种子独立运行，所得方案统一按式 (19) 的基准权重重新评价，结果列于表 5。

表 5 问题二的多算法对比，各方法独立运行 6 次

算法	最好目标值	平均目标值	标准差	最好解架次	达到 22 架次	平均用时/s
分区贪心构造	838.7	2118.5	1278.1	28	0/6	0.02
模拟退火，仅局部扰动	141.9	156.9	20.1	33	0/6	3.4
大邻域搜索，只接受改进	111.3	113.3	2.1	22	3/6	150.8
大邻域搜索与模拟退火混合	109.5	116.0	4.3	22	2/6	769.2

分区贪心构造瞬间完成，但 6 次中每次都出现时限违约，架次数在 26 到 37 之间随机型抽取剧烈波动，平均目标值是混合算法的 18 倍，说明单靠构造规则既找不到跨服务区的组批机会，也照顾不到时限。仅用局部扰动的模拟退火消除了全部违约，但只能调换架次顺序与机型，无法改变组批本身，6 次都停在 26 至 33 个架次，最好目标值仍比混合算法高 29.6%。这两组对照说明破坏与修复算子是本问取得好解的关键。

后两种方法都保留了破坏与修复，差别只在接受准则。只接受改进的大邻域搜索平均目标值 113.3、标准差 2.1，6 次中 3 次达到 22 架次，稳定性优于混合算法，且用时只有其五分之一；但它的最好解目标值为 111.3，完成时间 7396 s。混合算法的平均表现略逊，最好解却更优，目标值 109.5，完成时间 7039 s，比前者再提前 357 s。原因在于退火接受准则允许暂时接受较差的组批，从而有机会越过局部最优进入更好的区域，代价是单次运行的波动变大、收敛变慢。

据此给出选择：本问的解只需求一次，关心的是最终方案的质量而非平均表现，因此采用混合算法。为进一步降低对随机种子的依赖，混合算法在五组不同权重下共运行 18 次，所得方案一律按基准权重重新评价，取目标值最小者为最终方案。结果目标值最小的是侧重能耗那一组的一次运行，目标值 106.89，比基准权重下的最好解 109.5 再低 2.4%，这说明换一组权重相当于换一个搜索方向，多组权重并行比同一权重多次运行更容易找到好解。若在救援现场需要反复重算，只接受改进的大邻域搜索是更合适的选择，它平均约 150 s 就能稳定给出 22 至 25 个架次的方案。

5.6 结果分析与资源可行性检验

5.6.1 调度方案

最终方案共 22 个架次，运输能耗 63.86 kWh，全部任务完成时间 7009.7 s。其中 7 个架次访问两个服务区，其余 15 个为单点往返；机型使用上 A 型 8 架次、B 型 8 架次、C 型 6 架次，与问题一中 A 型完全不被使用形成鲜明对照。原因在于本问受实体机数量制约，A 型有 4 架而 B、C 型各 2 架，要在约 7000 s 内完成全部投送必须动用全部 8 架实体机，A 型的并行能力此时成为稀缺资源。这说明机型优劣的判断必须放在具体的资源约束下进行，脱离机队构成谈单机性能会得出相反的结论。

最终方案的 22 条航线在地图上的分布如图 16 所示，底图是 ArcMap 中由高程色带叠加山体阴影生成的地形，三种颜色的折线分别是西部、北部、东部三个地理簇的架次，虚线框是各簇的范围。所有航线都从 O01 呈放射状发出，两点架次在簇内折返，没有一条航线跨越两个簇，这是同簇约束的直接体现；北部簇距 O01 最远，航线最长，也是多点架次最集中的区域，因为远距离服务区单点往返时回程空飞的代价最高，合并投送的收益最大。

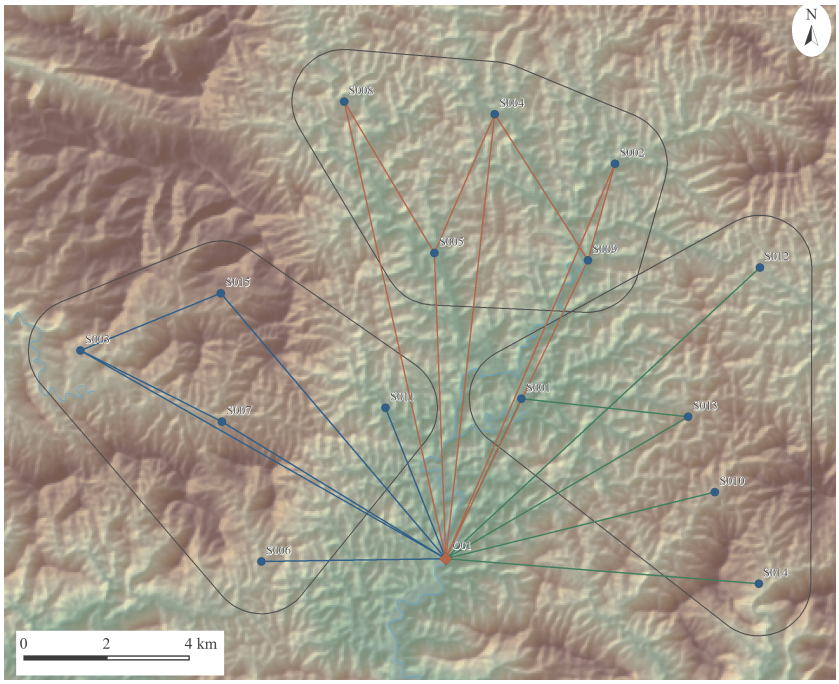


图 16　问题二最终方案的航线分布

把全部 22 条三维航迹载入 ArcScene 后的全景如图 17 所示，左侧内容列表中的三种颜色与图 16 一致，黄色圆点与黄色虚线是问题三将要用到的三个中继悬停点及中继机的飞行路线。从这个视角可以看到，每条航迹都由竖直的起降段与水平的巡航段组成，巡航高度随所经山脊的高低而变化。西部簇的航线要翻越西侧的高山，各架次最高巡航海拔的平均值为 458 m；北部簇虽然距离最远，航线却大体顺着河谷走，这一平均值只有 356 m，是三簇中最低的。距离远而爬升小，也是北部簇适合组织多点架次的原因之一，该簇 6 个架次中有 4 个是两点架次。

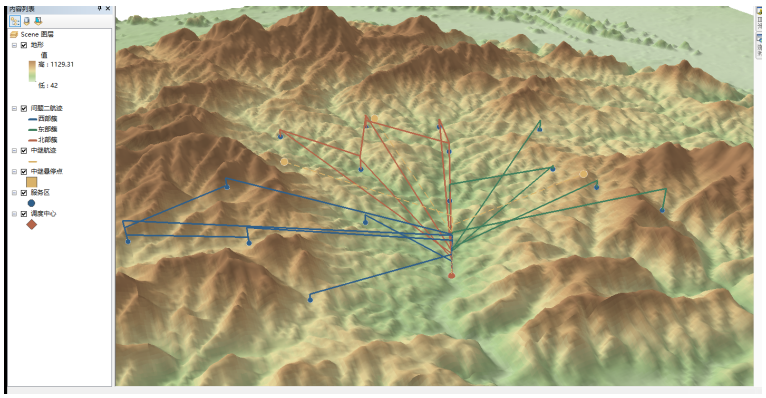


图 17　问题二全部架次的三维航迹

调度时序与送达情况如图 18 所示。图 18(a) 是按实体无人机分行的架次甘特图，同一

行内的架次几乎首尾相接，8 架机中只有 U01、U03、U05 各出现一次 78 至 774 s 的等待。这并不说明电池不紧张：每个机型的共享电池都多于实体机，一组电池在充电时另一组已经充满顶上，机体因此不必等待，而下文的核算表明电池峰值占用恰好等于库存，再少一组就会出现成片的等待。图 18(b) 是逐箱送达相对期望时限的富余时间，全部 80 个点都落在零线右侧，即时限要求全部满足。最靠近零线的是饮用水与医疗物资两类首批保障物资，最小富余都只有 3.9 min，而生活卫生用品的富余都在 2 h 以上，说明调度把余量集中让给了时限最紧的物资。

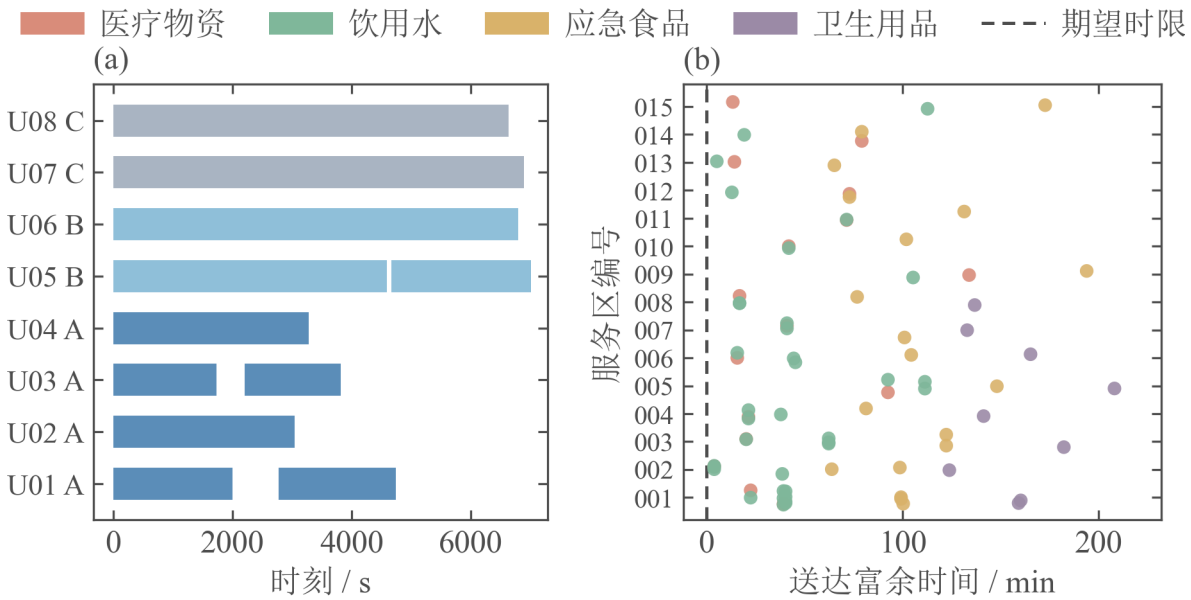


图 18　问题二的调度结果：(a) 架次甘特图；(b) 送达富余时间

运输架次安排按提交模板的列结构列于表 6，逐箱交付时刻列于表 7，为节省版面每行并排两条记录，服务区编号可由货箱编号前四位读出。

表 6　问题二的运输架次安排，对应提交模板 Q2_ 运输架次

架次编号	无人机	机型	电池	开始 (s)	访问服务区顺序	返回 (s)	能耗 (kWh)
Q2-001	U01	A	BA01	0.0	S002	2003.7	3.0187
Q2-002	U07	C	BC01	0.0	S001	1560.2	2.6386
Q2-003	U02	A	BA02	0.0	S007	1727.4	1.9709
Q2-004	U03	A	BA03	0.0	S007	1727.4	1.9054
Q2-005	U08	C	BC02	0.0	S001	1494.2	2.7204
Q2-006	U08	C	BC03	1494.2	S009→S002	3989.5	6.3837
Q2-007	U04	A	BA04	0.0	S006	1313.5	1.3571
Q2-008	U05	B	BB01	0.0	S006	1207.3	1.1382
Q2-009	U04	A	BA05	1313.5	S001→S013	3277.6	2.1563
Q2-010	U06	B	BB02	0.0	S010	1555.4	1.8232
Q2-011	U05	B	BB03	1207.3	S014	3077.0	2.1404
Q2-012	U06	B	BB04	1555.4	S012	3478.1	2.7524
Q2-013	U02	A	BA06	1727.4	S006	3040.9	1.3116
Q2-014	U05	B	BB01	3077.0	S005	4593.2	2.0861
Q2-015	U03	A	BA04	2205.5	S013	3817.6	2.0254

架次编号	无人机	机型	电池	开始 (s)	访问服务区顺序	返回 (s)	能耗 (kWh)
Q2-016	U06	B	BB02	3478.1	S009→S004	5607.6	3.0529
Q2-017	U01	A	BA03	2777.9	S015	4746.5	2.5577
Q2-018	U05	B	BB03	4671.2	S003→S015	7009.7	2.7928
Q2-019	U07	C	BC04	1560.2	S007→S003	4253.8	6.3923
Q2-020	U06	B	BB04	5607.6	S011	6796.0	1.0740
Q2-021	U08	C	BC02	3989.5	S005→S004	6637.3	6.1989
Q2-022	U07	C	BC01	4253.8	S005→S008	6894.4	6.3606

表 7　问题二的逐箱交付时刻，对应提交模板 Q2_ 逐箱交付，每行两条记录

货箱编号	架次编号	交付时刻 (s)	货箱编号	架次编号	交付时刻 (s)
S001-FOD-01	Q2-002	1248.3	S001-FOD-02	Q2-002	1248.3
S001-FOD-03	Q2-005	1182.3	S001-HYG-01	Q2-005	1182.3
S001-HYG-02	Q2-002	1248.3	S001-MED-01	Q2-002	1248.3
S001-MED-02	Q2-009	2252.1	S001-WAT-01	Q2-005	1182.3
S001-WAT-02	Q2-009	2252.1	S001-WAT-03	Q2-005	1182.3
S001-WAT-04	Q2-005	1182.3	S001-WAT-05	Q2-005	1182.3
S001-WAT-06	Q2-002	1248.3	S001-WAT-07	Q2-002	1248.3
S001-WAT-08	Q2-002	1248.3	S002-FOD-01	Q2-006	3369.0
S002-FOD-02	Q2-001	1283.8	S002-HYG-01	Q2-006	3369.0
S002-MED-01	Q2-006	3369.0	S002-WAT-01	Q2-006	3369.0
S002-WAT-02	Q2-006	3369.0	S002-WAT-03	Q2-006	3369.0
S002-WAT-04	Q2-001	1283.8	S003-FOD-01	Q2-019	3463.2
S003-FOD-02	Q2-019	3463.2	S003-HYG-01	Q2-019	3463.2
S003-MED-01	Q2-018	5989.4	S003-WAT-01	Q2-019	3463.2
S003-WAT-02	Q2-018	5989.4	S003-WAT-03	Q2-019	3463.2
S003-WAT-04	Q2-019	3463.2	S004-FOD-01	Q2-021	5919.7
S004-HYG-01	Q2-021	5919.7	S004-MED-01	Q2-021	5919.7
S004-WAT-01	Q2-016	4928.4	S004-WAT-02	Q2-021	5919.7
S004-WAT-03	Q2-021	5919.7	S005-FOD-01	Q2-022	5513.0
S005-HYG-01	Q2-022	5513.0	S005-MED-01	Q2-021	5248.7
S005-WAT-01	Q2-014	4117.9	S005-WAT-02	Q2-021	5248.7
S005-WAT-03	Q2-014	4117.9	S006-FOD-01	Q2-007	935.5
S006-HYG-01	Q2-008	882.4	S006-MED-01	Q2-013	2662.9
S006-WAT-01	Q2-013	2662.9	S006-WAT-02	Q2-007	935.5
S006-WAT-03	Q2-008	882.4	S007-FOD-01	Q2-003	1141.4
S007-HYG-01	Q2-019	2823.9	S007-MED-01	Q2-004	1141.4
S007-WAT-01	Q2-004	1141.4	S007-WAT-02	Q2-003	1141.4
S008-FOD-01	Q2-022	6192.4	S008-HYG-01	Q2-022	6192.4
S008-MED-01	Q2-022	6192.4	S008-WAT-01	Q2-022	6192.4
S008-WAT-02	Q2-022	6192.4	S009-FOD-01	Q2-006	2764.3
S009-MED-01	Q2-006	2764.3	S009-WAT-01	Q2-016	4478.9
S010-FOD-01	Q2-010	1085.2	S010-MED-01	Q2-010	1085.2
S010-WAT-01	Q2-010	1085.2	S011-FOD-01	Q2-020	6514.0
S011-MED-01	Q2-020	6514.0	S011-WAT-01	Q2-020	6514.0
S012-FOD-01	Q2-012	2826.7	S012-MED-01	Q2-012	2826.7
S012-WAT-01	Q2-012	2826.7	S013-FOD-01	Q2-015	3289.1

货箱编号	架次编号	交付时刻 (s)	货箱编号	架次编号	交付时刻 (s)
S013-MED-01	Q2-009	2749.1	S013-WAT-01	Q2-015	3289.1
S014-FOD-01	Q2-011	2449.7	S014-MED-01	Q2-011	2449.7
S014-WAT-01	Q2-011	2449.7	S015-FOD-01	Q2-017	4035.6
S015-MED-01	Q2-018	6399.0	S015-WAT-01	Q2-017	4035.6

5.6.2 资源可行性检验

对最优方案逐项复核资源可行性，结果如下。8 架实体无人机的任一时刻占用数分别为 A 型不超过 4 架、B 型不超过 2 架、C 型不超过 2 架，与库存完全一致；共享电池按机型分池核算，A 型峰值占用 6 组、B 型 4 组、C 型 4 组，均等于库存上限，说明电池确实是本问的紧约束。每组电池的相邻两次任务之间均留出了按式 (15) 计算的充电时长，无重叠。全部 22 个架次的能耗均不超过 $(1-\rho_g)E_g^{\mathrm{use}}$，返航荷电状态最低为 20.1%，紧贴 20% 的返航电量下限，说明能耗较低的方案是以用尽单架次电量为代价换来的。80 个货箱中每箱恰被安排一次，首批截止与期望送达要求全部满足，最晚一箱在 6514.0 s 送达，平均送达时刻 3248.8 s。每个架次访问的服务区都属同一地理簇。方案通过全部可行性检验。

逐架无人机与逐组电池的使用情况列于表 8：8 架实体机全部投入，A 型各飞 2 个架次、B 型各飞 4 个、C 型各飞 3 个；14 组共享电池也全部周转过，A 型池中周转最多的两组各承担 2 个架次，B 型池四组各 2 个，C 型池两组各 2 个。

表 8　问题二的无人机与电池使用情况

资源	编号	机型	次数	首次投入 (s)	末次结束 (s)	累计时长 (s)
无人机	U01	A	2	0.0	4746.5	3972.3
无人机	U02	A	2	0.0	3040.9	3040.9
无人机	U03	A	2	0.0	3817.6	3339.5
无人机	U04	A	2	0.0	3277.6	3277.6
无人机	U05	B	4	0.0	7009.7	6931.7
无人机	U06	B	4	0.0	6796.0	6796.0
无人机	U07	C	3	0.0	6894.4	6894.4
无人机	U08	C	3	0.0	6637.3	6637.3
电池	BA01	A	1	0.0	2003.7	2003.7
电池	BA02	A	1	0.0	1727.4	1727.4
电池	BA03	A	2	0.0	4746.5	3696.0
电池	BA04	A	2	0.0	3817.6	2925.6
电池	BA05	A	1	1313.5	3277.6	1964.1
电池	BA06	A	1	1727.4	3040.9	1313.5
电池	BB01	B	2	0.0	4593.2	2723.5
电池	BB02	B	2	0.0	5607.6	3684.9
电池	BB03	B	2	1207.3	7009.7	4208.2
电池	BB04	B	2	1555.4	6796.0	3111.1
电池	BC01	C	2	0.0	6894.4	4200.8
电池	BC02	C	2	0.0	6637.3	4142.0
电池	BC03	C	1	1494.2	3989.5	2495.3
电池	BC04	C	1	1560.2	4253.8	2693.6

电池峰值占用等于库存上限这一事实提示，电池是本问的紧约束：现有机体几乎没有

空闲，是因为电池刚好够用；一旦电池减少，最先出现的就是机体停在工位上等电池。资源扩充时应当先保证电池与机体的配比，而不是单纯增加机体。

5.6.3 四项指标的权衡关系

改变式 (19) 中的权重分别侧重不同指标，每组取其运行中的最好解，得到表 9 的对照。由于五组方案的时限违约都为零，配送及时性改用 80 个货箱的平均送达时刻衡量，数值越小越及时；最后一列是按基准权重重新评价的目标值。

表 9　问题二四项指标的权衡

侧重指标	架次数	能耗/kWh	完成时间/s	平均送达/s	多点架次	基准目标值
运输能耗，作为最终方案	22	63.86	7010	3248.8	7	106.89
全部任务完成时间	22	66.31	6448	3115.2	7	107.65
均衡	22	66.36	7039	3183.2	9	109.48
配送及时性	22	69.02	7376	3278.1	11	113.14
架次数	20	65.07	8758	3585.0	8	111.35

表 9 显示本问真正的冲突发生在能耗与完成时间之间。侧重完成时间的方案把完成时间压到 6448 s，比最终方案短 8.0%，平均送达时刻也最早，代价是能耗多出 2.45 kWh，即 3.8%；它多耗的电来自更满的装载与更分散的起飞时机，换来的是全部 8 架机更早收工。架次数与完成时间的冲突同样明显：把架次数从 22 压到 20 时完成时间延长到 8758 s，比最终方案长 24.9%，平均送达时刻推迟 10.3%，因为减少架次意味着每架次装得更满、飞得更久，且更少的架次填不满 8 架实体机的并行能力。

值得注意的是，侧重配送及时性的那一组并没有得到最及时的方案，它的平均送达时刻反而比均衡方案晚。原因在于各组方案的时限违约都已为零，加大延误权重在可行区域内没有作用对象，只会压低其余各项在目标函数中的相对分量，使搜索失去方向。使物资更早送达的主要手段是压缩完成时间，因为所有架次整体前移，货箱自然更早送到。在时限已全部满足的前提下，配送及时性与完成时间同向，与能耗和架次数反向。

本问的最终答案取基准目标值最小的方案：22 个架次，运输能耗 63.86 kWh，全部任务完成时间 7009.7 s，80 个货箱的时限要求全部满足。若救援指挥更看重尽早收工，可改用侧重完成时间的方案，以多用 3.8% 的电量换取提前 562 s 完成。

5.7 本章小结

本章把单点往返放开为多点多架次，建立了异构无人机多点多架次协同调度模型。建模上的关键处理是只以架次序列作为解，把派机、配电池与起飞时刻交给确定性仿真器还原，使任意解自动满足资源约束，从而把带资源约束的调度问题化为对序列的搜索。求解上用大邻域搜索与模拟退火的混合算法，四法对比表明破坏与修复算子是取得好解的关键，退火接受准则能把最好解再推低，代价是单次运行更不稳定，因此用五组权重并行运行来弥补。最终方案为 22 个架次、63.86 kWh、7009.7 s，时限零违约。检验发现三种机型的电池峰值占用都达到库存上限，电池是本问的紧约束，为下一问的资源安排提供依据。

六、 问题三模型的建立、分析与求解

6.1 问题分析

问题三在问题二的基础上增加了连续通信这一硬约束，并引入中继无人机作为新的调度对象，要求在保证运输无人机全程处于保障之下的前提下联合安排运输与中继。

本问的分析需要先回答一个前置问题：通信究竟在多大程度上构成约束。按题目给定的收发参数计算双向链路预算，取两个方向最大允许损耗的较小者作为门限，再由自由空间损耗公式反解出视距与遮挡两种情形下的最远通视距离，可以发现 15 个服务区全部位于直连的视距作用范围之内，但一旦视线被山体遮挡，可用距离骤降至不足 4 km，多数服务区随即超出。也就是说，本场景中链路是否可用几乎完全由地形是否遮挡视线决定，通信问题因而转化为数字高程模型上的视线判定问题。逐点判定的结果证实了这一判断：15 个服务区中仅 3 个可与固定网关直连。

确认中继是刚需之后，第二个问题是中继悬停点怎么选 [11]。中继必须同时满足两个条件：与固定网关之间回传链路可用，与被保障的运输无人机之间接入链路可用。前者只与悬停点本身有关，后者与运输无人机的瞬时位置有关，因此选址必须面向整条航迹而不是面向服务区。本文在研究区上按格网枚举候选点，先用回传视距筛掉不可用者，再以航段为单位做集合覆盖。这里特别要求被选中的悬停点能整段覆盖某个航段的全部失联采样，而不是逐采样点分别指派最近的可用点。前期试算表明，若允许逐点指派，一个架次会在飞行途中频繁切换保障者，进而要求多架中继同时在空，两架中继根本排不开；改为整段覆盖后，一个架次在一段航程内只依赖一个悬停点，中继需求量大幅下降。

第三个问题是联合排程。中继的服务窗口与运输架次的起飞时刻互相牵制：运输要等中继到位并完成建链才能进入失联段，中继的续航又限制了单次值守时长，而中继机与能源组件的数量同样有限。本文把中继班次表与运输架次序列放进同一个解，用同一套大邻域搜索同时搜索两者，并以逐 5 秒采样的无保障时长作为惩罚项，最终方案由一个独立的核验程序重新采样复算。

问题三的求解思路如图 19 所示。第一行回答通信在多大程度上构成约束，从链路预算一路推到只有三个服务区可直连；第二行回答中继放在哪里、何时值守，先选址再把班次放进解里与运输架次联合搜索。

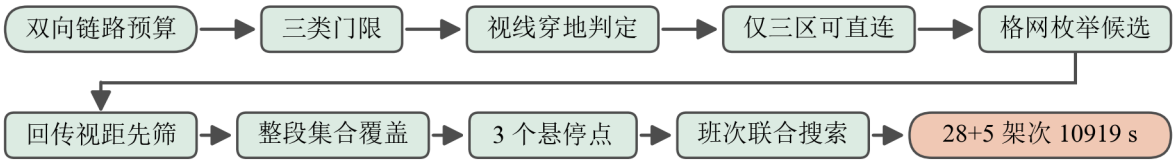


图 19 问题三的求解思路

6.2 通信链路预算与遮挡判定

有效接收门限由接收端 b 的接收灵敏度与衰落裕量确定

$$P_{\text{th}}^{b} = P_{\text{sens}}^{b} + M^{b} = -98 + 8 = -90 \text{ dBm}. \tag{20}$$

对通信方向 $a \to b$，最大允许总传播损耗为

$$L_{\max}^{a\to b} = P_{t}^{a} + G_{t}^{a} + G_{r}^{b} - L_{\text{sys}} - P_{\text{th}}^{b}. \tag{21}$$

由于指令下行与状态回传都必须保障，主体之间按双向链路判定，门限取两个方向的较小者

$$L_{\max}^{a\leftrightarrow b}=\min\left\{L_{\max}^{a\to b},\,L_{\max}^{b\to a}\right\}. \tag{22}$$

代入题目给定的收发参数，得到三类链路的门限：运输无人机与固定网关 G01 的直连链路为 122 dB，运输无人机与中继接入端的链路为 116 dB，中继回传端与 G01 的链路为 126 dB。

传播损耗按自由空间公式计算并在遮挡时附加固定损耗 [12]，自由空间传播损耗与总传播损耗分别为

$$L_{\mathrm{FSPL},ij}(t)=32.45+20\log_{10}f+20\log_{10}D_{ij}(t), \tag{23}$$

$$L_{\mathrm{path},ij}(t)=L_{\mathrm{FSPL},ij}(t)+L_{\mathrm{obs}}\,b_{ij}(t), \tag{24}$$

其中载波频率 f 以 MHz 计、三维直线距离 D_{ij} 以 km 计，$L_{\mathrm{obs}}=10$ dB。令 $L_{\mathrm{path},ij}(t)\le L_{\max}^{i\leftrightarrow j}$ 并反解 D_{ij}，得到三类链路在视距与遮挡两种情形下的最远作用距离，列于表 10。

表 10　三类链路的门限与最远作用距离

链路	双向门限/dB	视距最远/km	遮挡最远/km
运输无人机与 G01 直连	122	12.51	3.96
运输无人机与中继接入	116	6.27	1.98
中继与 G01 回传	126	19.83	6.27

15 个服务区与调度中心的水平距离在 2.81 至 8.08 km 之间，全部位于直连视距作用范围 12.51 km 以内，但遮挡后可用距离降至 3.96 km，多数服务区随即超出。

地形遮挡按视线穿地判定。对通信端点 $i=(x_i,y_i,z_i)$ 与 $j=(x_j,y_j,z_j)$，沿其水平投影直线以 5 米步长采样，若存在某个采样点处视线高程低于地面高程，即

$$b_{ij}(t)=\mathbb{I}\left\{\exists\, s\in(0,d_{ij}):\ z_i+\frac{s}{d_{ij}}(z_j-z_i)<Z(s)\right\}, \tag{25}$$

则判定该视线被遮挡。链路可用性为

$$A_{ij}(t)=\mathbb{I}\left\{L_{\mathrm{path},ij}(t)\le L_{\max}^{i\leftrightarrow j}\right\}. \tag{26}$$

固定网关 G01 设于 O01，其通信端点海拔为 O01 地面海拔加天线离地高度 20 米。按式 (25) 逐点判定 15 个服务区作业点与 G01 之间的视线，结果如图 20 所示。

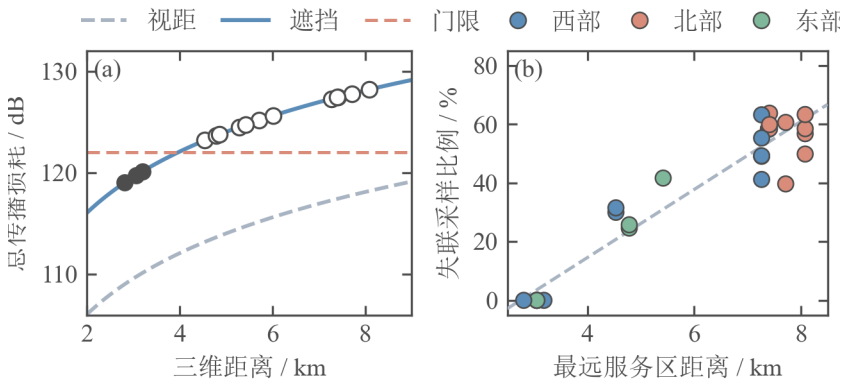


图 20　直连链路判定：(a) 传播损耗与门限；(b) 失联比例与航距

图 20(a) 把 15 个作业点的总传播损耗画在三维距离上，灰色虚线与蓝色实线分别是视距与加 10 dB 遮挡损耗后的损耗曲线，红色虚线是 122 dB 的直连门限。15 个点全部落在蓝色曲线上，说明它们到网关的视线无一例外都被地形遮挡；曲线在 3.96 km 处穿过门限，门限以下只剩 S001、S006、S011 三个实心点，其余十二个空心点必须依靠中继。可见被遮挡并不等于不可用，这三个服务区只是距离足够近，这正是式 (26) 与式 (25) 必须分开判定的原因。图 20(b) 把视角扩展到整条航迹：对定稿的 28 个运输架次逐 5 秒采样，统计离地阶段直连不可用的采样比例并画在该架次最远服务区的距离上，两者相关系数 0.96，每远 1 km 失联比例约高 11.6 个百分点；仅访问 S001、S006 或 S011 的 7 个架次全程可直连。

遮挡如此普遍，可从图 1 的地形看出：服务区多位于支谷末端，与调度中心之间隔着一道或数道山脊，而网关天线离地仅 20 米，视线几乎沿谷底延伸，稍有起伏即被截断。

为了检验上述判定不依赖于本文自编的视线程序，本文另用 ArcGIS 10.8.1 做了两项独立复核。其一是可视域分析：把数字高程模型投影到 UTM 第 49 带，以 G01 为观察点、观察点离地 20 米、目标离地 30 米求网关的通视范围，再按表 10 叠加视距 12.51 km 与遮挡 3.96 km 两档距离得到直连可用区域，在作业点上提取栅格值后与本文的判定逐个对照，15 个服务区全部一致。

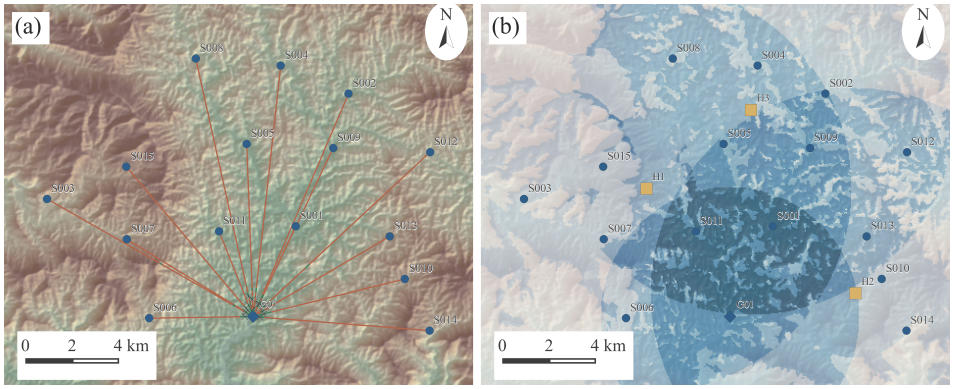


图 21　ArcGIS 链路分析：(a) 三维通视线；(b) 可用链路条数

其二是三维通视线分析，结果如图 21(a) 所示：用 3D Analyst 的通视线工具从 G01 天线向 15 个作业点各拉一条三维视线，绿色是能看通的部分，红色是被挡住的部分，叉号是每条视线遇到的第一个障碍点。15 条视线全部被挡，障碍点都紧邻网关，离 G01 只有 205 至 639 m，中位数 329 m，每条视线能看通的部分只占全长的 6% 至 18%。遮挡并非来自远处的高山，而是网关所在河谷两侧的近坡：天线离地仅 20 米，视线一出谷口就被挡住，改善直连的关键在于把网关抬高或移出谷底。

6.3　中继悬停点选址模型

记中继悬停候选点集合为 $\mathcal{H}$。候选点在研究区上按 400 米格网生成，每个平面位置取离地 100、200、300 米三个高度，并要求位于数字高程模型覆盖范围之内、离地高度不超过 300 米的规定上限。

候选点先按回传链路可用性筛选，保留满足

$$A_{h,G01} = 1, \qquad h \in \mathcal{H} \tag{27}$$

的点。筛选后剩余 1612 个候选点。

设运输航段的集合为 $\mathcal{S}$，航段 ℓ 上按 5 秒步长采样得到位置序列 $\{p_{\ell,1},\dots,p_{\ell,n_\ell}\}$，其中直连不可用的采样记为失联采样集合 $\mathcal{U}_\ell$。定义候选点 h 对航段 ℓ 的整段覆盖指示

$$c_{h\ell}=\prod_{p\in\mathcal{U}_\ell}A_{h,p}, \tag{28}$$

即 h 必须与该航段的每一个失联采样点都保持接入链路可用，才视为覆盖该航段。

整段覆盖与逐点覆盖的区别如图 22 所示。一条航段从 O01 飞向 S_i，沿途的采样点中两头可直连，中间一段失联。候选点 h_1 的接入范围只盖住失联段的前一部分，若允许逐点指派，它可以先保障前半段、再由别的悬停点接手后半段，一个架次就要在途中切换保障者；候选点 h_2 的接入范围把整段失联采样全部盖住，按式 (28) 只有 h_2 算作覆盖该航段。选址时只认 h_2 这类候选点，一个架次在一段航程内只依赖一个悬停点，中继的值守安排因此简单得多。

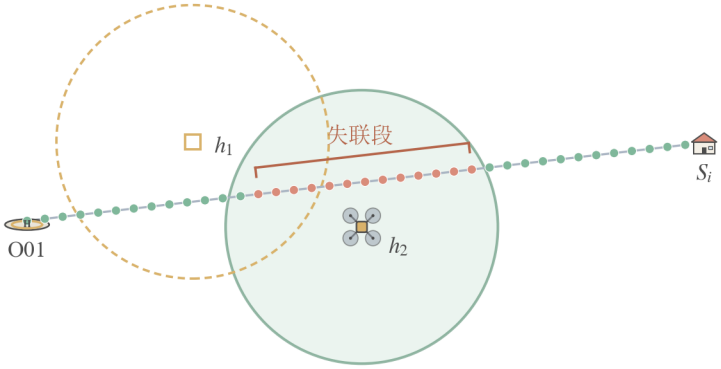


图 22　按航段整段覆盖的选址判据

引入 $y_h\in\{0,1\}$ 表示候选点 h 被选中，则悬停点选址是一个集合覆盖问题

$$\begin{aligned}\min\quad & \sum_{h\in\mathcal{H}}y_h\\ \text{s.t.}\quad & \begin{cases}\sum_{h\in\mathcal{H}}c_{h\ell}\,y_h\ge 1, & \forall\ell\in\mathcal{S},\ \mathcal{U}_\ell\neq\varnothing,\\ y_h\in\{0,1\}, & \forall h\in\mathcal{H}.\end{cases}\end{aligned} \tag{29}$$

该问题用按航段权重的贪心算法 [13] 求解，权重上对起降航段与交接悬停给予更高取值，因为这两类时段的持续时间最长。计算得到需要中继的项目共 258 个，全部可被某个候选点整段覆盖，贪心选出 3 个悬停点即实现完全覆盖，三点均取离地 300 米，分别位于研究区的西部、东部与北部。进一步检验发现每个悬停点恰好对应一个地理簇，说明服务区的空间聚集结构直接决定了中继的部署格局。

选出的中继与网关、服务区之间的链路在三维空间中的样子如图 23 所示。本文把三类链路写成带高程的三维线段，与数字高程模型一起载入 ArcScene，以 2.5 倍垂直夸张显示：蓝色是网关天线到三个悬停点的回传链路，绿色是悬停点到本簇需要中继的服务区的接入链路，红色是网关到 S001、S006、S011 的直连链路。三条红线在网关附近就断续地钻入山体，与图 21(a) 中障碍点紧挨网关的结论一致，它们之所以仍然可用，只是因为距离够短；而悬停点抬到山顶上方 300 米后，回传与接入两跳大多从山体上方越过，蓝线与绿线基本完整地露在地形之上。中继离地越高，两跳越容易同时越过山脊，这也是贪心选址最终把三个点都取在 300 米上限的原因。

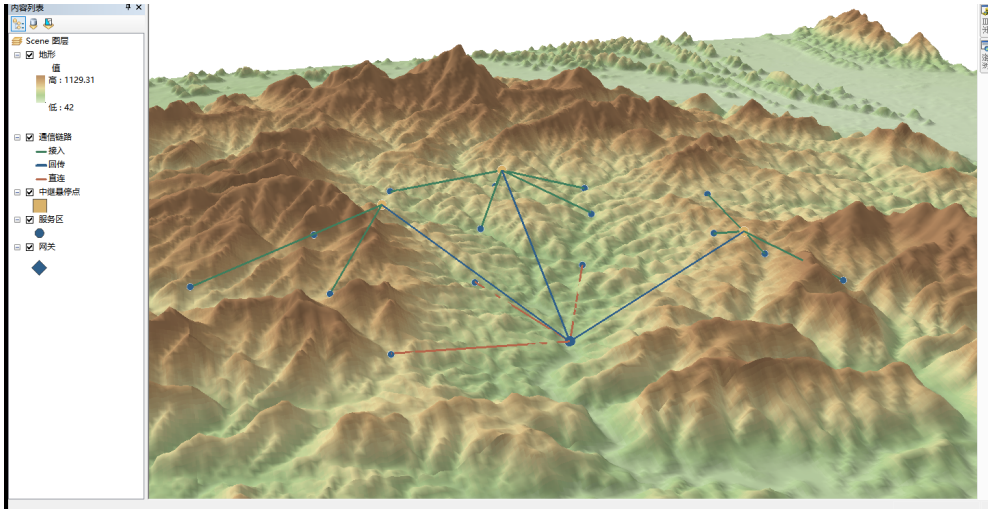


图 23　三类链路的三维视图

选址完成后各处可用链路的多少见图 21(b)。该图由 ArcGIS 可视域分析叠加得到：对网关与三个悬停点分别求通视范围，再按各自链路的视距与遮挡两档距离求出可用区域，逐像元累加可用链路的条数，颜色越深可用的链路越多，最浅的是没有任何可用链路的地方。网关附近 3.96 km 以内与三个悬停点的覆盖交叠处颜色最深，最多有 4 条链路可用；分析范围内约 48% 的区域没有可用链路。这张图以离地 30 米为目标高度，航线在更高的巡航海拔上通视条件更好，全程是否有保障以后文逐 5 秒的核验为准。15 个服务区作业点都至少有 1 条可用链路，其中 8 个只有 1 条，这些服务区一旦对应的悬停点缺席便会失联，这正是联合调度必须保证中继按时到位的原因。

6.4　运输与中继联合调度模型的建立

决策变量的建立。在问题二的架次序列 Π 之外，新增中继班次表

$$\mathcal{K} = \langle (h_1, \Delta_1), \ldots, (h_K, \Delta_K) \rangle,$$

其中 h_k 是第 k 个班次所用的悬停点，Δ_k 是其服务时长。中继机与能源组件的分配同样由确定性规则还原：班次按顺序派给最早空闲的中继机与最早充满的能源组件。

中继架次的时间与能耗。中继由 O01 出发，飞抵悬停点并完成建链后开始服务，服务结束后返回。记悬停点 h 的往返飞行时间为 t_h^{out} 与 t_h^{back}、往返能耗为 E_h^{out} 与 E_h^{back}。飞行时间沿用式 (5)；能耗按题目附录二的规定，水平巡航部分取巡航功率乘巡航时间，爬升附加部分沿用式 (7) 的关系且质量取计划起飞总质量 m_R，即

$$E_h^{\text{out}} = \frac{P^{\text{cru}}}{3600} \cdot \frac{d_{0h}}{v_R^c} + \frac{m_R\, g_0\, h_{0h}^{+}}{3.6 \times 10^6\, \eta_R^{\uparrow}}, \qquad E_h^{\text{back}} = \frac{P^{\text{cru}}}{3600} \cdot \frac{d_{h0}}{v_R^c} + \frac{m_R\, g_0\, h_{h0}^{+}}{3.6 \times 10^6\, \eta_R^{\uparrow}}, \tag{30}$$

其中 $P^{\text{cru}} = 1.15\,\text{kW}$，$v_R^c = 15\,\text{m/s}$，$m_R = 23.5\,\text{kg}$，$\eta_R^{\uparrow} = 0.72$。中继往返的航段同样按先垂直爬升、再水平巡航、后垂直下降三段计算，去程终点为悬停点。班次 k 的建链完成时刻与能耗为

$$t_k^{\text{link}} = s_k + t^{\text{prep}} + t_{h_k}^{\text{out}} + t^{\text{link}}, \tag{31}$$

$$E_k = E_{h_k}^{\text{out}} + E_{h_k}^{\text{back}} + \left(P^{\text{hov}} + P^{\text{comm}}\right) \frac{\Delta_k + t^{\text{link}}}{3600}. \tag{32}$$

其中 $t^{\text{prep}}=180$ s 为工位固定准备时间，$t^{\text{link}}=30$ s 为建链时间。班次 k 的返回时刻为 $t_k^{\text{ret}}=t_k^{\text{link}}+\Delta_k+t_{h_k}^{\text{back}}$，它占用中继机的区间取 $[s_k,\ t_k^{\text{ret}}+t^{\text{turn}}]$，$t^{\text{turn}}=300$ s 是题目给出的架次周转时间，中继机返回后须经过周转才能承接下一个班次。由式 (32) 与返航电量下限可反解出单个班次的最长服务时长

$$\Delta_k \le \Delta_h^{\max}=\frac{(1-\rho_R)E_R^{\text{use}}-E_h^{\text{out}}-E_h^{\text{back}}}{\left(P^{\text{hov}}+P^{\text{comm}}\right)/3600}-t^{\text{link}}. \tag{33}$$

代入参数得三个悬停点的最长服务时长在 7330 至 7648 s 之间，往返飞行本身只消耗 0.21 至 0.31 kWh，续航的绝大部分用于悬停值守。

连续通信约束。记运输架次 p 在悬停点 h 上的失联需求时段为 $\left[a_{ph}^{(l)},b_{ph}^{(l)}\right]$，这些时段由式 (28) 的覆盖关系与架次的时间剖面共同确定。连续通信要求每个需求时段都被同一悬停点的某个在役班次完整包含

$$\forall p,h,l,\ \exists k:\ h_k=h,\quad t_k^{\text{link}}\le s_p+a_{ph}^{(l)}\ \wedge\ s_p+b_{ph}^{(l)}\le t_k^{\text{link}}+\Delta_k. \tag{34}$$

若某个需求时段无法被包含，则把其时长计入无保障时长 Ξ 作为惩罚。

综上所述，最终得到**运输与中继联合调度模型**为

$$\min_{\Pi,\mathcal{K}}\quad J=\lambda_\Gamma\Gamma+\lambda_\Lambda\Lambda+\lambda_C\frac{C_{\text{mk}}}{1000}+\lambda_E\left(\sum_p E_p^T+\sum_k E_k\right)+\lambda_N P+\lambda_R K+\lambda_\Xi\frac{\Xi}{1000}$$

$$\text{s.t.}\quad\begin{cases}\text{问题二模型 (19) 的全部约束,}\\ \Delta_k\le\Delta_{h_k}^{\max},\qquad E_k\le(1-\rho_R)E_R^{\text{use}}, & \forall k,\\ \text{式 (34) 的连续通信包含关系,} & \forall p,h,l,\\ \text{中继机与能源组件的占用区间两两不重叠,}\\ \text{中继机区间含周转时间 } t^{\text{turn}}\text{，能源组件区间含充电时间 } t_{\text{chg}}(\sigma_k),\\ C_{\text{mk}}=\max\left\{\max_p r_p,\ \max_k t_k^{\text{ret}}\right\}.\end{cases} \tag{35}$$

其中 $\lambda_\Xi=10^4$，使任何无保障时长都远比其余指标更不可接受；$\lambda_R=1$ 与架次数同权，体现中继出动同样占用资源。

6.5 模型的求解

联合模型的搜索空间由架次序列与班次表两部分构成，二者互相牵制，因此不能分开优化。求解步骤如下。

Step1：以问题二基准权重下的最好解为初始架次序列，由其无中继情形下的排程反推各悬停点的需求时段，合并相邻需求并按式 (33) 拆分，得到初始班次表。

Step2：以概率 0.35 对班次表执行扰动，包括插入一个新班次、删除一个班次、调整某班次的服务时长、交换两个班次的先后、改变某班次的悬停点。

Step3：以概率 0.20 对架次序列执行局部扰动，其余情形执行破坏与修复，算子与问题二相同。

Step4：对新解运行联合排程仿真。中继班次按顺序派给最早空闲的中继机与最早充满的能源组件，运输架次的起飞时刻则推迟到其全部失联时段都落入对应悬停点的在役窗口之内，推迟量由式 (34) 反解。

Step5：按 Metropolis 准则接受新解，温度线性退火。

Step6：重复 Step2 至 Step5 共 30000 次，并行运行 17 组不同随机种子与权重配置。

Step7：对历史最优解执行班次修剪，把每个班次的服务时长缩到其实际服务的最后一段需求结束，删去未服务任何需求的班次，仅在目标不变差时接受修剪结果。

Step8：用独立的核验程序逐 5 秒重新采样复算，逐点判定直连与中继两种保障方式，确认无保障采样数为零。

6.5.1 分步求解与联合求解的对比

为说明把运输与中继放进同一个解的必要性，把联合搜索与两种分步做法对比。第一种是事后按需排中继：固定问题二的方案不动，由其失联时段反推中继需求，合并后直接派给两架中继。第二种是预排结构化班次：仍固定问题二的方案，但事先让一架中继钉在一个簇的悬停点连续值守、另一架在其余两簇之间轮换，再把运输架次的起飞推迟到对应窗口之内。两种分步做法都只做一次排程、不再搜索，恰好就是联合搜索各次运行的初始解，其结果可直接从运行日志读出。联合搜索在基准权重下以 7 个随机种子独立运行。结果列于表 11。

表 11　问题三分步求解与联合求解的对比

方法	运行次数	无保障时长/s	最好目标值	联合完成时间/s	零违约次数
分步法，事后按需排中继	1	2234	34488.1	8708	0
分步法，预排结构化班次	6	0 至 3366	12938.9	11692	0
联合搜索	7	0	151.6	10919	4

事后按需排中继的完成时间最短，但有 2234 s 的飞行处于无保障状态：两架中继满足不了问题二方案里同时出现的失联时段，这在题目的连续通信要求下是不可接受的。预排结构化班次能把无保障时长压到零，但代价是运输架次大面积推迟，6 次中每次都出现时限违约，最好的一次目标值仍高达 12938.9。联合搜索的 7 次运行全部做到零无保障时长，其中 4 次同时做到时限零违约，最好解目标值 151.6，比最好的分步方案低两个数量级。联合搜索单次平均用时约 1200 s。对比说明中继能否跟上取决于运输架次如何组批与排序，只有两者一起搜索，才能在只有两架中继的条件下同时满足通信与时限。

6.6 结果分析与保障率检验

6.6.1 联合调度方案

最终方案共 28 个运输架次与 5 个中继架次，运输能耗 80.04 kWh，中继能耗 5.82 kWh，总能耗 85.87 kWh，联合任务完成时间 10919.4 s。与问题二相比，运输架次由 22 增至 28，完成时间由 7009.7 s 延长至 10919.4 s，增幅 55.8%。这一代价主要来自等待：运输架次必须等中继到位并完成建链才能起飞，全部架次因此累计推迟 4414.1 s。架次结构也随之改变，多点架次由 7 个增至 15 个，同簇服务区更多地串成两点架次，使同一地理簇的失联航段集中落进同一个中继在役窗口，减少中继出动次数；而每个架次的装载量相应减小，架次总数随之增加。

运输架次列于表 12，中继架次按提交模板的列结构列于表 13。

表 12　问题三的运输架次安排

架次编号	无人机	机型	电池	开始 (s)	访问服务区顺序	返回 (s)	能耗 (kWh)
Q3-001	U01	A	BA01	0.0	S001	1250.9	1.2403
Q3-002	U02	A	BA02	0.0	S007	1667.4	1.8726
Q3-003	U05	B	BB01	0.0	S014	1869.8	2.1404
Q3-004	U03	A	BA03	0.0	S007	1727.4	1.9054
Q3-005	U06	B	BB02	0.0	S001	1149.1	1.2204
Q3-006	U07	C	BC01	2030.1	S002→S009	4393.4	6.2602
Q3-007	U04	A	BA04	0.0	S001	1250.9	1.2823
Q3-008	U01	A	BA05	1250.9	S001	2501.8	1.2823
Q3-009	U04	A	BA06	1250.9	S010→S001	3282.7	2.3988
Q3-010	U06	B	BB03	1149.1	S013→S012	3294.6	2.7067
Q3-011	U05	B	BB04	1869.8	S013→S012	4015.3	2.6841
Q3-012	U08	C	BC02	0.0	S006	1495.6	2.4536
Q3-013	U02	A	BA01	2109.2	S002→S009	4385.0	3.0053
Q3-014	U03	A	BA04	2121.4	S005→S008	4554.5	3.2166
Q3-015	U08	C	BC03	1495.6	S001→S010	3816.3	4.4437
Q3-016	U06	B	BB02	3294.6	S005→S008	5454.5	3.0331
Q3-017	U08	C	BC04	3816.3	S004→S005	6464.0	6.3480
Q3-018	U05	B	BB01	4015.3	S004	5936.9	3.0354
Q3-019	U06	B	BB03	5454.5	S001	6543.6	0.9620
Q3-020	U07	C	BC02	4873.7	S007→S003→S015	7793.2	6.1808
Q3-021	U05	B	BB04	5936.9	S006→S003	8267.1	2.5369
Q3-022	U01	A	BA02	2708.4	S008	4879.5	3.3085
Q3-023	U04	A	BA03	4633.8	S011→S015→S003	7481.9	3.3574
Q3-024	U02	A	BA05	4937.7	S003→S015	7481.0	3.3573
Q3-025	U06	B	BB02	7435.5	S003	9456.0	2.4242
Q3-026	U03	A	BA06	4554.5	S005→S002	6988.5	3.1122
Q3-027	U01	A	BA01	5753.2	S011	6975.3	1.1913
Q3-028	U05	B	BB03	8267.1	S009→S008	10553.1	3.0836

表 13　问题三的中继架次安排，对应提交模板 Q3_ 中继架次

中继架次	中继机	组件	开始 (s)	经度 (°)	纬度 (°)	海拔 (m)	建链 (s)	结束 (s)	返回 (s)	能耗 (kWh)
R-001	R01	PR01	0	109.2682	23.0154	665.9	605	4354	4793	1.369
R-002	R02	PR02	0	109.2054	23.0436	757.2	679	1279	1801	0.448
R-003	R02	PR03	2101	109.2364	23.0656	583.5	2848	6708	7283	1.500
R-004	R01	PR04	5093	109.2054	23.0436	757.2	5772	10050	10571	1.571
R-005	R02	PR05	7583	109.2364	23.0656	583.5	8330	10344	10919	0.936

五个中继班次中，服务最久的 R-004 与 R-003 分别值守 4277 s 与 3860 s，约为式 (33) 给出上限的一半，说明限制中继的并非单次续航，而是只有两架中继机；R-002 仅服务 600 s，是为覆盖开头一个孤立的失联时段而临时起飞。两架中继机与六组能源组件中实际用到五组，能源组件有一组冗余。

6.6.2　通信保障检验

对最终方案用独立程序逐 5 秒重新采样，共 9188 个采样点，全部处于直连或中继保障之下，无保障采样数为零。按保障时长计，直连承担 64.7%、中继承担 35.3%；按采样点计，中继保障的采样点为 3998 个，占 43.5%，两者不同是因为地面准备阶段只计为一个采样点而时长较长。

通信保障的时序如图 24 所示。图 24(a) 逐架次给出直连与中继的交替分段，底部五条短横线是五个中继架次的在役时段，可以看出中继的在役区间基本连续覆盖了整个任务周期；图 24(b) 给出各架次的直连与中继时长，单个架次的中继时长占比在 0 至 53.3% 之间，28 个架次中有 7 个完全不需要中继，它们都是只访问 S001、S006 或 S011 的单点往返。S011 的作业点同样可与网关直连，在 Q3-027 中它单独成架次、全程直连；在 Q3-023 中它与 S015、S003 串成三点架次，后两段的失联时段需要中继，该架次因此有 1211 s 处于中继保障之下。

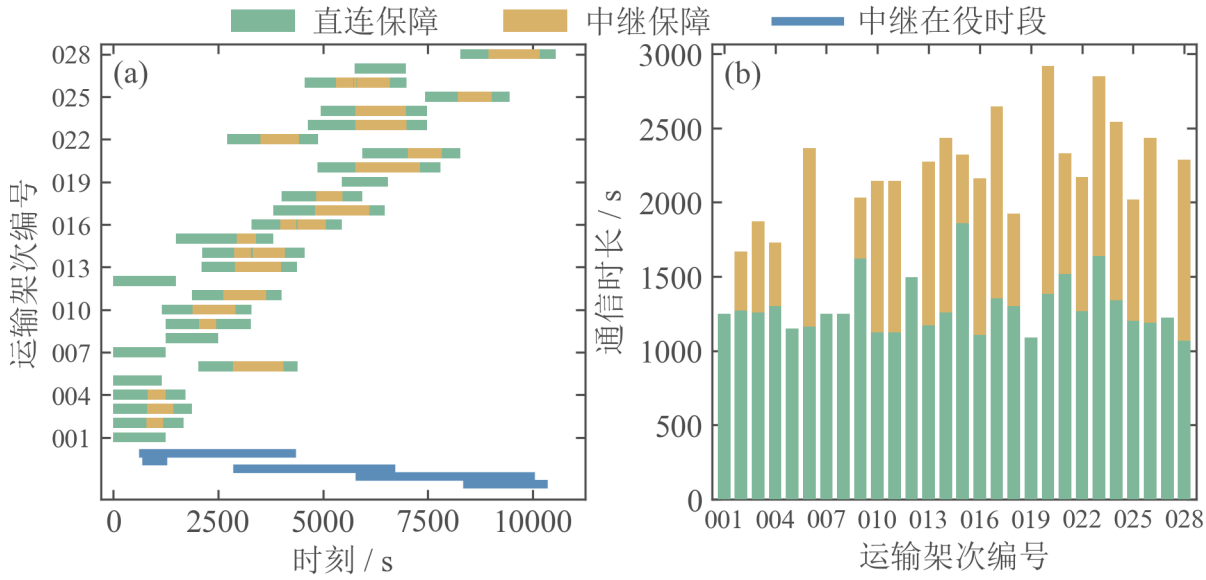


图 24　问题三的通信保障：(a) 保障方式时序；(b) 直连与中继时长

按架次汇总的通信保障情况列于表 14，完整的分段记录共 385 行见支撑材料中的结果文件。

表 14　问题三的通信保障按架次汇总，对应提交模板 Q3_ 通信保障

运输架次	分段数	直连时长 (s)	中继时长 (s)	中继占比 (%)	涉及中继架次
Q3-001	9	1250.9	0.0	0.0	
Q3-002	11	1269.7	397.7	23.9	R-002
Q3-003	11	1256.8	613.0	32.8	R-001
Q3-004	11	1299.7	427.7	24.8	R-002
Q3-005	9	1149.1	0.0	0.0	
Q3-006	17	1160.6	1202.7	50.9	R-001;R-003
Q3-007	9	1250.9	0.0	0.0	
Q3-008	9	1250.9	0.0	0.0	
Q3-009	15	1622.9	408.9	20.1	R-001
Q3-010	15	1122.4	1023.1	47.7	R-001
Q3-011	15	1122.4	1023.1	47.7	R-001

运输架次	分段数	直连时长 (s)	中继时长 (s)	中继占比 (%)	涉及中继架次
Q3-012	9	1495.6	0.0	0.0	
Q3-013	17	1172.0	1103.8	48.5	R-001;R-003
Q3-014	20	1257.6	1175.6	48.3	R-001;R-003
Q3-015	15	1858.0	462.7	19.9	R-001
Q3-016	20	1107.9	1051.9	48.7	R-001;R-003
Q3-017	15	1354.6	1293.1	48.8	R-003
Q3-018	11	1302.7	618.9	32.2	R-003
Q3-019	9	1089.1	0.0	0.0	
Q3-020	20	1381.4	1538.1	52.7	R-003;R-004
Q3-021	15	1516.2	814.0	34.9	R-004
Q3-022	12	1268.0	903.1	41.6	R-001;R-003
Q3-023	20	1637.5	1210.6	42.5	R-003;R-004
Q3-024	16	1338.2	1205.2	47.4	R-003;R-004
Q3-025	11	1202.5	818.0	40.5	R-004
Q3-026	19	1186.9	1247.0	51.2	R-003
Q3-027	9	1222.0	0.0	0.0	
Q3-028	16	1066.9	1219.1	53.3	R-004;R-005

物资交付方面，80 个货箱的首批截止要求与期望送达时间要求全部满足，最晚交付时刻 9882.4 s，平均交付时刻 4044.8 s。逐箱交付时刻列于表 15。

表 15　问题三的逐箱交付时刻，每行两条记录

货箱编号	架次编号	交付时刻 (s)	货箱编号	架次编号	交付时刻 (s)
S001-FOD-01	Q3-007	908.6	S001-FOD-02	Q3-019	6252.1
S001-FOD-03	Q3-008	2159.5	S001-HYG-01	Q3-015	2671.8
S001-HYG-02	Q3-015	2671.8	S001-MED-01	Q3-015	2671.8
S001-MED-02	Q3-001	908.6	S001-WAT-01	Q3-008	2159.5
S001-WAT-02	Q3-009	2940.3	S001-WAT-03	Q3-007	908.6
S001-WAT-04	Q3-005	857.7	S001-WAT-05	Q3-005	857.7
S001-WAT-06	Q3-015	2671.8	S001-WAT-07	Q3-001	908.6
S001-WAT-08	Q3-015	2671.8	S002-FOD-01	Q3-006	3481.4
S002-FOD-02	Q3-006	3481.4	S002-HYG-01	Q3-026	6268.5
S002-MED-01	Q3-013	3423.0	S002-WAT-01	Q3-013	3423.0
S002-WAT-02	Q3-006	3481.4	S002-WAT-03	Q3-006	3481.4
S002-WAT-04	Q3-006	3481.4	S003-FOD-01	Q3-025	8723.7
S003-FOD-02	Q3-021	7534.8	S003-HYG-01	Q3-025	8723.7
S003-MED-01	Q3-020	6674.7	S003-WAT-01	Q3-024	6317.0
S003-WAT-02	Q3-020	6674.7	S003-WAT-03	Q3-023	6628.6
S003-WAT-04	Q3-020	6674.7	S004-FOD-01	Q3-017	5357.8
S004-HYG-01	Q3-017	5357.8	S004-MED-01	Q3-017	5357.8
S004-WAT-01	Q3-018	5257.7	S004-WAT-02	Q3-018	5257.7
S004-WAT-03	Q3-017	5357.8	S005-FOD-01	Q3-017	5960.1
S005-HYG-01	Q3-017	5960.1	S005-MED-01	Q3-026	5653.4
S005-WAT-01	Q3-016	4305.5	S005-WAT-02	Q3-017	5960.1
S005-WAT-03	Q3-014	3220.2	S006-FOD-01	Q3-021	6789.3
S006-HYG-01	Q3-012	1143.4	S006-MED-01	Q3-012	1143.4
S006-WAT-01	Q3-012	1143.4	S006-WAT-02	Q3-012	1143.4

货箱编号	架次编号	交付时刻 (s)	货箱编号	架次编号	交付时刻 (s)
S006-WAT-03	Q3-012	1143.4	S007-FOD-01	Q3-020	6143.4
S007-HYG-01	Q3-020	6143.4	S007-MED-01	Q3-004	1141.4
S007-WAT-01	Q3-002	1081.4	S007-WAT-02	Q3-004	1141.4
S008-FOD-01	Q3-022	4074.2	S008-HYG-01	Q3-028	9882.4
S008-MED-01	Q3-014	3749.2	S008-WAT-01	Q3-016	4783.8
S008-WAT-02	Q3-022	4074.2	S009-FOD-01	Q3-006	3905.0
S009-MED-01	Q3-013	3822.3	S009-WAT-01	Q3-028	9267.9
S010-FOD-01	Q3-015	3309.7	S010-MED-01	Q3-009	2325.9
S010-WAT-01	Q3-015	3309.7	S011-FOD-01	Q3-027	6646.4
S011-MED-01	Q3-023	5527.0	S011-WAT-01	Q3-027	6646.4
S012-FOD-01	Q3-011	3363.8	S012-MED-01	Q3-011	3363.8
S012-WAT-01	Q3-010	2643.2	S013-FOD-01	Q3-010	2181.9
S013-MED-01	Q3-010	2181.9	S013-WAT-01	Q3-011	2872.5
S014-FOD-01	Q3-003	1242.4	S014-MED-01	Q3-003	1242.4
S014-WAT-01	Q3-003	1242.4	S015-FOD-01	Q3-024	6770.2
S015-MED-01	Q3-023	6166.3	S015-WAT-01	Q3-020	7131.6

6.6.3 五项指标的权衡关系

改变式 (35) 中的权重分别侧重不同指标，得到表 16 的对照。

表 16 问题三五项指标的权衡

侧重指标	运输架次	运输能耗/kWh	中继架次	中继能耗/kWh	联合完成时间/s
总能耗，作为最终方案	28	80.04	5	5.82	10919
均衡	27	84.87	4	4.98	10462
配送及时性	27	85.92	4	5.82	10792
架次数	24	89.07	5	5.82	11499
联合完成时间	29	91.12	4	5.56	10846

与问题二不同，本问的指标冲突集中在中继的使用方式上。五组权重共 17 次运行，按基准权重重新评价后目标值最低者出自侧重总能耗的那一组，说明换一个搜索方向比在同一权重下多次运行更容易找到好解。该方案用 28 个运输架次换来 80.04 kWh 的运输能耗，比均衡权重下的最好解低 4.83 kWh，代价是联合完成时间延长 457 s、中继能耗高出 0.84 kWh，加权后仍净胜。侧重架次数时运输架次压到 24 个，但完成时间最长，达 11499 s，说明更少的架次意味着每个架次装得更满、飞得更久。侧重联合完成时间时运输架次增至 29 个，用更细的架次划分换取更灵活的起飞时机，完成时间缩短到 10846 s。侧重配送及时性的方案与均衡组接近，因为各组都已做到时限零违约，加大延误权重在可行区域内没有作用对象。

综合来看，本问的最终答案取基准目标值最小的方案：28 个运输架次加 5 个中继架次，总能耗 85.87 kWh，联合任务完成时间 10919.4 s，全程通信保障率 100%。

6.7 本章小结

本章在问题二的基础上引入连续通信约束与中继调度，建立了运输与中继联合调度模型。链路预算的计算表明本场景中通信可用性几乎完全由地形遮挡决定，15 个服务区中仅 3 个可与固定网关直连，中继是刚需。悬停点选址采用按航段整段覆盖的集合覆盖模型，这

一处理避免了保障者在飞行途中频繁切换，使两架中继得以排开，选出的 3 个悬停点恰好各覆盖一个地理簇。联合排程把班次表与架次序列放进同一个解一并搜索，最终方案为 28 个运输架次加 5 个中继架次、85.87 kWh、10919.4 s，独立核验的 9188 个采样点全部处于保障之下。与问题二对比可知，通信保障使完成时间延长约 56%，这是山区应急投送必须付出的代价。

七、 问题四模型的建立、分析与求解

7.1 问题分析

问题四要求在不改动问题三方案的前提下把 15 个服务区切分成 2 个和 3 个独立执行的任务组，并核算各组所需资源、比较两种分区方式。

首先要确定可切分的粒度。题目规定同一运输架次涉及的服务区必须划入同一组，这一条在服务区之间诱导出一个等价关系：若两个服务区被同一架次访问，则它们必须同组；这种关系的传递闭包把服务区并成若干个不可再分的原子块，分区只能在原子块之间进行。原子块的数量既决定了分组方案的多少，也决定了分为 3 组是否可行。

其次要确定资源怎么算。由于方案中各架次的开始与返回时刻已经固定，某一类资源在某个时刻的并发占用数就是该组独立执行时必须配备的数量。这把资源核算化为一个区间重叠计数问题，不需要重新求解调度。需要注意四类资源的占用区间口径不同：运输无人机在架次期间被占用；共享电池除架次期间外还要加上充电时段，因为充电完成前不能再次投入；中继无人机要加上架次周转时间；中继能源组件同样要加上充电时段。

最后要确定比较的维度。题目列出了资源配置规模、资源冗余、组间工作量均衡以及与现有库存的缺口四项。本文把前三项量化为八类资源的合计台套数、相对于整体执行的增量、以及各组累计作业时长的极差比，把第四项量化为逐类资源的短缺数，并按缺口、规模、失衡的优先顺序对全部合法分区排序。

问题四的求解思路如图 25 所示。第一行确定可切分的粒度，从问题三的方案出发用并查集得到原子块并枚举合法分区；第二行对每个分区按区间重叠计数核算资源峰值，再按缺口、规模、失衡排序选出两种分区方式各自的最优方案。

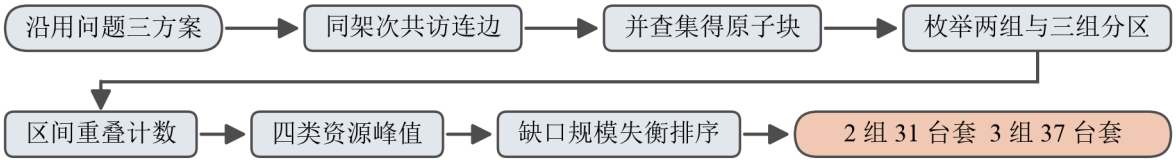


图 25 问题四的求解思路

7.2 任务原子块的识别

设服务区集合为 $\mathcal{V} = \{S_1, \ldots, S_{15}\}$。对问题三方案中的每个运输架次 p，其访问的服务区集合记为 $\mathcal{R}_p$。定义关系

$$S_i \sim S_j \iff \exists p:\ S_i \in \mathcal{R}_p \ \wedge\ S_j \in \mathcal{R}_p. \tag{36}$$

该关系的传递闭包是一个等价关系，其等价类即为不可再分的任务原子块。用并查集实现：初始时每个服务区自成一类，遍历全部架次，把同一架次访问的服务区逐一合并。

对问题三的最终方案执行上述合并，得到 5 个原子块

$$\begin{aligned}\mathcal{A}_1 &= \{S002, S004, S005, S008, S009\},\\ \mathcal{A}_2 &= \{S003, S006, S007, S011, S015\},\\ \mathcal{A}_3 &= \{S001, S010\},\quad \mathcal{A}_4 = \{S012, S013\},\quad \mathcal{A}_5 = \{S014\}.\end{aligned} \tag{37}$$

合并过程如图 26 所示。图中服务区按真实的相对位置摆放，凡被问题三方案中同一架次连续访问的两个服务区之间连一条边，连通在一起的服务区就是一个原子块，外面的圆角包络标出其范围。北部块是一条从 S008 经 S005、S004 到 S002、S009 的链，西部块由 S007 居中连起其余四个服务区，两块都含 5 个服务区，与问题三中北部、西部两个悬停点覆盖的地理簇一致。东部簇则碎成三块：S010 与 S001 由架次 Q3-009 与 Q3-015 连成一块，S013 与 S012 由 Q3-010 与 Q3-011 连成一块，S014 只出现在单点架次 Q3-003 里，与任何服务区都没有共访，自成一块。五块之间没有任何边相连，调度中心 O01 位于各块之间的谷地。

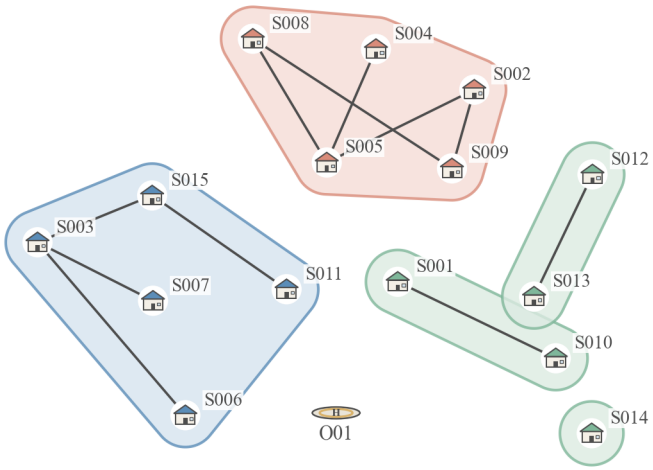


图 26　原子块合并

原子块与地理簇不再一一对应，原因在于问题三为满足连续通信重新组织了架次。题目从问题二起就限制一个架次只能组合同簇的服务区，这条约束保证了跨簇的架次不存在，但簇内连通到什么程度仍取决于搜索：问题三的架次必须让同一簇的失联航段集中落进同一个中继在役窗口，东部簇因此被拆成大量单点架次，簇内只剩两条连访边，连通性不足以把 5 个服务区并成一块。相比之下西部与北部两簇的失联时段更集中，架次可以更多地串联，两块都保持完整。这说明分区的粒度并非只由空间聚类决定，通信约束会通过对组批的影响间接改变可分的结构；问题二中不加同簇限制的 6 次对照运行里，原子块数更是随运行在 1 到 7 之间波动，分组结构本身就依赖搜索的偶然结果。

五个原子块给出 15 种分为 2 组的方案与 25 种分为 3 组的方案。这一结果比预期更零碎，也直接决定了后文的资源配置结论：粒度过细会让单个任务组承担的任务量悬殊，工作量失衡指标随之恶化。

7.3　分区资源核算模型的建立

设任务组划分为 $\{\mathcal{G}_1, \ldots, \mathcal{G}_K\}$，每个 $\mathcal{G}_k$ 是若干原子块的并。记归属于组 k 的运输架次集合为 $\mathcal{P}_k$，中继架次集合为 $\mathcal{K}_k$，后者由通信保障关系确定，即凡为 $\mathcal{P}_k$ 中任一架次提供过保障的中继架次都计入 $\mathcal{K}_k$。

占用区间的定义。四类资源的占用区间分别为

$$\begin{cases} \text{运输无人机：} & I_p^{U} = [s_p,\ r_p]\,, \\ \text{共享电池：} & I_p^{B} = [s_p,\ r_p + t_{\text{chg}}(\sigma_p)]\,, \\ \text{中继无人机：} & I_k^{R} = [s_k,\ t_k^{\text{ret}} + t^{\text{turn}}]\,, \\ \text{能源组件：} & I_k^{C} = [s_k,\ t_k^{\text{ret}} + t_{\text{chg}}(\sigma_k)]\,. \end{cases} \tag{38}$$

需求量的计算。某类资源在组 k 的需求量等于其占用区间的最大重叠数

$$n_k^{\bullet} = \max_{t}\ \left|\{I \in \mathcal{I}_k^{\bullet}\ :\ t \in I\}\right|, \tag{39}$$

其中 $\mathcal{I}_k^{\bullet}$ 是该类资源在组 k 内的全部占用区间。运输无人机与共享电池还须按机型分池分别计算。式 (39) 用扫描线实现：把每个区间的左右端点排成事件序列，左端点计 $+1$、右端点计 -1，按时间扫描取前缀和的最大值。

比较指标。记八类资源为 $\mathcal{T} = \{A, B, C, b_A, b_B, b_C, R, CP\}$，库存为 N_τ^{inv}。定义资源配置规模、资源缺口与组间工作量失衡为

$$\Theta = \sum_{\tau\in\mathcal{T}}\sum_{k=1}^{K} n_k^{\tau}, \qquad \Psi = \sum_{\tau\in\mathcal{T}}\left[\sum_{k=1}^{K} n_k^{\tau} - N_\tau^{\text{inv}}\right]^{+}, \tag{40}$$

$$\Omega = \frac{K\left(\max_k W_k - \min_k W_k\right)}{\sum_k W_k}, \qquad W_k = \sum_{p\in\mathcal{P}_k}\left(r_p - s_p\right), \tag{41}$$

其中 W_k 是组 k 的累计作业时长，Ω 是归一化的极差，取值越小越均衡。

综上所述，最终得到**任务分区资源核算模型**为

$$\begin{aligned} &\text{lex-}\min_{\{\mathcal{G}_k\}} \quad (\ \Psi,\ \Theta,\ \Omega\ ) \\ &\text{s.t.} \quad \begin{cases} \bigcup_{k=1}^{K}\mathcal{G}_k = \mathcal{V}, \qquad \mathcal{G}_k\cap\mathcal{G}_{k'} = \varnothing,\ k\neq k', & \\ \mathcal{G}_k \neq \varnothing, & \forall k, \\ \forall p,\ \exists k:\ \mathcal{R}_p \subseteq \mathcal{G}_k, & \\ n_k^{\tau} \text{ 由式 (39) 确定}, & \forall k, \tau. \end{cases} \end{aligned} \tag{42}$$

第三条约束即同架次共访的服务区必须同组，它等价于要求每个 $\mathcal{G}_k$ 是若干原子块的并。原子块有 5 个，$K = 2$ 时合法方案 15 种、$K = 3$ 时 25 种，规模都很小，可直接枚举求解。

7.4 分区方案的比较与资源缺口

7.4.1 资源配置结果

两种分区的最优方案列于表 17。$K = 2$ 的最优方案把含 S014 的两个原子块分出一块，其余 14 个服务区合成一组；$K = 3$ 的最优方案再把 {S012, S013} 分出去，S014 自成一组。两种方案都属于「一个大组加若干小组」的形态，因为原子块本身的大小悬殊，最大的块占 5 个服务区，最小的只有 1 个，任何分法都无法把任务量摊平。

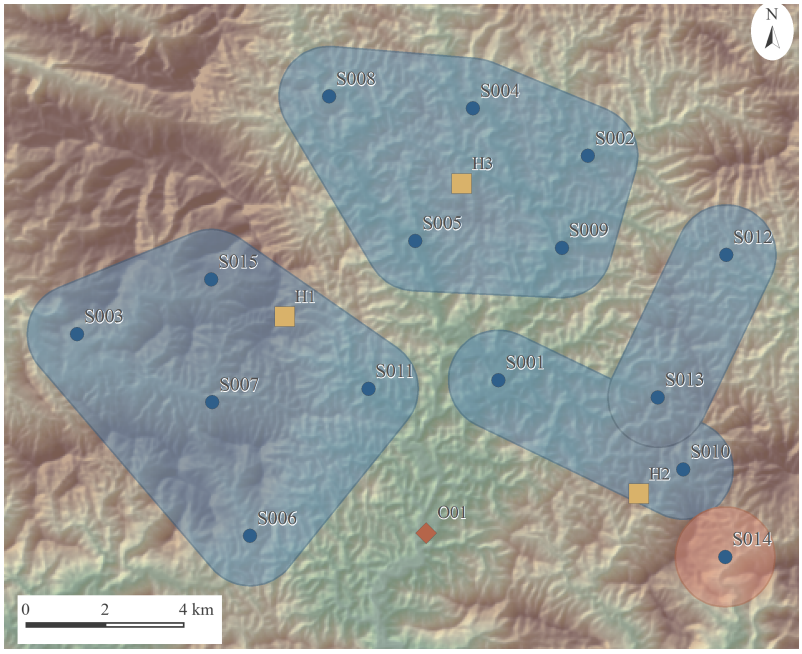


图 27　分为 2 组时的最优分组

$K = 2$ 的分组在地图上的位置如图 27 所示，蓝色是合并后的第一组，红色是单独成组的第二组即 S014，黄色方块是三个中继悬停点。第一组覆盖研究区的绝大部分，S014 只是东部边缘的一个孤立点。两组独立执行时都要各配一套运输机、电池与中继，S014 那一组只送 3 个货箱却同样要占用一架 B 型机、一组电池和一架中继机，这正是分组带来的固定开销。

表 17　两种分区方式的资源配置方案，对应提交模板 Q4_ 分区配置

K	任务组	服务区列表	A 机	B 机	C 机	A 池	B 池	C 池	中继	组件
2	G2-1	S001; S010; S002; S004; S005; S008; S009; S003; S006; S007; S011; S015; S012; S013	4	2	2	6	4	4	2	3
2	G2-2	S014	0	1	0	0	1	0	1	1
3	G3-1	S001; S010; S002; S004; S005; S008; S009; S003; S006; S007; S011; S015	4	2	2	6	4	4	2	3
3	G3-2	S012; S013	0	2	0	0	2	0	1	1
3	G3-3	S014	0	1	0	0	1	0	1	1

八类资源的需求与现有库存的对照列于表 18。

现有库存八类资源合计 30 台套。整体不分区执行时共需 27 台套，除能源组件用 3 组、富余 3 组之外，其余七类都与库存持平，没有任何余量。分为 2 组时合计升至 31 台套，比不分区多出 4 台套，缺口 3 台套，为 B 型机 1 架、B 型电池 1 组、中继机 1 架；分为 3 组时合计升至 37 台套，多出 10 台套，缺口扩大到 8 台套，为 B 型机 3 架、B 型电池 3 组、中继机 2 架。

分区代价的构成如图 28 所示。图 28(a) 给出八类资源在三种口径下的需求，横线是现有库存，超出横线的部分即为缺口，可以看出缺口集中在 B 型机、B 型电池与中继机上；图

表 18　资源配置与缺口对照

口径	A 机	B 机	C 机	A 电池	B 电池	C 电池	中继机	能源组件	合计
现有库存	4	2	2	6	4	4	2	6	—
整体不分区需求	4	2	2	6	4	4	2	3	27
分 2 组合计	4	3	2	6	5	4	3	4	31
分 2 组缺口	0	1	0	0	1	0	1	0	3
分 3 组合计	4	5	2	6	7	4	4	5	37
分 3 组缺口	0	3	0	0	3	0	2	0	8

28(b) 给出各任务组的累计作业时长与承担货箱数，两种分法都是一个大组担下几乎全部任务，其余小组各自只承担一两个架次，作业时长相差两个数量级，且分得越细小组越多、失衡越明显。

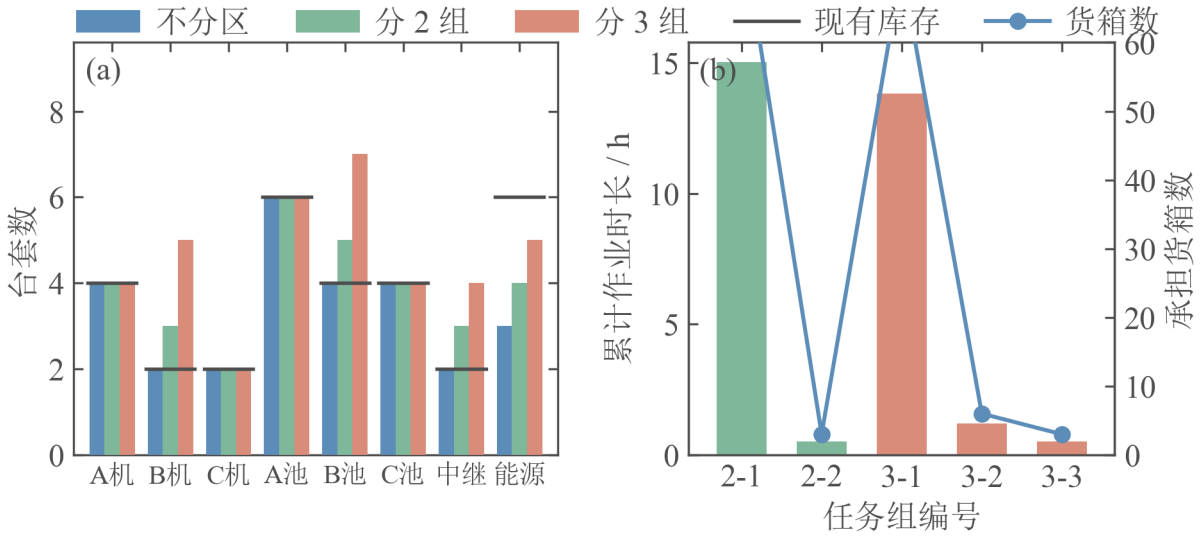


图 28　问题四的分区代价：(a) 资源需求；(b) 组间工作量

7.4.2　两种分区方式的比较

把四个维度的结果汇总于表 19。

表 19　两种分区方式的综合比较

比较维度	不分区	分 2 组	分 3 组
资源配置规模 / 台套	27	31	37
相对不分区的冗余 / 台套	0	4	10
资源缺口 / 台套	0	3	8
组间工作量失衡指标	—	1.866	2.570
最大组作业时长 / h	15.55	15.03	13.84

分得越细，三项代价同时变差。分为 2 组时第一组承担 77 个货箱、作业时长 15.03 h，第二组只送 3 个货箱、作业时长 0.52 h，失衡指标 1.866；分为 3 组时小组增至两个，最大组降到 13.84 h，但另两组分别只有 1.19 h 与 0.52 h，极差反而拉大，失衡指标升到 2.570。

资源配置规模由 31 台套增至 37 台套，缺口由 3 台套增至 8 台套。也就是说，在这一问的原子块结构下，分 3 组唯一的好处是最大组早收工 1.19 h，其余三项全部变差。

若救援指挥的首要目标是各方向同步展开，应选择分 3 组的方案，代价是必须额外调入 B 型运输机 3 架、B 型电池 3 组、中继机 2 架，合计 8 台套；若资源紧张而允许先后展开，则分 2 组的方案更为现实，只需补充 B 型运输机 1 架、B 型电池 1 组、中继机 1 架，合计 3 台套，A 型机、C 型机、A 型电池、C 型电池与中继能源组件都不缺。

7.4.3　资源缺口的成因

两种分区都出现了缺口，且分得越细缺口越大。缺口的根本来源是资源不得跨组调配这一约束切断了时间上的错峰复用。

在不分区执行时，一架无人机可以先服务西部的某个架次，返回充电后再服务东部的另一个架次，两次任务在时间上错开，同一台设备被复用了两次。分组之后这两个架次属于不同的组，必须由两台设备分别承担，即便它们在时间上并不冲突。因此分区带来的额外需求不是因为任务量增加，而是因为复用机会被人为取消。原子块越细，跨组的错峰对越多，被切断的复用也就越多，这正是缺口从 3 台套扩大到 8 台套的原因。

从资源对照还可以看出各类资源的缺口程度差别很大。A 型机四架全程在飞，峰值占用恰好等于库存；C 型机只有两架，两组独立执行时各自只需要一架，反而够用；中继能源组件在不分区时富余 3 组、分 3 组时也只用 5 组，仍有余量。真正吃紧的是 B 型机与其电池：B 型机只有两架而任务量最大，分 2 组时两组的峰值恰好错开，只需补 1 架；分 3 组时三个组的峰值无法再错开，要补 3 架，B 型电池同步缺 3 组。中继机同样只有两架，分组后每个含悬停点的组都要各配一架，分 3 组时缺 2 架。这提示在资源配置阶段就应当为可能的分区执行预留冗余，尤其是 B 型机与中继机这两类库存最紧的装备。

7.5　模型的检验

资源核算的正确性用两种独立算法相互校核。第一种是正文的扫描线法，按区间端点排序后取前缀和最大值；第二种是逐秒计数法，在 0 至 12000 s 上每隔 1 s 数一次各类资源被占用的个数，取最大值。对两种分区的全部 5 个任务组、三种运输机型逐一比对，两种算法给出的并发峰值完全一致。同时核对了分区的合法性：每个服务区恰属一组，每组非空，问题三方案中每个运输架次访问的服务区都落在同一组内，符合题目要求。

7.6　本章小结

本章在不改动问题三方案的前提下研究了任务分区与资源配置。通过同架次共访关系的传递闭包识别出 5 个不可再分的任务原子块，使分区问题的解空间被压缩到可枚举的规模；通过区间重叠计数把资源核算化为扫描线问题，避免了重新求解调度。结果表明分为 2 组需 31 台套、缺口 3 台套、失衡指标 1.866，分为 3 组需 37 台套、缺口 8 台套、失衡指标 2.570。与问题二相比，本问的原子块不再与地理簇一一对应：问题三为满足连续通信把东部簇拆成了大量单点架次，簇内连通性下降，原子块从三块碎成五块，直接导致两种分法都出现「一个大组加若干小组」的失衡形态，且分得越细越失衡。缺口的成因是资源不得跨组调配切断了时间上的错峰复用，而非任务量本身的增加，补缺的重点落在库存最紧的 B 型机与中继机上。

八、 模型的评价

8.1 模型的优点

本文四个模型共用同一套航段几何、时间与能耗口径，前一问的结论直接作为后一问的输入，因此四问的结果可以相互印证。问题一用约束规划得到的精确解为问题二的启发式算法提供了单点往返情形下的对照；问题二搜索得到的优质方案作为问题三的初始解，使联合搜索从一个已知可行的起点出发；问题三的方案又完整地传递给问题四，分区核算不需要重新求解调度。这种层层承接的结构避免了各问之间口径不一致带来的矛盾。

在建模处理上，本文用一个确定性可行性仿真器替代了显式的资源分配变量。派机、配电池与起飞时刻由规则还原，使得任意架次序列都自动满足实体机数量、电池数量、不重叠与充电周转等全部资源约束，搜索空间由此压缩到组批与顺序本身。这一处理让 80 个货箱、8 架实体机、14 组电池的规模在启发式框架下变得可解，否则仅资源分配一项就会引入数千个整数变量。

在通信建模上，本文不止于列出链路预算公式，而是先算出三类链路在视距与遮挡两种情形下的最远作用距离，据此判断本场景的链路可用性由地形主导，从而把通信问题转化为数字高程模型上的视线判定问题。中继选址采用按航段整段覆盖的集合覆盖模型，避免了保障者在飞行途中频繁切换，这是仅用两架中继完成全部保障的关键。

在结果检验上，本文对每一问都做了独立复算。问题一的组批由约束规划精确求解，架次数与逐服务区下界相等，已证最优；问题二的最终方案经资源可行性程序逐项核对；问题三的通信保障由一个不依赖预计算表的独立程序逐 5 秒重新采样，确认 9188 个采样点全部有保障；问题四的资源核算用扫描线直接从时刻表推出，与调度求解过程完全解耦。

8.2 模型的缺点

首先是能耗口径的不确定性。题目给出了等效航程关系却未给出水平能耗与爬升附加能耗的闭式，本文按多旋翼功率随总重单调上升的物理规律自行确定了两项的形式。这一口径在量纲与单调性上均自洽，也与题目对中继无人机的规定相容，但它并非题目的原意，若组委会采用另一种折算方式，最大安全载荷与架次能耗的数值会随之改变，不过各机型之间的相对关系与全文的结论不会受影响。

其次是直线航段假设带来的能耗高估。题目规定巡航海拔取沿线最高地面高程加 50 米，而由图 5 可见该最高点往往只是航段中的一个孤立山尖，其余航程实际留有远大于 50 米的净空。若允许无人机沿地形起伏调整高度，或允许绕过个别高点，爬升总量与能耗都会明显下降。本文未做此项优化，因为题目的计算规则已把航段固定为直线。

再次是启发式解的最优性无法保证。问题二与问题三的规模超出了精确方法的能力，本文采用的混合算法无法证明所得方案是全局最优。从多算法对比看，混合算法在基准权重下 6 次独立运行的目标值标准差为 4.3，只有 2 次达到 22 个架次，单次结果对随机种子敏感；本文用五组权重共 18 次运行取最好值来弥补，但与真正的最优值之间可能仍有差距。问题三的联合搜索更不稳定，7 次运行中只有 4 次同时做到通信与时限零违约。

最后是通信模型的简化。本文按自由空间损耗加固定遮挡损耗判定链路，未考虑降雨衰减、多径效应与同频干扰。洪涝灾害现场往往伴随强降雨，2.4 GHz 频段的雨衰虽不严重，但地表积水与植被含水量变化会改变反射特性，实际可用距离可能低于本文的计算值。

8.3 模型的改进

针对能耗高估的问题，可以在保持题目计算规则的前提下引入中间航路点。把一条直线航段拆成两段折线，让折点避开沿线的孤立高点，即可显著降低巡航海拔。这一改进只

需在航段几何的计算中加入一层路径搜索，能耗模型本身不必改动，代价是航程略有增加，两者的权衡可以用一个带高程惩罚的最短路问题统一刻画。

针对中继调度，可以放开悬停位置在服务期间固定这一假设。若允许中继在值守期间沿一条短弧线缓慢移动，其覆盖范围可以随被保障对象的位置滚动前移，同一架中继能服务的航段数会增加，中继架次数有望进一步下降。相应地，模型需要把悬停点由一个定点改为一条轨迹，链路判定也要沿轨迹逐点进行。

针对时限的刻画，可以把当前的硬约束加线性惩罚改为随时间递增的凸惩罚。应急场景下物资晚到一分钟与晚到一小时的后果差别是非线性的，凸惩罚更贴近实际需求，也能让搜索在接近时限时自动加大避让力度。

8.4 模型的推广

本文建立的框架不局限于无人机运输。其核心结构是在地形约束下同时安排作业单元与保障单元，并用能源周转把空间路径与时间排程耦合起来，这一结构可以直接推广到以下场景。

一是山区电力与通信设施的抢修调度。抢修队的行进路线受地形制约，携带的物料有重量上限，作业期间同样需要保持与指挥中心的联络，把本文的航段几何换成道路可达性、把电池换成车辆油料，模型形式不变。

二是应急医疗的空中转运。伤员转运对时限的要求更为严苛，而直升机的载重与航程同样随载荷衰减，本文的最大安全载荷模型与时限惩罚可以直接套用，中继调度则对应临时空中指挥平台的部署。

三是大范围农林作业的机群调度。植保无人机的作业同样受地块分布、药液载重与电池周转制约，本文关于电池是并行能力瓶颈的结论以及两阶段充电的分析在该场景中同样成立，只需把时限约束换成作业窗口。

参考文献

[1] 新华社，记者手记：抵近广西横州镇龙乡，https://www.xinhuanet.com/politics/20260709/acf8e4b353304bb78007d8224f1cd2ef/c.html，2026 年 9 月 20 日。

[2] Copernicus Data Space Ecosystem, Copernicus DEM: Global and European Digital Elevation Model, https://dataspace.copernicus.eu, 2026 年 9 月 20 日.

[3] Dorling K, Heinrichs J, Messier G G, et al., Vehicle Routing Problems for Drone Delivery, IEEE Transactions on Systems, Man, and Cybernetics: Systems, 47(1): 70-85, 2017.

[4] Zhang J, Campbell J F, Sweeney D C, et al., Energy Consumption Models for Delivery Drones: A Comparison and Assessment, Transportation Research Part D: Transport and Environment, 90: 102668, 2021.

[5] Perron L, Furnon V, OR-Tools CP-SAT Solver, https://developers.google.com/optimization, 2026 年 9 月 20 日.

[6] Laporte G, Fifty Years of Vehicle Routing, Transportation Science, 43(4): 408-416, 2009.

[7] Texas Instruments, Li-Ion Battery Charger Solution Using an MSP430 MCU, https://www.ti.com/lit/an/slaa287b/slaa287b.pdf, 2026 年 9 月 20 日.

[8] Shaw P, Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems, Principles and Practice of Constraint Programming, Lecture Notes in Computer Science 1520, Berlin: Springer, 417-431, 1998.

[9] Ropke S, Pisinger D, An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows, Transportation Science, 40(4): 455-472, 2006.

[10] Kirkpatrick S, Gelatt C D, Vecchi M P, Optimization by Simulated Annealing, Science, 220(4598): 671-680, 1983.

[11] Zeng Y, Zhang R, Lim T J, Wireless Communications with Unmanned Aerial Vehicles: Opportunities and Challenges, IEEE Communications Magazine, 54(5): 36-42, 2016.

[12] International Telecommunication Union, Recommendation ITU-R P.525-5: Calculation of Free-Space Attenuation, Geneva: ITU, 2024.

[13] Chvatal V, A Greedy Heuristic for the Set-Covering Problem, Mathematics of Operations Research, 4(3): 233-235, 1979.

附　录

附录 1　支撑材料文件清单

提交的支撑材料按下列结构组织。

```
支撑材料/
├── 结果文件.xlsx                  按官方模板填写的六张结果表
│   ├── Q1_单点组批                18 行
│   ├── Q2_运输架次                22 行
│   ├── Q2_逐箱交付                80 行
│   ├── Q3_中继架次                 5 行
│   ├── Q3_通信保障               385 行
│   └── Q4_分区配置                 5 行
├── 检查说明.md                    四问的核验办法与核验结果
├── 代码/                          可运行程序，运行方式见检查说明
│   ├── common/
│   │   ├── paths.py               数据与结果目录的定位
│   │   ├── data.py                读附件五个数据表
│   │   ├── terrain.py             DEM 读取、投影、航段剖面、视线遮挡
│   │   ├── flight.py              等效航程、航段时间与能耗、最大安全载荷
│   │   ├── comm.py                链路预算、传播损耗、航迹采样与直连判定
│   │   └── q2model.py             架次评价、资源周转仿真、目标函数
│   ├── q0_check.py                附件数据的读入检查
│   ├── q1.py                      问题一：载荷与组批的约束规划求解
│   ├── q1_dp.py                   问题一：子集动态规划的独立重解
│   ├── q2.py                      问题二：混合启发式主体
│   ├── q2_ga.py                   问题二：遗传算法
│   ├── q2_ga_run.sh               问题二：遗传算法多种子跑批
│   ├── q2_run.py                  问题二：多种子多权重跑批
│   ├── q2_pick.py                 问题二：结果汇总与最优解选取
│   ├── q2_ablation.sh             问题二：多算法对比跑批
│   ├── q2_ablation_sum.py         问题二：多算法对比汇总
│   ├── q2_smoke.py                问题二：小规模冒烟测试
│   ├── q3_geom.py                 问题三：候选点枚举与集合覆盖选址
│   ├── q3_joint.py                问题三：联合调度主体
│   ├── q3_run.py                  问题三：多种子多权重跑批
```

```
│   ├── q3_pick.py          问题三：班次修剪与最优解选取
│   ├── q3_relay.py         问题三：中继航段时间与能耗
│   ├── q3_analyze.py       问题三：直连与失联时段统计
│   ├── q3_verify.py        问题三：独立核验程序
│   ├── q4.py               问题四：原子块识别与分区资源核算
│   ├── export.py           按官方模板导出结果文件
│   └── srv.py              计算任务的远程执行与结果回传
└── 中间结果/
    ├── q1/                 载荷表、组批表、权衡与灵敏度
    ├── q2/                 各权重各种子的方案与汇总
    ├── q3/                 几何预计算、联合方案与核验记录
    └── q4/                 原子块、分区枚举与资源核算
```

附录 2　机型与链路参数取值

四问计算所用的机型参数与链路参数取自附件，汇总列于表 20 与表 21。

表 20　运输无人机机型参数

参数	单位	A 型	B 型	C 型
含电池空载总质量	kg	70	65	69.9
最大载货质量	kg	25	30	80
可用装载体积	m^3	0.06	0.073	0.25
计划巡航速度	m/s	12	15	15
空载标准航程	m	25000	28000	26000
满载标准航程	m	20000	16000	12000
电池可用能量	kWh	4.5	4	8
最大爬升、下降速度	m/s	3、2.5	3、2.5	2.5、2
接收点基础交接时间	s	150	150	180
每箱增加交接时间	s	30	30	36
实体机架数	架	4	2	2
共享电池组数	组	6	4	4
等效完全充电时间	s	1800	2400	3000

三型相同的取值：返航电量下限 20%，爬升能耗效率 0.72，下降能耗效率 0，工位固定准备时间 300 s，每箱装载时间 30 s。

附录 3　人工智能工具使用说明

本文使用人工智能工具辅助编程，用于代码编写、调试与结果整理，工具信息列于表 22。

按《人工智能工具及输出使用规定》逐条说明如下。第一条与第二条：人工智能工具只用在编程环节，问题分析、数学模型的建立、口径假设、算法设计与结果判断全部由参赛队独立完成，正文文字由参赛队自行撰写。第三条：正文中的模型与公式或来自题目给定条件，或来自表 22 以外的正式发表文献，没有采用无法确认来源的模型与公式，参考文献中因此不列人工智能工具。第四条：全部数值结果由参赛队自己的程序运行得到，不是人工智能工具的输出，无需另行标注。第五条：支撑材料中每个程序的开头都写有声明注释，标明工具名称、版本、开发机构与版本发布日期。第六条：赛题未要求提交人工智能工具的输入内容与后处理策略，本文的求解程序与中间结果已随支撑材料一并提交，可完整复现全部结果。

表 21　中继无人机与通信链路参数

类别	参数	单位	取值
中继	含能源组件空载总质量	kg	21
中继	中继通信模块质量	kg	2.5
中继	计划起飞总质量	kg	23.5
中继	计划巡航速度	m/s	15
中继	巡航功率	kW	1.15
中继	悬停功率	kW	1.05
中继	通信附加功率	kW	0.05
中继	能源组件可用能量	kWh	3.2
中继	最大爬升、下降速度	m/s	4、3
中继	工位固定准备时间	s	180
中继	建链时间	s	30
中继	架次周转时间	s	300
中继	最大悬停离地高度	m	300
中继	机架数、能源组件总数	架、组	2、6
中继	等效完全充电时间	s	1800
通信	载波频率	MHz	2400
通信	系统损耗	dB	3
通信	地形遮挡附加损耗	dB	10
通信	接收灵敏度	dBm	−98
通信	衰落裕量	dB	8
通信	运输无人机发射功率、天线增益	dBm、dBi	20、3
通信	中继接入端发射功率、天线增益	dBm、dBi	20、6
通信	中继回传端发射功率、天线增益	dBm、dBi	19、8
通信	网关发射功率、天线增益	dBm、dBi	27、12
通信	网关天线离地高度	m	20

中继返航电量下限 20%，爬升能耗效率 0.72，下降能耗效率 0。

表 22　人工智能工具使用情况

工具名称	版本	开发机构	使用环节
DeepSeek	DeepSeek-V3	深度求索	辅助编程与调试

附录 4　程序清单

下面按支撑材料 `代码/` 目录的顺序列出全部可运行程序。每个程序开头的声明注释按《人工智能工具及输出使用规定》第五条标明所用工具。各文件内容互不重复：共用层 `common/` 只出现一次，四问的求解程序、跑批脚本与核验程序逐一列出；问题三的核验程序 `q3_verify.py` 出于复核独立性，不引用求解程序的任何函数，另行实现通信状态判定与资源占用检查，因此其中少量循环与求解程序写法相似，但该文件是独立实现，不属于重复代码。

common/paths.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import os
```

```
import sys

def _u(p):
    if sys.version_info[0] == 2 and isinstance(p, bytes):
        return p.decode("mbcs")
    return p

CODE = os.path.dirname(os.path.dirname(os.path.abspath(_u(__file__))))
ROOT = os.path.dirname(CODE)
RESULTS = os.path.join(ROOT, u"results")
GIS = os.path.join(ROOT, u"gis")

def _first(*cands):
    for c in cands:
        if os.path.isdir(c):
            return c
    return cands[-1]

SAITI = _first(os.path.join(ROOT, u"赛题"),
               os.path.join(os.path.dirname(os.path.dirname(ROOT)), u"赛题", u"D题"))
DATA = os.path.join(SAITI, u"数据")
DEM_TIF = os.path.join(DATA, u"镇龙乡地理空间数据", u"镇龙乡及周边地理数据",
                       u"数字高程模型数据（DEM）", u"镇龙乡及周边30米DEM.tif")

FIGS = _first(os.path.join(os.path.dirname(ROOT), u"初稿", u"figs"), os.path.join(
    ROOT, u"figs"))
SHOTS = _first(os.path.join(os.path.dirname(ROOT), u"初稿", u"截图"), os.path.join(
    ROOT, u"截图"))
```

common/data.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构深度求索，版本发布日期 2026 年。

from dataclasses import dataclass, field
from pathlib import Path
import pandas as pd
from . import paths

ROOT = Path(__file__).resolve().parents[2]
DATA = Path(paths.DATA)
BASE = DATA / "无人机应急物资运输基础数据"
GEO = DATA / "镇龙乡地理空间数据" / "镇龙乡及周边地理数据"
DEM_TIF = GEO / "数字高程模型数据（DEM）" / "镇龙乡及周边30米DEM.tif"

@dataclass
class Node:
    id: str
    name: str
    lon: float
    lat: float
    ground: float
    work_alt: float
    pop: int = 0

@dataclass
class DroneType:
    id: str
    name: str
    m_empty: float
    Q: float
    V: float
    v_cruise: float
    L0: float
```

```
    LF: float
    E_use: float
    rho: float
    t_prep: float
    t_load_box: float
    t_hand_base: float
    t_hand_box: float
    v_up: float
    v_down: float
    eta_up: float
    eta_down: float
    n_batt: int = 0
    T_full: float = 0
    drones: list = field(default_factory=list)

@dataclass
class RelayType:
    id: str
    name: str
    m_empty: float
    m_module: float
    m_takeoff: float
    v_cruise: float
    P_cruise: float
    E_use: float
    rho: float
    t_prep: float
    t_link: float
    t_turn: float
    v_up: float
    v_down: float
    eta_up: float
    eta_down: float
    P_hover: float
    P_comm: float
    h_max: float
    n_pack: int = 0
    T_full: float = 0
    drones: list = field(default_factory=list)

@dataclass
class Box:
    id: str
    area: str
    kind: str
    mass: float
    vol: float
    first: bool
    t_first: float
    t_expect: float
    prio: float

@dataclass
class Comm:
    f_MHz: float
    L_sys: float
    L_obs: float
    P_sens: float
    M: float
    Pt_uav: float
    G_uav: float
    Pt_relay_acc: float
    G_relay_acc: float
    Pt_relay_bh: float
    G_relay_bh: float
```

```
    Pt_gw: float
    G_gw: float
    h_gw: float

def _f(x):
    return float(x)

def load_nodes():
    df = pd.read_excel(BASE / "调度中心与服务区.xlsx", header=None)
    r = df.iloc[2]
    depot = Node(str(r[0]), str(r[1]), _f(r[2]), _f(r[3]), _f(r[4]), _f(r[4]))
    areas = []
    for k in range(6, len(df)):
        r = df.iloc[k]
        if pd.isna(r[0]):
            continue
        g = _f(r[4])
        areas.append(Node(str(r[0]), str(r[1]), _f(r[2]), _f(r[3]), g, g + 30.0, int(
            r[5])))
    return depot, areas

def load_drone_types():
    df = pd.read_excel(BASE / "运输无人机数据.xlsx", header=None)
    types = {}
    for k in (2, 3, 4):
        r = df.iloc[k]
        t = DroneType(str(r[0]), str(r[1]), _f(r[2]), _f(r[3]), _f(r[4]), _f(r[5]),
            _f(r[6]), _f(r[7]),
                      _f(r[8]), _f(r[9]) / 100.0, _f(r[10]), _f(r[11]), _f(r[12]), _f
                          (r[13]),
                      _f(r[14]), _f(r[15]), _f(r[16]), _f(r[17]))
        types[t.id] = t
    for k in range(8, 16):
        r = df.iloc[k]
        if pd.isna(r[0]):
            continue
        types[str(r[1])].drones.append(str(r[0]))
    for k in range(19, 22):
        r = df.iloc[k]
        types[str(r[0])].n_batt = int(r[1])
        types[str(r[0])].T_full = _f(r[2])
    return types

def load_relay_type():
    df = pd.read_excel(BASE / "中继无人机数据.xlsx", header=None)
    r = df.iloc[2]
    t = RelayType(str(r[0]), str(r[1]), _f(r[2]), _f(r[3]), _f(r[4]), _f(r[5]), _f(r
        [6]), _f(r[7]),
                  _f(r[8]) / 100.0, _f(r[9]), _f(r[10]), _f(r[11]), _f(r[12]), _f(r
                      [13]), _f(r[14]),
                  _f(r[15]), _f(r[16]), _f(r[17]), _f(r[18]))
    for k in (6, 7):
        t.drones.append(str(df.iloc[k][0]))
    r = df.iloc[11]
    t.n_pack = int(r[1])
    t.T_full = _f(r[2])
    return t

def load_boxes():
    df = pd.read_excel(BASE / "物资需求与配送时限.xlsx", sheet_name="逐箱货箱清单")
    boxes = []
    for _, r in df.iterrows():
        tf = r.iloc[6]
        boxes.append(Box(str(r.iloc[0]), str(r.iloc[1]), str(r.iloc[2]), _f(r.iloc
            [3]), _f(r.iloc[4]),
```

```
                            str(r.iloc[5]) == "是", float("inf") if pd.isna(tf) else _f(
                                tf),
                            _f(r.iloc[7]), _f(r.iloc[8])))
    return boxes

def load_comm():
    df = pd.read_excel(BASE / "通信链路参数.xlsx", header=None)
    v = [_f(df.iloc[k][4]) for k in range(2, 16)]
    return Comm(*v)

if __name__ == "__main__":
    d, a = load_nodes()
    print(d)
    print(len(a), a[0], a[-1])
    for t in load_drone_types().values():
        print(t)
    print(load_relay_type())
    b = load_boxes()
    print(len(b), b[0])
    print(load_comm())
```

common/terrain.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import numpy as np
import rasterio
from pyproj import Transformer
from .data import DEM_TIF

STEP = 5.0
CLEAR = 50.0

_to_utm = Transformer.from_crs("EPSG:4326", "EPSG:32649", always_xy=True)
_to_ll = Transformer.from_crs("EPSG:32649", "EPSG:4326", always_xy=True)

class Terrain:
    def __init__(self, path=DEM_TIF):
        with rasterio.open(path) as d:
            self.z = d.read(1).astype(np.float64)
            self.transform = d.transform
            self.bounds = d.bounds
            self.nrow, self.ncol = self.z.shape
        self.x0 = self.transform.c
        self.y0 = self.transform.f
        self.dx = self.transform.a
        self.dy = self.transform.e
        self._cache = {}

    def rc(self, lon, lat):
        col = np.floor((np.asarray(lon) - self.x0) / self.dx).astype(int)
        row = np.floor((np.asarray(lat) - self.y0) / self.dy).astype(int)
        col = np.clip(col, 0, self.ncol - 1)
        row = np.clip(row, 0, self.nrow - 1)
        return row, col

    def elev(self, lon, lat):
        r, c = self.rc(lon, lat)
        return self.z[r, c]

    def inside(self, lon, lat):
        b = self.bounds
        return b.left <= lon <= b.right and b.bottom <= lat <= b.top

```

```
    @staticmethod
    def xy(lon, lat):
        return _to_utm.transform(lon, lat)

    @staticmethod
    def ll(x, y):
        return _to_ll.transform(x, y)

    def horiz_dist(self, a, b):

        x1, y1 = self.xy(*a)
        x2, y2 = self.xy(*b)
        return float(np.hypot(x2 - x1, y2 - y1))

    def profile(self, a, b, step=STEP):

        x1, y1 = self.xy(*a)
        x2, y2 = self.xy(*b)
        d = float(np.hypot(x2 - x1, y2 - y1))
        n = max(int(np.ceil(d / step)), 1)
        s = np.linspace(0.0, d, n + 1)
        xs = x1 + (x2 - x1) * s / d if d > 0 else np.full_like(s, x1)
        ys = y1 + (y2 - y1) * s / d if d > 0 else np.full_like(s, y1)
        lon, lat = self.ll(xs, ys)
        return s, self.elev(lon, lat), d

    def cruise_alt(self, a, b):

        key = (a, b) if a <= b else (b, a)
        if key not in self._cache:
            _, z, d = self.profile(a, b)
            self._cache[key] = (float(z.max()) + CLEAR, d)
        return self._cache[key]

    def los_blocked(self, p, q, step=STEP):

        s, z, d = self.profile(p[:2], q[:2], step)
        if d <= 0:
            return False
        line = p[2] + (q[2] - p[2]) * s / d
        return bool(np.any(line[1:-1] < z[1:-1]))
```

common/flight.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构深度求索，版本发布日期 2026 年。

import math
from dataclasses import dataclass
from .data import DroneType, Node

G = 9.81
J2KWH = 1.0 / 3.6e6

def eq_range(t: DroneType, q: float) -> float:
    q = min(max(q, 0.0), t.Q)
    return t.L0 - (t.L0 - t.LF) * (q / t.Q) ** 1.5

@dataclass
class Leg:
    d: float
    h_up: float
    h_down: float
    z_cruise: float
    t: float
```

```
    E_hor: float
    E_up: float

    @property
    def E(self):
        return self.E_hor + self.E_up

def leg(t: DroneType, terrain, a: Node, b: Node, q: float) -> Leg:
    z_c, d = terrain.cruise_alt((a.lon, a.lat), (b.lon, b.lat))
    h_up = max(z_c - a.work_alt, 0.0)
    h_down = max(z_c - b.work_alt, 0.0)
    tt = h_up / t.v_up + d / t.v_cruise + h_down / t.v_down
    e_hor = t.E_use * d / eq_range(t, q)
    e_up = (t.m_empty + q) * G * h_up / t.eta_up * J2KWH
    return Leg(d, h_up, h_down, z_c, tt, e_hor, e_up)

def sortie_budget(t: DroneType) -> float:
    return (1.0 - t.rho) * t.E_use

def round_trip(t: DroneType, terrain, depot: Node, area: Node, q: float, n_box: int):

    out = leg(t, terrain, depot, area, q)
    back = leg(t, terrain, area, depot, 0.0)
    E = out.E + back.E
    t_fly = out.t + back.t
    t_ground = t.t_prep + t.t_load_box * n_box + t.t_hand_base + t.t_hand_box * n_box
    soc = 1.0 - E / t.E_use
    return dict(E=E, t_fly=t_fly, t_total=t_fly + t_ground, soc=soc, out=out, back=
        back)

def max_safe_payload(t: DroneType, terrain, depot: Node, area: Node, rho=None, tol=1e
    -3):

    rho = t.rho if rho is None else rho
    budget = (1.0 - rho) * t.E_use

    def E(q):
        return leg(t, terrain, depot, area, q).E + leg(t, terrain, area, depot, 0.0).
            E

    if E(t.Q) <= budget:
        return t.Q
    if E(0.0) > budget:
        return 0.0
    lo, hi = 0.0, t.Q
    while hi - lo > tol:
        mid = 0.5 * (lo + hi)
        if E(mid) <= budget:
            lo = mid
        else:
            hi = mid
    return lo
```

common/comm.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
    深度求索，版本发布日期 2026 年。

import math
import numpy as np
from .data import load_comm
from .q2model import NODES, legs, types, sortie_eval, T, depot

C = load_comm()
P_TH = C.P_sens + C.M
```

```
10
11 def _lmax(pt_a, gt_a, gr_b):
12     return pt_a + gt_a + gr_b - C.L_sys - P_TH
13
14 THR_DIRECT = min(_lmax(C.Pt_uav, C.G_uav, C.G_gw), _lmax(C.Pt_gw, C.G_gw, C.G_uav))
15 THR_ACCESS = min(_lmax(C.Pt_uav, C.G_uav, C.G_relay_acc), _lmax(C.Pt_relay_acc, C.
       G_relay_acc, C.G_uav))
16 THR_BACKHAUL = min(_lmax(C.Pt_relay_bh, C.G_relay_bh, C.G_gw), _lmax(C.Pt_gw, C.G_gw,
        C.G_relay_bh))
17 FSPL0 = 32.45 + 20 * math.log10(C.f_MHz)
18
19 def max_range_km(thr, blocked):
20     return 10 ** ((thr - FSPL0 - (C.L_obs if blocked else 0)) / 20)
21
22 GW = (depot.lon, depot.lat, depot.ground + C.h_gw)
23
24 def dist3d(p, q):
25     x1, y1 = T.xy(p[0], p[1])
26     x2, y2 = T.xy(q[0], q[1])
27     return math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2 + (p[2] - q[2]) ** 2)
28
29 def path_loss(p, q):
30     D = dist3d(p, q) / 1000.0
31     if D < 1e-6:
32         return 0.0, False
33     b = T.los_blocked(p, q)
34     return FSPL0 + 20 * math.log10(D) + (C.L_obs if b else 0.0), b
35
36 def available(p, q, thr):
37     L, b = path_loss(p, q)
38     return L <= thr, L, b
39
40 def trajectory(g, stops, start, dt=5.0):
41
42     t = types[g]
43     all_boxes = [b for _, bx in stops for b in bx]
44     pts = []
45     clock = start
46
47     t_ground = t.t_prep + t.t_load_box * len(all_boxes)
48     pts.append((clock, "ground", depot.lon, depot.lat, depot.ground, 0))
49     clock += t_ground
50     prev = 0
51     route = [(a, len(bx)) for a, bx in stops] + [(0, 0)]
52     for area, nb in route:
53         d, hu, hd, zc = legs[prev][area]
54         A, B = NODES[prev], NODES[area]
55
56         tu = hu / t.v_up
57         for s in np.arange(0, tu, dt):
58             pts.append((clock + s, "climb", A.lon, A.lat, A.work_alt + t.v_up * s,
                   prev))
59         clock += tu
60
61         tc = d / t.v_cruise
62         for s in np.arange(0, tc, dt):
63             f = s / tc if tc > 0 else 0
64             pts.append((clock + s, "cruise", A.lon + (B.lon - A.lon) * f, A.lat + (B.
                   lat - A.lat) * f, zc, prev))
65         clock += tc
66
67         td = hd / t.v_down
68         for s in np.arange(0, td, dt):
69             pts.append((clock + s, "descent", B.lon, B.lat, zc - t.v_down * s, area))
70         clock += td
```

```
        if area != 0:
            th = t.t_hand_base + t.t_hand_box * nb
            for s in np.arange(0, th, dt):
                pts.append((clock + s, "handover", B.lon, B.lat, B.work_alt, area))
            clock += th
        prev = area
    pts.append((clock, "ground", depot.lon, depot.lat, depot.ground, 0))
    return pts

def direct_status(pts):

    out = []
    for (tt, ph, lon, lat, alt, node) in pts:
        if ph == "ground":
            out.append((tt, ph, lon, lat, alt, node, True, 0.0, False))
            continue
        ok, L, b = available((lon, lat, alt), GW, THR_DIRECT)
        out.append((tt, ph, lon, lat, alt, node, ok, L, b))
    return out

if __name__ == "__main__":
    print("门限 直连", THR_DIRECT, "接入", THR_ACCESS, "回传", THR_BACKHAUL)
    for thr, n in ((THR_DIRECT, "直连"), (THR_ACCESS, "接入"), (THR_BACKHAUL, "回传")
        ):
        print(n, "视距 km", round(max_range_km(thr, False), 2), "遮挡 km", round(
            max_range_km(thr, True), 2))
```

common/q2model.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import math
from functools import lru_cache
from .data import load_nodes, load_drone_types, load_boxes
from .terrain import Terrain
from .flight import eq_range, G, J2KWH

depot, areas = load_nodes()
types = load_drone_types()
boxes = load_boxes()
T = Terrain()
NODES = [depot] + areas
AREA_IDX = {a.id: i + 1 for i, a in enumerate(areas)}
BOX_AREA = [AREA_IDX[b.area] for b in boxes]
TYPE_IDS = list(types)
NB = len(boxes)

import json as _json
from pathlib import Path as _Path
_CL_FILE = _Path(__file__).resolve().parents[2] / "results" / "clusters.json"

def _make_clusters(k=3):
    from sklearn.cluster import KMeans
    xy = [T.xy(a.lon, a.lat) for a in areas]
    km = KMeans(n_clusters=k, n_init=20, random_state=0).fit(xy)
    lab = km.labels_.tolist()

    order = sorted(set(lab), key=lambda c: min(areas[i].lon for i in range(len(areas)
        ) if lab[i] == c))
    ren = {c: n for n, c in enumerate(order)}
    return {areas[i].id: ren[lab[i]] for i in range(len(areas))}

if _CL_FILE.exists():
```

```
    CLUSTER = _json.load(open(_CL_FILE, encoding="utf-8"))
else:
    CLUSTER = _make_clusters()
    _CL_FILE.parent.mkdir(parents=True, exist_ok=True)
    _json.dump(CLUSTER, open(_CL_FILE, "w", encoding="utf-8"), ensure_ascii=False,
        indent=1)
NODE_CLUSTER = [-1] + [CLUSTER[a.id] for a in areas]

legs = [[None] * len(NODES) for _ in NODES]
for i, a in enumerate(NODES):
    for j, b in enumerate(NODES):
        if i == j:
            legs[i][j] = (0.0, 0.0, 0.0, a.work_alt)
            continue
        z_c, d = T.cruise_alt((a.lon, a.lat), (b.lon, b.lat))
        legs[i][j] = (d, max(z_c - a.work_alt, 0.0), max(z_c - b.work_alt, 0.0), z_c)

def leg_time(g, i, j):
    t = types[g]
    d, hu, hd, _ = legs[i][j]
    return hu / t.v_up + d / t.v_cruise + hd / t.v_down

def leg_energy(g, i, j, q):
    t = types[g]
    d, hu, _, _ = legs[i][j]
    return t.E_use * d / eq_range(t, q) + (t.m_empty + q) * G * hu / t.eta_up * J2KWH

def t_charge(soc, T_full):
    if soc < 0.9:
        return T_full * (0.65 * (0.9 - soc) / 0.9 + 0.35)
    return T_full * 0.35 * (1.0 - soc) / 0.1

@lru_cache(maxsize=200000)
def sortie_eval(g, stops):

    t = types[g]
    all_boxes = [b for _, bx in stops for b in bx]
    mass = sum(boxes[b].mass for b in all_boxes)
    vol = sum(boxes[b].vol for b in all_boxes)
    if mass > t.Q + 1e-9 or vol > t.V + 1e-9 or not all_boxes:
        return (False, 0, 0, 0, {}, [])
    if len({NODE_CLUSTER[a] for a, _ in stops}) > 1:
        return (False, 0, 0, 0, {}, [])
    q = mass
    E = 0.0
    clock = t.t_prep + t.t_load_box * len(all_boxes)
    prev = 0
    deliv = {}
    detail = []
    for area, bx in stops:
        e = leg_energy(g, prev, area, q)
        tt = leg_time(g, prev, area)
        E += e
        clock += tt
        detail.append((prev, area, q, tt, e, clock))
        clock += t.t_hand_base + t.t_hand_box * len(bx)
        for b in bx:
            deliv[b] = clock
        q -= sum(boxes[b].mass for b in bx)
        prev = area
    e = leg_energy(g, prev, 0, 0.0)
    tt = leg_time(g, prev, 0)
    E += e
    clock += tt
    detail.append((prev, 0, 0.0, tt, e, clock))
```

```
    ok = E <= (1.0 - t.rho) * t.E_use + 1e-9
    soc = 1.0 - E / t.E_use
    return (ok, clock, E, soc, deliv, detail)

def schedule(sorties):

    avail = {g: {d: 0.0 for d in types[g].drones} for g in TYPE_IDS}
    ready = {g: {f"B{g}{k+1:02d}": 0.0 for k in range(types[g].n_batt)} for g in
        TYPE_IDS}
    plan = []
    for g, stops in sorties:
        ok, dur, E, soc, deliv, _ = sortie_eval(g, stops)
        d = min(avail[g], key=avail[g].get)
        b = min(ready[g], key=ready[g].get)
        start = max(avail[g][d], ready[g][b])
        ret = start + dur
        avail[g][d] = ret
        ready[g][b] = ret + t_charge(soc, types[g].T_full)
        plan.append((d, b, start, ret))
    return plan

W = dict(late=5.0, hard=50.0, mk=3.0, E=1.0, N=1.0)

def evaluate(sorties, w=W):
    plan = schedule(sorties)
    late = 0.0
    hard = 0.0
    E = 0.0
    mk = 0.0
    infeasible = 0
    deliv_t = {}
    for (g, stops), (d, b, start, ret) in zip(sorties, plan):
        ok, dur, e, soc, deliv, _ = sortie_eval(g, stops)
        if not ok:
            infeasible += 1
        E += e
        mk = max(mk, ret)
        for bi, off in deliv.items():
            tb = start + off
            deliv_t[bi] = tb
            bx = boxes[bi]
            if tb > bx.t_first:
                hard += bx.prio * (tb - bx.t_first) / 1000.0 + 1.0
            if bx.kind == "医疗物资" and tb > bx.t_expect:
                hard += bx.prio * (tb - bx.t_expect) / 1000.0 + 1.0
            elif tb > bx.t_expect:
                late += bx.prio * (tb - bx.t_expect) / 1000.0
    J = w["late"] * late + w["hard"] * hard + w["mk"] * mk / 1000.0 + w["E"] * E + w
        ["N"] * len(sorties) + 1e6 * infeasible
    return J, dict(late=late, hard=hard, mk=mk, E=E, N=len(sorties), infeasible=
        infeasible, deliv=deliv_t, plan=plan)

def check(sorties):

    seen = {}
    for si, (g, stops) in enumerate(sorties):
        for _, bx in stops:
            for b in bx:
                assert b not in seen, f"box {b} twice"
                seen[b] = si
    assert len(seen) == NB, f"{NB - len(seen)} boxes missing"
    plan = schedule(sorties)
    busy = {}
    for (g, stops), (d, b, s, r) in zip(sorties, plan):
        ok, dur, E, soc, deliv, _ = sortie_eval(g, stops)
```

```
        assert ok, f"sortie {stops} infeasible"
        assert soc >= types[g].rho - 1e-9
        busy.setdefault(d, []).append((s, r))
        busy.setdefault(b, []).append((s, r + t_charge(soc, types[g].T_full)))
    for k, iv in busy.items():
        iv.sort()
        for (s1, r1), (s2, r2) in zip(iv, iv[1:]):
            assert s2 >= r1 - 1e-6, f"{k} overlap"
    return True
```

q0_check.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构深度求索，版本发布日期 2026 年。

import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
import pandas as pd
from common.data import load_nodes, load_drone_types, load_boxes
from common.terrain import Terrain
from common.flight import round_trip, max_safe_payload, leg

depot, areas = load_nodes()
types = load_drone_types()
boxes = load_boxes()
T = Terrain()
print("DEM 内所有节点:", all(T.inside(n.lon, n.lat) for n in [depot] + areas))
print("O01 DEM 高程", round(float(T.elev(depot.lon, depot.lat)), 1), "表", depot.ground)

rows = []
for a in areas:
    z_c, d = T.cruise_alt((depot.lon, depot.lat), (a.lon, a.lat))
    demand_m = sum(b.mass for b in boxes if b.area == a.id)
    demand_v = sum(b.vol for b in boxes if b.area == a.id)
    n = sum(1 for b in boxes if b.area == a.id)
    r = dict(area=a.id, d_km=round(d / 1000, 3), z_cruise=round(z_c, 1), h_up=round(z_c - depot.work_alt, 1),
             h_down=round(z_c - a.work_alt, 1), n_box=n, mass=demand_m, vol=round(demand_v, 3))
    for g, t in types.items():
        qs = max_safe_payload(t, T, depot, a)
        rt = round_trip(t, T, depot, a, qs, 1)
        r[f"qsafe_{g}"] = round(qs, 2)
        r[f"E_{g}"] = round(rt["E"], 3)
        r[f"tfly_{g}"] = round(rt["t_fly"])
        r[f"soc_{g}"] = round(rt["soc"] * 100, 1)
    rows.append(r)
df = pd.DataFrame(rows)
pd.set_option("display.width", 250)
pd.set_option("display.max_columns", 40)
print(df.to_string(index=False))
print("总箱数", len(boxes), "总质量", sum(b.mass for b in boxes), "总体积", round(sum(b.vol for b in boxes), 3))

t = types["A"]
lg = leg(t, T, depot, areas[0], 25.0)
print("A 满载去 S001:", lg)
```

q1.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构深度求索，版本发布日期 2026 年。
```

```
2
3  import sys
4  import json
5  from pathlib import Path
6  sys.path.insert(0, str(Path(__file__).resolve().parent))
7  import numpy as np
8  import pandas as pd
9  from ortools.sat.python import cp_model
10 from common.data import load_nodes, load_drone_types, load_boxes, ROOT
11 from common.terrain import Terrain
12 from common.flight import round_trip, max_safe_payload, leg
13
14 OUT = ROOT / "results" / "q1"
15 OUT.mkdir(parents=True, exist_ok=True)
16
17 depot, areas = load_nodes()
18 types = load_drone_types()
19 boxes = load_boxes()
20 T = Terrain()
21 TYPE_IDS = list(types)
22
23 def energy_table(t, area, rho):
24
25     qs = max_safe_payload(t, T, depot, area, rho)
26     qmax = int(np.floor(qs + 1e-9))
27     tab = [0]
28     for m in range(1, int(t.Q) + 1):
29         if m <= qmax:
30             e = leg(t, T, depot, area, m).E + leg(t, T, area, depot, 0.0).E
31             tab.append(int(round(e * 1000)))
32         else:
33             tab.append(10 ** 9)
34     return qs, qmax, tab
35
36 def solve_area(area, rho, mode="lex", fixed_n=None, time_limit=60):
37
38     items = [b for b in boxes if b.area == area.id]
39     n = len(items)
40     tabs, qsafe = {}, {}
41     for g, t in types.items():
42         qs, qmax, tab = energy_table(t, area, rho)
43         tabs[g] = tab
44         qsafe[g] = qs
45
46     tfly = {g: round_trip(types[g], T, depot, area, 0.0, 0)["t_fly"] for g in types}
47     K = n
48     m = cp_model.CpModel()
49     x = {}
50     for i in range(n):
51         for g in TYPE_IDS:
52             for k in range(K):
53                 x[i, g, k] = m.NewBoolVar(f"x{i}{g}{k}")
54     u = {(g, k): m.NewBoolVar(f"u{g}{k}") for g in TYPE_IDS for k in range(K)}
55     mass = {}
56     E = {}
57     cnt = {}
58     for g in TYPE_IDS:
59         t = types[g]
60         for k in range(K):
61             mass[g, k] = m.NewIntVar(0, int(t.Q), f"m{g}{k}")
62             m.Add(mass[g, k] == sum(int(items[i].mass) * x[i, g, k] for i in range(n)
                   ))
63             vol_mL = sum(int(round(items[i].vol * 1e6)) * x[i, g, k] for i in range(n
                   ))
64             m.Add(vol_mL <= int(round(t.V * 1e6)))
```

```
            cnt[g, k] = sum(x[i, g, k] for i in range(n))
            m.Add(cnt[g, k] >= 1).OnlyEnforceIf(u[g, k])
            m.Add(cnt[g, k] == 0).OnlyEnforceIf(u[g, k].Not())
            E[g, k] = m.NewIntVar(0, 10 ** 9, f"E{g}{k}")
            m.AddElement(mass[g, k], tabs[g], E[g, k])
            m.Add(E[g, k] < 10 ** 9)
            if k > 0:
                m.Add(u[g, k] <= u[g, k - 1])
                m.Add(mass[g, k] <= mass[g, k - 1])
    for i in range(n):
        m.AddExactlyOne(x[i, g, k] for g in TYPE_IDS for k in range(K))
    n_sortie = sum(u.values())
    tot_E = sum(E.values())
    tot_T = sum(int(round(tfly[g] + types[g].t_prep + types[g].t_hand_base)) * u[g, k
        ]
                + int(round(types[g].t_load_box + types[g].t_hand_box)) * cnt[g, k]
                for g in TYPE_IDS for k in range(K))
    if fixed_n is not None:
        m.Add(n_sortie == fixed_n)
    solver = cp_model.CpSolver()
    solver.parameters.max_time_in_seconds = time_limit
    solver.parameters.num_workers = 8

    def opt(obj):
        m.Minimize(obj)
        st = solver.Solve(m)
        assert st in (cp_model.OPTIMAL, cp_model.FEASIBLE), f"{area.id} infeasible"
        return int(round(solver.ObjectiveValue()))

    if mode == "lex":
        best_n = opt(n_sortie)
        m.Add(n_sortie == best_n)
        best_E = opt(tot_E)
        m.Add(tot_E == best_E)
        opt(tot_T)
    elif mode == "energy":
        opt(tot_E)
    elif mode == "time":
        opt(tot_T)
    sorties = []
    for g in TYPE_IDS:
        for k in range(K):
            if solver.Value(u[g, k]):
                ids = [items[i] for i in range(n) if solver.Value(x[i, g, k])]
                q = sum(b.mass for b in ids)
                rt = round_trip(types[g], T, depot, area, q, len(ids))
                sorties.append(dict(area=area.id, type=g, boxes=[b.id for b in ids],
                    mass=q,
                                    vol=round(sum(b.vol for b in ids), 3), t_total=
                                        round(rt["t_total"], 1),
                                    t_fly=round(rt["t_fly"], 1), E=round(rt["E"], 4),
                                         soc=round(rt["soc"] * 100, 2)))
    sorties.sort(key=lambda s: (s["type"], -s["mass"]))
    summary = dict(area=area.id, n=len(sorties), E=round(sum(s["E"] for s in sorties)
        , 4),
                   T=round(sum(s["t_total"] for s in sorties), 1), qsafe={g: round(v,
                        2) for g, v in qsafe.items()})
    return sorties, summary

def run_all(rho, mode="lex"):
    allS, allM = [], []
    for a in areas:
        s, sm = solve_area(a, rho, mode)
        allS += s
        allM.append(sm)
```

```
    return allS, allM

if __name__ == "__main__":
    rho0 = types["A"].rho

    rows = []
    for a in areas:
        r = dict(area=a.id)
        for g, t in types.items():
            r[g] = round(max_safe_payload(t, T, depot, a), 2)
        rows.append(r)
    qdf = pd.DataFrame(rows)
    qdf.to_csv(OUT / "qsafe.csv", index=False, encoding="utf-8-sig")
    print(qdf.to_string(index=False))

    S, M = run_all(rho0, "lex")
    df = pd.DataFrame([dict(架次编号=f"Q1-{i+1:03d}", 服务区编号=s["area"], 机型编号=s["type"],
                            货箱编号列表=";".join(s["boxes"]), 总质量=s["mass"], 总体积=s["vol"],
                            往返时间=s["t_total"], 架次能耗=s["E"], 返航SOC=s["soc"])
                       for i, s in enumerate(S)])
    df.to_csv(OUT / "Q1_单点组批.csv", index=False, encoding="utf-8-sig")
    msum = pd.DataFrame(M)
    print(msum.to_string(index=False))
    tot = dict(n=len(S), E=round(sum(s["E"] for s in S), 3), T=round(sum(s["t_total"] for s in S), 1),
               by_type={g: sum(1 for s in S if s["type"] == g) for g in TYPE_IDS})
    print("字典序总计", tot)

    trade = {"lex": tot}
    for mode in ("energy", "time"):
        S2, _ = run_all(rho0, mode)
        trade[mode] = dict(n=len(S2), E=round(sum(s["E"] for s in S2), 3), T=round(sum(s["t_total"] for s in S2), 1),
                           by_type={g: sum(1 for s in S2 if s["type"] == g) for g in TYPE_IDS})
        pd.DataFrame(S2).to_csv(OUT / f"sorties_{mode}.csv", index=False, encoding="utf-8-sig")
        print(mode, trade[mode])
    json.dump(trade, open(OUT / "tradeoff.json", "w", encoding="utf-8"), ensure_ascii=False, indent=1)

    prow = []
    for a in areas:
        _, sm = solve_area(a, rho0, "lex")
        nmin = sm["n"]
        for extra in range(0, 3):
            try:
                s, sm2 = solve_area(a, rho0, "energy", fixed_n=nmin + extra)
                prow.append(dict(area=a.id, n=nmin + extra, E=sm2["E"], T=sm2["T"],
                                 types="".join(sorted(x["type"] for x in s))))
            except AssertionError:
                pass
    pd.DataFrame(prow).to_csv(OUT / "pareto_by_area.csv", index=False, encoding="utf-8-sig")

    srow = []
    for rho in (0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45):
        qs = {g: [round(max_safe_payload(t, T, depot, a, rho), 2) for a in areas] for g, t in types.items()}
        S3, bad = [], []
        for a in areas:
            try:
                s, _ = solve_area(a, rho, "lex")
```

```
                S3 += s
            except AssertionError:
                bad.append(a.id)
        srow.append(dict(rho=rho, n=len(S3), E=round(sum(s["E"] for s in S3), 3),
                         T=round(sum(s["t_total"] for s in S3), 1), infeasible=";".
                             join(bad),
                         **{f"n_{g}": sum(1 for s in S3 if s["type"] == g) for g in
                             TYPE_IDS},
                         **{f"qmin_{g}": min(qs[g]) for g in TYPE_IDS},
                         **{f"qmean_{g}": round(float(np.mean(qs[g])), 2) for g in
                             TYPE_IDS}))
        pd.DataFrame({"area": [a.id for a in areas], **qs}).to_csv(OUT / f"qsafe_rho{
            int(rho*100)}.csv", index=False, encoding="utf-8-sig")
        print(srow[-1])
    pd.DataFrame(srow).to_csv(OUT / "sensitivity_rho.csv", index=False, encoding="utf
        -8-sig")
    print("done")
```

q1_dp.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
    深度求索，版本发布日期 2026 年。

import json
import sys
import time
from pathlib import Path

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))
import numpy as np
from common.data import load_nodes, load_drone_types, load_boxes, ROOT
from common.terrain import Terrain
from common.flight import round_trip, max_safe_payload

OUT = ROOT / "results" / "q1"
OUT.mkdir(parents=True, exist_ok=True)

depot, areas = load_nodes()
types = load_drone_types()
boxes = load_boxes()
T = Terrain()
TYPE_IDS = list(types)

def better(a, b):

    return b is None or (a[0], a[1], a[2]) < (b[0], b[1], b[2])

def solve_area(area, rho):
    items = [b for b in boxes if b.area == area.id]
    n = len(items)
    mass = [int(round(b.mass)) for b in items]
    vol = [b.vol for b in items]
    qmax, vmax, etab = {}, {}, {}
    for g, t in types.items():
        qmax[g] = int(np.floor(max_safe_payload(t, T, depot, area, rho) + 1e-9))
        vmax[g] = t.V
        etab[g] = {}
    N = 1 << n

    sm = [0] * N
    sv = [0.0] * N
    for s in range(1, N):
        low = s & -s
        i = low.bit_length() - 1
```

```
        sm[s] = sm[s ^ low] + mass[i]
        sv[s] = sv[s ^ low] + vol[i]

    cost = {}
    for g, t in types.items():
        cost[g] = {}
        for s in range(1, N):
            if sm[s] > qmax[g] or sv[s] > vmax[g] + 1e-9:
                continue
            k = bin(s).count("1")
            r = round_trip(t, T, depot, area, sm[s], k)
            cost[g][s] = (r["E"], r["t_total"])
    dp = [None] * N
    dp[0] = (0, 0.0, 0.0)
    for mask in range(N):
        cur = dp[mask]
        if cur is None:
            continue
        rem = (N - 1) & ~mask
        if not rem:
            continue
        low = rem & -rem
        sub = rem
        while sub:
            if sub & low:
                for g in TYPE_IDS:
                    c = cost[g].get(sub)
                    if c is None:
                        continue
                    cand = (cur[0] + 1, cur[1] + c[0], cur[2] + c[1])
                    if better(cand, dp[mask | sub]):
                        dp[mask | sub] = cand
            sub = (sub - 1) & rem
    best = dp[N - 1]

    plan = []
    mask = N - 1
    while mask:
        cur = dp[mask]
        rem = mask
        low = rem & -rem
        found = False
        sub = rem
        while sub and not found:
            if sub & low:
                for g in TYPE_IDS:
                    c = cost[g].get(sub)
                    if c is None:
                        continue
                    prev = (cur[0] - 1, cur[1] - c[0], cur[2] - c[1])
                    pm = mask ^ sub
                    if dp[pm] is not None and better(dp[pm], None) and \
                            abs(dp[pm][0] - prev[0]) < 1e-9 and \
                            abs(dp[pm][1] - prev[1]) < 1e-6 and \
                            abs(dp[pm][2] - prev[2]) < 1e-6:
                        plan.append((g, [items[i].id for i in range(n) if sub >> i &
                            1],
                                     sm[sub], round(c[0], 4), round(c[1], 1)))
                        mask = pm
                        found = True
                        break
            sub = (sub - 1) & rem
        if not found:
            break
    return dict(area=area.id, n_box=n, sorties=best[0], E=round(best[1], 4),
```

```
                T=round(best[2], 1), plan=plan, qsafe={g: round(v, 2) for g, v in
                    qsafe_of(area, rho).items()})

def qsafe_of(area, rho):
    return {g: max_safe_payload(t, T, depot, area, rho) for g, t in types.items()}

def main():
    t0 = time.time()
    rows = []
    for a in areas:
        r = solve_area(a, types[TYPE_IDS[0]].rho)
        rows.append(r)
        print("%s 箱%2d 架次%2d 能耗%8.4f 时间%9.1f  (%.0fs)" %
              (r["area"], r["n_box"], r["sorties"], r["E"], r["T"], time.time() - t0)
                  , flush=True)
    tot_n = sum(r["sorties"] for r in rows)
    tot_e = sum(r["E"] for r in rows)
    tot_t = sum(r["T"] for r in rows)
    print("合计 架次%d 能耗%.4f kWh 时间%.1f s" % (tot_n, tot_e, tot_t))
    json.dump(dict(by_area=rows, total=dict(sorties=tot_n, E=round(tot_e, 4), T=round
        (tot_t, 1))),
              open(OUT / "dp_by_area.json", "w", encoding="utf-8"),
              ensure_ascii=False, indent=1)
    print("写好", OUT / "dp_by_area.json")

if __name__ == "__main__":
    main()
```

q2.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
    深度求索，版本发布日期 2026 年。

import sys
import json
import math
import random
import time
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
from common.q2model import (boxes, types, TYPE_IDS, BOX_AREA, NB, sortie_eval,
    evaluate, check, W)
from common.data import ROOT

OUT = ROOT / "results" / "q2"
OUT.mkdir(parents=True, exist_ok=True)

def urgency(b):
    bx = boxes[b]
    return (bx.t_first, bx.t_expect, -bx.prio)

def clean(sorties):
    out = []
    for g, stops in sorties:
        st = tuple((a, bx) for a, bx in stops if bx)
        if st:
            out.append((g, st))
    return out

def add_box_to_stops(stops, b, pos=None):

    a = BOX_AREA[b]
    for k, (ar, bx) in enumerate(stops):
        if ar == a:
```

```
            new = list(stops)
            new[k] = (ar, tuple(sorted(bx + (b,))))
            return [tuple(new)]
    res = []
    positions = range(len(stops) + 1) if pos is None else [pos]
    for p in positions:
        new = list(stops)
        new.insert(p, (a, (b,)))
        res.append(tuple(new))
    return res

def remove_boxes(sorties, rem):
    rem = set(rem)
    out = []
    for g, stops in sorties:
        st = tuple((a, tuple(x for x in bx if x not in rem)) for a, bx in stops)
        out.append((g, st))
    return clean(out)

def initial(rng):

    by_area = {}
    for b in range(NB):
        by_area.setdefault(BOX_AREA[b], []).append(b)
    sorties = []
    for a, bl in by_area.items():
        bl.sort(key=urgency)
        cur = []
        g = rng.choice(TYPE_IDS)
        for b in bl:
            trial = (g, ((a, tuple(sorted(cur + [b]))),))
            if cur and not sortie_eval(*trial)[0]:
                sorties.append((g, ((a, tuple(sorted(cur))),)))
                cur = [b]
                g = rng.choice(TYPE_IDS)
            else:
                cur.append(b)
        if cur:
            sorties.append((g, ((a, tuple(sorted(cur))),)))
    sorties.sort(key=lambda s: min(urgency(b) for _, bx in s[1] for b in bx))
    return sorties

def repair(sorties, removed, rng, w, noise=0.0):
    removed = sorted(removed, key=urgency)
    cur = list(sorties)
    for b in removed:
        best, bestJ = None, math.inf
        cands = []
        for si, (g, stops) in enumerate(cur):
            for st in add_box_to_stops(stops, b):
                if sortie_eval(g, st)[0]:
                    c = list(cur)
                    c[si] = (g, st)
                    cands.append(c)
        for g in TYPE_IDS:
            st = ((BOX_AREA[b], (b,)),)
            for pos in {0, len(cur) // 2, len(cur)}:
                c = list(cur)
                c.insert(pos, (g, st))
                cands.append(c)
        for c in cands:
            J = evaluate(c, w)[0] * (1.0 + noise * rng.random())
            if J < bestJ:
                bestJ, best = J, c
        cur = best
```

```
    return cur

def destroy(sorties, rng, w, k):
    mode = rng.random()
    allb = list(range(NB))
    if mode < 0.35:
        rem = rng.sample(allb, k)
    elif mode < 0.6:
        J, info = evaluate(sorties, w)
        sc = sorted(allb, key=lambda b: -(max(0.0, info["deliv"].get(b, 1e9) - boxes[
            b].t_expect) * boxes[b].prio + rng.random() * 500))
        rem = sc[:k]
    elif mode < 0.8:
        si = rng.randrange(len(sorties))
        rem = [b for _, bx in sorties[si][1] for b in bx]
        if len(rem) < k:
            rem += rng.sample([b for b in allb if b not in rem], k - len(rem))
    else:
        a = rng.choice(sorted(set(BOX_AREA)))
        rem = [b for b in allb if BOX_AREA[b] == a]
        rem = rng.sample(rem, min(k, len(rem)))
    return remove_boxes(sorties, rem), rem

def local_moves(sorties, rng):

    s = list(sorties)
    r = rng.random()
    if r < 0.4 and len(s) > 1:
        i, j = rng.sample(range(len(s)), 2)
        s[i], s[j] = s[j], s[i]
    elif r < 0.7:
        i = rng.randrange(len(s))
        g = rng.choice(TYPE_IDS)
        if sortie_eval(g, s[i][1])[0]:
            s[i] = (g, s[i][1])
    else:
        i = rng.randrange(len(s))
        g, stops = s[i]
        if len(stops) > 1:
            st = list(stops)
            rng.shuffle(st)
            if sortie_eval(g, tuple(st))[0]:
                s[i] = (g, tuple(st))
    return s

import os

MODE = os.environ.get("Q2_MODE", "hybrid")

def alns(seed=0, iters=3000, w=W, log=None, T0=None):
    rng = random.Random(seed)
    cur = initial(rng)
    cur = repair([], list(range(NB)), rng, w) if rng.random() < 0.5 else cur
    Jc, _ = evaluate(cur, w)
    best, Jb = cur, Jc
    T0 = T0 or max(Jc * 0.02, 1.0)
    t0 = time.time()
    for it in range(iters):
        temp = T0 * (1 - it / iters) + 1e-3
        if MODE == "sa" or rng.random() < 0.3:
            new = local_moves(cur, rng)
        else:
            k = rng.randint(3, 12)
            part, rem = destroy(cur, rng, w, k)
            new = repair(part, rem, rng, w, noise=0.05)
```

```
        Jn, _ = evaluate(new, w)
        accept_worse = MODE != "lns" and rng.random() < math.exp(-(Jn - Jc) / temp)
        if Jn < Jc or accept_worse:
            cur, Jc = new, Jn
            if Jn < Jb - 1e-9:
                best, Jb = new, Jn
        if log and it % 100 == 0:
            info = evaluate(best, w)[1]
            print(f"seed{seed} it{it} Jb={Jb:.2f} cur={Jc:.2f} N={info['N']} E={info
                ['E']:.2f} mk={info['mk']:.0f} late={info['late']:.2f} hard={info['
                hard']:.2f} {time.time()-t0:.0f}s", file=log, flush=True)
    return best, Jb

def dump(sorties, w, path):
    J, info = evaluate(sorties, w)
    check(sorties)
    rows = []
    for i, ((g, stops), (d, b, s, r)) in enumerate(zip(sorties, info["plan"])):
        ok, dur, E, soc, deliv, detail = sortie_eval(g, stops)
        rows.append(dict(id=f"Q2-{i+1:03d}", drone=d, type=g, battery=b, start=round(
            s, 1),
                         route=[a for a, _ in stops], boxes=[list(bx) for _, bx in
                             stops],
                         ret=round(r, 1), E=round(E, 4), soc=round(soc * 100, 2),
                         deliv={boxes[k].id: round(s + v, 1) for k, v in deliv.items
                             ()}))
    out = dict(J=J, w=w, N=info["N"], E=round(info["E"], 4), mk=round(info["mk"], 1),
         late=round(info["late"], 4),
               hard=round(info["hard"], 4), sorties=rows)
    json.dump(out, open(path, "w", encoding="utf-8"), ensure_ascii=False, indent=1)
    return out

if __name__ == "__main__":
    seed = int(sys.argv[1]) if len(sys.argv) > 1 else 0
    iters = int(sys.argv[2]) if len(sys.argv) > 2 else 3000
    tag = sys.argv[3] if len(sys.argv) > 3 else "base"
    w = dict(W)
    if len(sys.argv) > 4:
        w.update(json.loads(sys.argv[4]))
    log = open(OUT / f"log_{tag}_s{seed}.txt", "w")
    t_start = time.time()
    if MODE == "greedy":
        best = initial(random.Random(seed))
        Jb = evaluate(best, w)[0]
    else:
        best, Jb = alns(seed, iters, w, log)
    elapsed = time.time() - t_start
    out = dump(best, w, OUT / f"sol_{tag}_s{seed}.json")
    out["mode"], out["elapsed"] = MODE, round(elapsed, 1)
    json.dump(out, open(OUT / f"sol_{tag}_s{seed}.json", "w", encoding="utf-8"),
        ensure_ascii=False, indent=1)
    print(json.dumps({k: v for k, v in out.items() if k != "sorties"}, ensure_ascii=
        False))
```

q2_ga.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构深度求索，版本发布日期 2026 年。

import json
import random
import sys
import time
from pathlib import Path

```

```
HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))
from common.q2model import evaluate, check, NB, W
import q2

OUT = HERE.parent / "results" / "q2"

def boxes_of(s):
    return {b for _, bx in s for b in bx}

def cross(pa, pb, rng, w):

    keys = [s[0] for s in pa]
    pick = rng.sample(range(len(pa)), max(1, len(pa) // 2))
    child = [pa[i] for i in sorted(pick)]
    used = boxes_of(child)
    for s in pb:
        bs = boxes_of([s])
        if not (bs & used):
            child.append(s)
            used |= bs
    missing = [i for i in range(NB) if i not in used]
    if missing:
        child = q2.repair(child, missing, rng, w, noise=0.03)
    return child

def mutate(sol, rng, w):
    if rng.random() < 0.55:
        k = rng.randint(3, 12)
        cut, removed = q2.destroy(sol, rng, w, k)
        return q2.repair(cut, removed, rng, w, noise=0.05)
    return q2.local_moves(sol, rng)

def fitness(sol, w):
    J, r = evaluate(sol, w)
    return J, r

def main(seed, gens, pop_size=24):
    rng = random.Random(seed)
    w = dict(W)
    t0 = time.time()
    log = open(OUT / ("log_ga_s%d.txt" % seed), "w")

    pop = [q2.initial(rng)]
    for f in [OUT / "final.json"] + sorted(OUT.glob("sol_*.json")):
        try:
            d = json.load(open(f, encoding="utf-8"))
            pop.append([(s["type"], tuple((a, tuple(b))
                                          for a, b in zip(s["route"], s["boxes"])))
                        for s in d["sorties"]])
        except Exception:
            continue
    if len(pop) > pop_size:
        pop = [pop[0]] + rng.sample(pop[1:], pop_size - 1)
    while len(pop) < pop_size:
        pop.append(mutate(rng.choice(pop), rng, w))

    scored = [(fitness(s, w)[0], s) for s in pop]
    scored.sort(key=lambda x: x[0])
    best_J, best_sol = scored[0]

    for g in range(gens):
        elite = [s for _, s in scored[:max(2, pop_size // 6)]]
        kids = list(elite)
        while len(kids) < pop_size:
```

```
            a = min(rng.sample(scored, 3), key=lambda x: x[0])[1]
            b = min(rng.sample(scored, 3), key=lambda x: x[0])[1]
            c = cross(a, b, rng, w)
            if rng.random() < 0.85:
                c = mutate(c, rng, w)
            kids.append(c)
        scored = [(fitness(s, w)[0], s) for s in kids]
        scored.sort(key=lambda x: x[0])
        if scored[0][0] < best_J:
            best_J, best_sol = scored[0][0], scored[0][1]
            _, r = evaluate(best_sol, w)
            print("gen%5d J=%.3f N=%d E=%.2f mk=%.1f  %.0fs" %
                  (g, best_J, r["N"], r["E"], r["mk"], time.time() - t0), file=log,
                      flush=True)
        if g % 20 == 0:
            print("gen%5d J=%.3f  %.0fs" % (g, best_J, time.time() - t0), flush=True)

    _, r = evaluate(best_sol, w)
    chk = check(best_sol)
    out = dict(J=best_J, N=r["N"], E=round(r["E"], 4), mk=round(r["mk"], 1),
               late=round(r["late"], 3), hard=round(r["hard"], 3),
               infeasible=r["infeasible"], feasible=bool(chk) if not isinstance(chk,
                   dict) else chk.get("ok"))
    json.dump(out, open(OUT / ("ga_s%d.json" % seed), "w", encoding="utf-8"),
        ensure_ascii=False)
    q2.dump(best_sol, w, OUT / ("sol_ga_s%d.json" % seed))
    print("seed%d 最好 J=%.3f N=%d E=%.2f mk=%.1f 用时%.0fs" %
          (seed, best_J, r["N"], r["E"], r["mk"], time.time() - t0), flush=True)
    log.close()

if __name__ == "__main__":
    main(int(sys.argv[1]), int(sys.argv[2]) if len(sys.argv) > 2 else 150,
         int(sys.argv[3]) if len(sys.argv) > 3 else 24)
```

q2_run.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import sys
import json
import subprocess
from pathlib import Path
from concurrent.futures import ThreadPoolExecutor

HERE = Path(__file__).resolve().parent
ITERS = int(sys.argv[1]) if len(sys.argv) > 1 else 40000
CONFIGS = {
    "base": {},
    "time": {"late": 20.0, "hard": 200.0},
    "mk": {"mk": 12.0},
    "energy": {"E": 4.0},
    "sortie": {"N": 6.0},
}
SEEDS = {"base": range(6), "time": range(3), "mk": range(3), "energy": range(3), "
    sortie": range(3)}
jobs = []
for tag, w in CONFIGS.items():
    for s in SEEDS[tag]:
        jobs.append((tag, s, w))

def run(job):
    tag, s, w = job
    cmd = [sys.executable, str(HERE / "q2.py"), str(s), str(ITERS), tag, json.dumps(w
        )]
```

```
    r = subprocess.run(cmd, capture_output=True, text=True)
    return tag, s, r.stdout.strip().splitlines()[-1] if r.stdout.strip() else r.stderr[-500:]

with ThreadPoolExecutor(max_workers=len(jobs)) as ex:
    for tag, s, line in ex.map(run, jobs):
        print(tag, s, line, flush=True)
```

q2_pick.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import sys
import json
import glob
import shutil
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
import pandas as pd
from common.data import ROOT
from common.q2model import evaluate, check, W, NODES

D = Path(sys.argv[1]) if len(sys.argv) > 1 else ROOT / "results" / "q2"
rows = []

WEIGHT_TAGS = {"base", "time", "mk", "energy", "sortie"}
for f in sorted(glob.glob(str(D / "sol_*_s*.json"))):
    s = json.load(open(f, encoding="utf-8"))
    tag = Path(f).stem.split("_")[1]
    if tag not in WEIGHT_TAGS:
        continue
    sorties = [(x["type"], tuple((a, tuple(bx)) for a, bx in zip(x["route"], x["boxes"]))) for x in s["sorties"]]
    Jbase, info = evaluate(sorties, W)
    rows.append(dict(file=Path(f).name, tag=tag, J=round(s["J"], 2), Jbase=round(Jbase, 2), N=s["N"], E=round(s["E"], 2),
                     avgT=round(sum(info["deliv"].values()) / len(info["deliv"]), 1),
                     mk=round(s["mk"]), late=round(s["late"], 2), hard=round(s["hard"], 2),
                     nA=sum(1 for x in s["sorties"] if x["type"] == "A"), nB=sum(1 for x in s["sorties"] if x["type"] == "B"),
                     nC=sum(1 for x in s["sorties"] if x["type"] == "C"),
                     multi=sum(1 for x in s["sorties"] if len(x["route"]) > 1)))
df = pd.DataFrame(rows)
pd.set_option("display.width", 200)
print(df.sort_values(["tag", "J"]).to_string(index=False))
df.to_csv(D / "summary.csv", index=False, encoding="utf-8-sig")

best = df.sort_values("J").groupby("tag").head(1)
print("\n各权重最优：")
print(best.to_string(index=False))
best.to_csv(D / "tradeoff.csv", index=False, encoding="utf-8-sig")
if len(df):

    fb = df.sort_values("Jbase").iloc[0]["file"]
    s = json.load(open(D / fb, encoding="utf-8"))
    sorties = [(x["type"], tuple((a, tuple(bx)) for a, bx in zip(x["route"], x["boxes"]))) for x in s["sorties"]]
    check(sorties)
    shutil.copy(D / fb, D / "final.json")
    print("final =", fb)
    for x in s["sorties"]:
        print(x["id"], x["drone"], x["battery"], round(x["start"]), "→".join(NODES[a].id for a in x["route"]),
```

```
                round(x["ret"]), x["E"], x["soc"])
```

q2_ablation_sum.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import sys
import json
import re
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
import numpy as np
from common.q2model import evaluate, W

D = Path(__file__).resolve().parents[1] / "results" / "q2"
rows = {}
for tag, name in (("greedy", "分区贪心构造"), ("sa", "模拟退火，仅局部扰动"),
                  ("lns", "大邻域搜索，只接受改进"), ("base", "大邻域搜索与模拟退火混合")):
    rs = []
    for s in range(6):
        f = D / f"sol_{tag}_s{s}.json"
        sol = json.load(open(f, encoding="utf-8"))
        sorties = [(x["type"], tuple((a, tuple(bx)) for a, bx in zip(x["route"], x["boxes"])))
                   for x in sol["sorties"]]
        J, info = evaluate(sorties, W)
        el = sol.get("elapsed")
        if el is None:
            last = open(D / f"log_{tag}_s{s}.txt", encoding="utf-8").read().strip().splitlines()[-1]
            el = float(re.search(r"(\d+)s$", last).group(1))
        rs.append((J, info["N"], info["E"], info["mk"], el, info["hard"], info["late"]))
    a = np.array(rs)
    ib = int(np.argmin(a[:, 0]))
    rows[tag] = dict(name=name, J_best=a[ib, 0], N_best=int(a[ib, 1]), E_best=a[ib, 2],
                     mk_best=a[ib, 3], J_mean=a[:, 0].mean(), J_std=a[:, 0].std(ddof=0),
                     N_list=[int(x) for x in a[:, 1]], hit22=int((a[:, 1] <= 22).sum()),
                     t_mean=a[:, 4].mean(), hard_max=a[:, 5].max())
    r = rows[tag]
    print(f"{name:16s} best J={r['J_best']:.1f} N={r['N_best']} E={r['E_best']:.2f} mk={r['mk_best']:.0f} | "
          f"mean J={r['J_mean']:.1f} std={r['J_std']:.1f} N={r['N_list']} hit22={r['hit22']}/6 "
          f"t={r['t_mean']:.1f}s hard_max={r['hard_max']:.2f}")
json.dump(rows, open(D / "ablation.json", "w", encoding="utf-8"), ensure_ascii=False, indent=1)
```

q2_smoke.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import sys
import time
import random
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
from common.q2model import evaluate, check, sortie_eval, NB, types, legs
```

```
from q2 import initial, repair, destroy, alns, W

rng = random.Random(0)
s0 = initial(rng)
J, info = evaluate(s0)
print("initial N", info["N"], "E", round(info["E"], 2), "mk", round(info["mk"]), "
    late", round(info["late"], 2), "hard", round(info["hard"], 2), "J", round(J, 2))
check(s0)
t = time.time()
for _ in range(200):
    evaluate(s0)
print("evaluate ms", round((time.time() - t) / 200 * 1000, 2))
t = time.time()
part, rem = destroy(s0, rng, W, 8)
s1 = repair(part, rem, rng, W)
print("repair 8 boxes s", round(time.time() - t, 2))
J1, info1 = evaluate(s1)
print("after repair N", info1["N"], "E", round(info1["E"], 2), "mk", round(info1["mk
    "]), "J", round(J1, 2))
t = time.time()
best, Jb = alns(seed=1, iters=60, log=sys.stdout)
print("60 iters s", round(time.time() - t, 1), "Jb", round(Jb, 2))
check(best)
```

q3_geom.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
    深度求索，版本发布日期 2026 年。

import sys
import pickle
from pathlib import Path
from multiprocessing import Pool
sys.path.insert(0, str(Path(__file__).resolve().parent))
import numpy as np
from common.q2model import NODES, legs, types, T, depot, TYPE_IDS
from common.data import ROOT
from common.comm import available, GW, THR_DIRECT, THR_ACCESS, THR_BACKHAUL

OUT = ROOT / "results" / "q3"
OUT.mkdir(parents=True, exist_ok=True)
DT = 5.0
GRID = 400.0
HEIGHTS = (100.0, 200.0, 300.0)

def leg_samples(g, i, j):

    t = types[g]
    d, hu, hd, zc = legs[i][j]
    A, B = NODES[i], NODES[j]
    pts = []
    clock = 0.0
    tu = hu / t.v_up
    for s in np.arange(0, tu, DT):
        pts.append((clock + s, "climb", A.lon, A.lat, A.work_alt + t.v_up * s))
    clock += tu
    tc = d / t.v_cruise
    for s in np.arange(0, tc, DT):
        f = s / tc
        pts.append((clock + s, "cruise", A.lon + (B.lon - A.lon) * f, A.lat + (B.lat
            - A.lat) * f, zc))
    clock += tc
    td = hd / t.v_down
    for s in np.arange(0, td, DT):
        pts.append((clock + s, "descent", B.lon, B.lat, zc - t.v_down * s))
```

```
    return pts

def _direct(p):
    return available(p, GW, THR_DIRECT)[0]

def _bh(c):
    return available(c[:3], GW, THR_BACKHAUL)[0]

def _cover(args):
    c, pos = args
    return [available(c[:3], p, THR_ACCESS)[0] for p in pos]

if __name__ == "__main__":

    samples = {}
    for g in TYPE_IDS:
        for i in range(len(NODES)):
            for j in range(len(NODES)):
                if i != j and (i == 0 or j == 0 or True):
                    samples[g, i, j] = leg_samples(g, i, j)
    hand = {a: (NODES[a].lon, NODES[a].lat, NODES[a].work_alt) for a in range(1, len(
        NODES))}

    pos_index = {}
    positions = []

    def pid(p):
        key = (round(p[0], 6), round(p[1], 6), round(p[2], 1))
        if key not in pos_index:
            pos_index[key] = len(positions)
            positions.append(key)
        return pos_index[key]

    sample_pid = {k: [pid(p[2:5]) for p in v] for k, v in samples.items()}
    hand_pid = {a: pid(p) for a, p in hand.items()}
    print("航段", len(samples), "采样", sum(len(v) for v in samples.values()), "去重
        位置", len(positions), flush=True)

    with Pool(18) as p:
        direct = p.map(_direct, positions, chunksize=64)
    direct = np.array(direct, dtype=bool)
    fail_ids = np.where(~direct)[0]
    print("直连失败位置", len(fail_ids), flush=True)

    lons = [n.lon for n in NODES]
    lats = [n.lat for n in NODES]
    x0, y0 = T.xy(min(lons), min(lats))
    x1, y1 = T.xy(max(lons), max(lats))
    pad = 1000.0
    cands = []
    for x in np.arange(x0 - pad, x1 + pad + 1, GRID):
        for y in np.arange(y0 - pad, y1 + pad + 1, GRID):
            lon, lat = T.ll(x, y)
            if T.inside(lon, lat):
                gz = float(T.elev(lon, lat))
                for h in HEIGHTS:
                    cands.append((float(lon), float(lat), gz + h, h))
    with Pool(18) as p:
        bh = p.map(_bh, cands, chunksize=32)
    cands = [c for c, ok in zip(cands, bh) if ok]
    print("候选", len(cands), flush=True)

    from common.q2model import NODE_CLUSTER
    ncl = max(NODE_CLUSTER) + 1
    fail_pos = [positions[k] for k in fail_ids]
```

```
    col_of = {int(k): c for c, k in enumerate(fail_ids)}
    with Pool(18) as p:
        cov = np.array(p.map(_cover, [(c, fail_pos) for c in cands], chunksize=2),
            dtype=bool)
    print("覆盖矩阵", cov.shape, flush=True)

    items = {}
    for (g, i, j), ids in sample_pid.items():
        if i != 0 and j != 0 and NODE_CLUSTER[i] != NODE_CLUSTER[j]:
            continue
        cols = [col_of[q] for q in ids if not direct[q]]
        items[(g, i, j)] = cols
    for a, q in hand_pid.items():
        items[("H", a)] = [] if direct[q] else [col_of[q]]
    need = {k: v for k, v in items.items() if v}
    keys = list(need)
    full = np.zeros((len(cands), len(keys)), dtype=bool)
    for ci in range(len(cands)):
        row = cov[ci]
        for ki, k in enumerate(keys):
            full[ci, ki] = row[need[k]].all()
    wt = np.array([3.0 if (k[0] == "H" or k[1] == 0 or k[2] == 0) else 1.0 for k in
        keys])
    print("需要中继的项目", len(keys), "任一候选可整段覆盖的项目", int(full.any(axis
        =0).sum()), flush=True)

    dist0 = np.array([T.horiz_dist((c[0], c[1]), (depot.lon, depot.lat)) for c in
        cands])
    uncovered = np.ones(len(keys), dtype=bool)
    chosen = []
    while uncovered.any():
        gain = (full & uncovered) @ wt
        if gain.max() <= 0:
            break
        k = int(np.argmax(gain - 1e-9 * dist0))
        chosen.append(k)
        uncovered &= ~full[k]
        print("选点", cands[k], "剩余未整段覆盖项目", int(uncovered.sum()), flush=
            True)

    for c in list(chosen):
        rest = [x for x in chosen if x != c]
        if rest and (full[rest].any(axis=0) | uncovered).all():
            chosen = rest
    hovers = [cands[c] for c in chosen]

    cluster_hover = {}
    for c in range(ncl):
        idx = [ki for ki, k in enumerate(keys) if (k[0] == "H" and NODE_CLUSTER[k[1]]
             == c) or
                (k[0] != "H" and NODE_CLUSTER[k[2] if k[2] != 0 else k[1]] == c)]
        sc = [(full[h, idx] @ wt[idx], hi) for hi, h in enumerate(chosen)]
        cluster_hover[c] = max(sc)[1]
    print("最终悬停点", hovers, "簇对应", cluster_hover, flush=True)
    item_full = {}
    for ki, k in enumerate(keys):
        item_full[k] = [hi for hi, h in enumerate(chosen) if full[h, ki]]
    for k in items:
        if k not in item_full:
            item_full[k] = list(range(len(chosen)))
    cover_by = {}
    for col, k in enumerate(fail_ids):
        cover_by[int(k)] = [hi for hi, h in enumerate(chosen) if cov[h, col]]
    n_unc = sum(1 for k in fail_ids if not cover_by[int(k)])
    print("失败位置", len(fail_ids), "无覆盖", n_unc, "项目无整段覆盖", sum(1 for k
```

```
            in keys if not item_full[k]), flush=True)
160     pickle.dump(dict(samples=samples, sample_pid=sample_pid, hand_pid=hand_pid,
            positions=positions,
161                      direct=direct, hovers=hovers, cover_by=cover_by, DT=DT,
                             cluster_hover=cluster_hover,
162                      item_full=item_full),
163                 open(OUT / "geom.pkl", "wb"))
164     print("saved geom.pkl")
```

q3_joint.py

```
1  # 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
       深度求索，版本发布日期 2026 年。
2
3  import sys
4  import json
5  import math
6  import pickle
7  import random
8  import time
9  from functools import lru_cache
10 from pathlib import Path
11 sys.path.insert(0, str(Path(__file__).resolve().parent))
12 import numpy as np
13 from common.q2model import (boxes, types, NODES, legs, TYPE_IDS, NB, sortie_eval,
       schedule, evaluate as evaluate2, check, W)
14 from common.data import ROOT, load_relay_type
15 from common.flight import G, J2KWH
16 from common.q2model import t_charge, T, depot
17 import q2
18
19 OUT = ROOT / "results" / "q3"
20 OUT.mkdir(parents=True, exist_ok=True)
21 GEOM = pickle.load(open(OUT / "geom.pkl", "rb"))
22 DT = GEOM["DT"]
23 HOVERS = GEOM["hovers"]
24 NH = len(HOVERS)
25 DIRECT = GEOM["direct"]
26 COVER = GEOM["cover_by"]
27 SAMPLES = GEOM["samples"]
28 SPID = GEOM["sample_pid"]
29 HPID = GEOM["hand_pid"]
30 R = load_relay_type()
31 GAP_MERGE = 1200.0
32 W3 = dict(W, NR=1.0, uncov=1e4)
33
34 def relay_flight(h):
35     zc, d = T.cruise_alt((depot.lon, depot.lat), (h[0], h[1]))
36     zc = max(zc, h[2])
37     hu, hd = zc - depot.ground, zc - h[2]
38     t_out = hu / R.v_up + d / R.v_cruise + hd / R.v_down
39     E_out = R.P_cruise * (d / R.v_cruise) / 3600 + R.m_takeoff * G * hu / R.eta_up *
           J2KWH
40     t_back = hd / R.v_up + d / R.v_cruise + hu / R.v_down
41     E_back = R.P_cruise * (d / R.v_cruise) / 3600 + R.m_takeoff * G * hd / R.eta_up *
           J2KWH
42     return t_out, E_out, t_back, E_back
43
44 RF = [relay_flight(h) for h in HOVERS]
45 BUDGET = (1 - R.rho) * R.E_use
46 P_SRV = (R.P_hover + R.P_comm) / 3600.0
47 S_MAX = [(BUDGET - f[1] - f[3]) / P_SRV - R.t_link for f in RF]
48
49 CLUSTER_HOVER = GEOM.get("cluster_hover", {})
50 from common.q2model import NODE_CLUSTER
```

```

def _pick(cands, prev):
    if prev in cands:
        return prev
    return cands[0]

ITEM_FULL = GEOM.get("item_full", {})

@lru_cache(maxsize=200000)
def relay_demand(g, stops):

    t = types[g]
    all_boxes = [b for _, bx in stops for b in bx]
    clock = t.t_prep + t.t_load_box * len(all_boxes)
    prev = 0
    route = [(a, len(bx)) for a, bx in stops] + [(0, 0)]
    keys = []
    for area, nb in route:
        keys.append((g, prev, area))
        if area != 0:
            keys.append(("H", area))
        prev = area
    pref = CLUSTER_HOVER.get(NODE_CLUSTER[stops[0][0]], 0)
    common = None
    for k in keys:
        s = set(ITEM_FULL.get(k, range(NH)))
        common = s if common is None else common & s
    single = (pref if pref in common else min(common)) if common else None
    events = []
    last = single if single is not None else pref
    prev = 0
    for area, nb in route:
        k = (g, prev, area)
        item_h = single if single is not None else _pick(ITEM_FULL[k], last) if
            ITEM_FULL.get(k) else None
        for (rt, ph, lon, lat, alt), pidx in zip(SAMPLES[g, prev, area], SPID[g, prev
            , area]):
            if not DIRECT[pidx]:
                if item_h is not None:
                    h = item_h
                else:
                    c = COVER.get(pidx, [])
                    h = _pick(c, last) if c else -1
                events.append((clock + rt, h))
                last = h if h >= 0 else last
        d, hu, hd, zc = legs[prev][area]
        clock += hu / t.v_up + d / t.v_cruise + hd / t.v_down
        if area != 0:
            th = t.t_hand_base + t.t_hand_box * nb
            pidx = HPID[area]
            if not DIRECT[pidx]:
                k = ("H", area)
                if single is not None:
                    h = single
                elif ITEM_FULL.get(k):
                    h = _pick(ITEM_FULL[k], last)
                else:
                    c = COVER.get(pidx, [])
                    h = _pick(c, last) if c else -1
                for s in np.arange(0, th, DT):
                    events.append((clock + s, h))
                last = h if h >= 0 else last
            clock += th
        prev = area
    iv = {}
```

```
    unc = 0
    for rt, h in events:
        if h < 0:
            unc += 1
            continue
        L = iv.setdefault(h, [])
        if L and rt <= L[-1][1] + DT + 1e-6:
            L[-1][1] = rt + DT
        else:
            L.append([rt, rt + DT])
    return {h: [tuple(x) for x in v] for h, v in iv.items()}, unc

D_MIN = 600.0

def relay_windows(shifts):

    ravail = {d: 0.0 for d in R.drones}
    rready = {f"PR{k+1:02d}": 0.0 for k in range(R.n_pack)}
    relays = []
    windows = {h: [] for h in range(NH)}
    for h, D in shifts:
        D = min(max(D, D_MIN), S_MAX[h])
        t_out, E_out, t_back, E_back = RF[h]
        d = min(ravail, key=ravail.get)
        p = min(rready, key=rready.get)
        start = max(ravail[d], rready[p])
        t_link = start + R.t_prep + t_out + R.t_link
        t_end = t_link + D
        t_ret = t_end + t_back
        E = E_out + E_back + P_SRV * (D + R.t_link)
        soc = 1 - E / R.E_use
        ravail[d] = t_ret + R.t_turn
        rready[p] = t_ret + t_charge(soc, R.T_full)
        relays.append(dict(h=h, drone=d, pack=p, start=start, t_link=t_link, t0=
            t_link, t1=t_end, t_ret=t_ret, E=E, soc=soc))
        windows[h].append((t_link, t_end))
    return relays, windows

def joint_schedule(sorties, shifts):

    relays, windows = relay_windows(shifts)
    avail = {g: {d: 0.0 for d in types[g].drones} for g in TYPE_IDS}
    ready = {g: {f"B{g}{k+1:02d}": 0.0 for k in range(types[g].n_batt)} for g in
        TYPE_IDS}
    plan = []
    total_delay = 0.0
    uncovered = 0.0
    for g, stops in sorties:
        ok, dur, E, soc, deliv, _ = sortie_eval(g, stops)
        d = min(avail[g], key=avail[g].get)
        b = min(ready[g], key=ready[g].get)
        s0 = max(avail[g][d], ready[g][b])
        iv, unc = relay_demand(g, stops)
        uncovered += unc * DT
        s = s0
        bad = 0.0
        for _ in range(30):
            s_new = s
            bad = 0.0
            for h, L in iv.items():
                W_h = windows[h]
                for a, bb in L:
                    if any(wl <= s + a + 1e-6 and s + bb <= we + 1e-6 for wl, we in
                        W_h):
                        continue
```

```
                    cand = [wl - a for wl, we in W_h if wl - a >= s - 1e-6 and we -
                        wl >= bb - a - 1e-6]
                    if cand:
                        s_new = max(s_new, min(cand))
                    else:
                        bad += bb - a
            if s_new <= s + 1e-6:
                break
            s = s_new
        uncovered += bad
        total_delay += s - s0
        ret = s + dur
        avail[g][d] = ret
        ready[g][b] = ret + t_charge(soc, types[g].T_full)
        plan.append((d, b, s, ret))
    return plan, relays, total_delay, uncovered

def evaluate3(sorties, shifts, w=W3):
    plan, relays, delay, unc = joint_schedule(sorties, shifts)
    late = hard = E = mk = 0.0
    infeasible = 0
    deliv_t = {}
    for (g, stops), (d, b, start, ret) in zip(sorties, plan):
        ok, dur, e, soc, deliv, _ = sortie_eval(g, stops)
        if not ok:
            infeasible += 1
        E += e
        mk = max(mk, ret)
        for bi, off in deliv.items():
            tb = start + off
            deliv_t[bi] = tb
            bx = boxes[bi]
            if tb > bx.t_first:
                hard += bx.prio * (tb - bx.t_first) / 1000.0 + 1.0
            if bx.kind == "医疗物资" and tb > bx.t_expect:
                hard += bx.prio * (tb - bx.t_expect) / 1000.0 + 1.0
            elif tb > bx.t_expect:
                late += bx.prio * (tb - bx.t_expect) / 1000.0
    ER = sum(r["E"] for r in relays)
    mkR = max((r["t_ret"] for r in relays), default=0.0)
    mk_joint = max(mk, mkR)
    J = w["late"] * late + w["hard"] * hard + w["mk"] * mk_joint / 1000.0 + w["E"] *
        (E + ER) \
        + w["N"] * len(sorties) + w["NR"] * len(relays) + w["uncov"] * unc / 1000.0 +
            1e6 * infeasible
    info = dict(late=late, hard=hard, mk=mk, E=E, N=len(sorties), infeasible=
        infeasible, deliv=deliv_t, plan=plan,
                relays=relays, ER=ER, NR=len(relays), mkR=mkR, delay=delay, unc=unc,
                    mk_joint=mk_joint)
    return J, info

def load_init(path):
    sol = json.load(open(path, encoding="utf-8"))
    sorties = [(s["type"], tuple((a, tuple(bx)) for a, bx in zip(s["route"], s["boxes
        "]))) for s in sol["sorties"]]
    if sol.get("shifts"):
        shifts = [tuple(x) for x in sol["shifts"]]
    else:
        shifts = [(r.get("h", r.get("hi")), r.get("t1", r.get("t_end")) - r["t_link
            "]) for r in sol.get("relays", [])]
    return sorties, shifts

def initial_shifts(sorties):

```

```
    plan = schedule(sorties)
    demand = {h: [] for h in range(NH)}
    for (g, stops), (d, b, start, ret) in zip(sorties, plan):
        iv, _ = relay_demand(g, stops)
        for h, L in iv.items():
            for a, bb in L:
                demand[h].append((start + a, start + bb))
    shifts = []
    for h in range(NH):
        L = sorted(demand[h])
        if not L:
            continue
        cur = [L[0][0], L[0][1]]
        for a, bb in L[1:]:
            if a <= cur[1] + GAP_MERGE:
                cur[1] = max(cur[1], bb)
            else:
                shifts.append((cur[0], h, cur[1] - cur[0]))
                cur = [a, bb]
        shifts.append((cur[0], h, cur[1] - cur[0]))
    out = []
    for t0, h, D in sorted(shifts):
        n = max(1, math.ceil(D / S_MAX[h]))
        for _ in range(n):
            out.append((h, D / n + 300.0))
    return out

def structured_shifts(sorties, variant, horizon=9000.0, roam=1500.0):

    mains = sorted(set(CLUSTER_HOVER.values()))
    if len(mains) < 3:
        return initial_shifts(sorties)
    fixed = mains[variant % 3]
    others = [h for h in mains if h != fixed]
    if (variant // 3) % 2:
        others = others[::-1]
    shifts = []
    t = 0.0
    while t < horizon:
        shifts.append((fixed, S_MAX[fixed]))
        t += S_MAX[fixed] + 1500
    t = 0.0
    k = 0
    while t < horizon:
        h = others[k % 2]
        shifts.append((h, roam))
        t += roam + 1200
        k += 1
    return shifts

def trim_shifts(sorties, shifts, w=W3, rounds=4):

    best_J, _ = evaluate3(sorties, shifts, w)
    best = list(shifts)
    cur = list(shifts)
    for _ in range(rounds):
        plan, relays, _, _ = joint_schedule(sorties, cur)
        last_end = [None] * len(relays)
        for (g, stops), (d, b, s, ret) in zip(sorties, plan):
            iv, _ = relay_demand(g, stops)
            for h, L in iv.items():
                for a, bb in L:
                    t0, t1 = s + a, s + bb
                    for k, r in enumerate(relays):
                        if r["h"] == h and r["t_link"] <= t0 + 1e-6 and t1 <= r["t1"]
```

```
                                + 1e-6:
                            last_end[k] = max(last_end[k] or 0, t1)
                            break
        new = []
        for k, (h, D) in enumerate(cur):
            if last_end[k] is None:
                continue
            new.append((h, max(D_MIN, last_end[k] - relays[k]["t_link"] + 1.0)))
        if new == cur:
            break
        J, _ = evaluate3(sorties, new, w)
        if J <= best_J + 1e-9:
            best_J, best = J, list(new)
            cur = new
        else:
            break
    return best

def shift_move(shifts, rng):
    s = list(shifts)
    r = rng.random()
    main = [h for h in set(CLUSTER_HOVER.values())] or list(range(NH))
    if r < 0.2 or not s:
        h = rng.choice(main) if rng.random() < 0.85 else rng.randrange(NH)
        s.insert(rng.randint(0, len(s)), (h, rng.uniform(1200, S_MAX[h])))
    elif r < 0.4:
        s.pop(rng.randrange(len(s)))
    elif r < 0.7:
        i = rng.randrange(len(s))
        h, D = s[i]
        s[i] = (h, min(max(D + rng.uniform(-1500, 1500), D_MIN), S_MAX[h]))
    elif r < 0.85 and len(s) > 1:
        i, j = rng.sample(range(len(s)), 2)
        s[i], s[j] = s[j], s[i]
    else:
        i = rng.randrange(len(s))
        h = rng.choice(main) if rng.random() < 0.85 else rng.randrange(NH)
        s[i] = (h, s[i][1])
    return s

def alns3(init_sorties, init_shifts, seed, iters, w, log):
    rng = random.Random(seed)
    cur_s, cur_r = list(init_sorties), list(init_shifts)
    q2.evaluate = lambda s, ww=w: evaluate3(s, cur_r, ww)
    Jc, _ = evaluate3(cur_s, cur_r, w)
    best, Jb = (cur_s, cur_r), Jc
    T0 = max(Jc * 0.02, 1.0)
    t0 = time.time()
    for it in range(iters):
        temp = T0 * (1 - it / iters) + 1e-3
        r = rng.random()
        new_s, new_r = cur_s, cur_r
        if r < 0.35:
            new_r = shift_move(cur_r, rng)
        elif r < 0.55:
            new_s = q2.local_moves(cur_s, rng)
        else:
            k = rng.randint(3, 12)
            part, rem = q2.destroy(cur_s, rng, w, k)
            new_s = q2.repair(part, rem, rng, w, noise=0.05)
        Jn, _ = evaluate3(new_s, new_r, w)
        if Jn < Jc or rng.random() < math.exp(-(Jn - Jc) / temp):
            cur_s, cur_r, Jc = new_s, new_r, Jn
            q2.evaluate = lambda s, ww=w, rr=cur_r: evaluate3(s, rr, ww)
            if Jn < Jb - 1e-9:
```

```
                best, Jb = (new_s, new_r), Jn
        if log and it % 100 == 0:
            i = evaluate3(best[0], best[1], w)[1]
            print(f"seed{seed} it{it} Jb={Jb:.2f} N={i['N']} E={i['E']:.2f} mk={i['mk
                ']:.0f} NR={i['NR']} ER={i['ER']:.2f} mkR={i['mkR']:.0f} delay={i['
                delay']:.0f} unc={i['unc']:.0f} late={i['late']:.2f} hard={i['hard
                ']:.2f} {time.time()-t0:.0f}s", file=log, flush=True)
    return best, Jb

def dump3(sorties, shifts, w, path):
    J, info = evaluate3(sorties, shifts, w)
    check(sorties)
    rows = []
    for i, ((g, stops), (d, b, s, r)) in enumerate(zip(sorties, info["plan"])):
        ok, dur, E, soc, deliv, detail = sortie_eval(g, stops)
        rows.append(dict(id=f"Q3-{i+1:03d}", drone=d, type=g, battery=b, start=round(
            s, 1),
                         route=[a for a, _ in stops], boxes=[list(bx) for _, bx in
                             stops],
                         ret=round(r, 1), E=round(E, 4), soc=round(soc * 100, 2),
                         deliv={boxes[k].id: round(s + v, 1) for k, v in deliv.items
                             ()}))
    relay_rows, relays = [], []
    for k, r in enumerate(info["relays"]):
        h = HOVERS[r["h"]]
        rid = f"R-{k+1:03d}"
        relays.append(dict(id=rid, drone=r["drone"], pack=r["pack"], start=round(r["
            start"], 1), lon=h[0], lat=h[1], alt=h[2], agl=h[3],
                           t_link=round(r["t_link"], 1), t_end=round(r["t1"], 1),
                               t_ret=round(r["t_ret"], 1), E=round(r["E"], 4),
                           service=round(r["t1"] - r["t0"], 1), hi=r["h"], t0=r["t0
                               "], t1=r["t1"], ok_from=r["t_link"]))
        relay_rows.append([rid, r["drone"], r["pack"], round(r["start"], 1), round(h
            [0], 6), round(h[1], 6), round(h[2], 1),
                           round(r["t_link"], 1), round(r["t1"], 1), round(r["t_ret
                               "], 1), round(r["E"], 4)])
    out = dict(J=J, w=w, N=info["N"], E=round(info["E"], 4), mk=round(info["mk"], 1),
         late=round(info["late"], 4),
               hard=round(info["hard"], 4), NR=info["NR"], ER=round(info["ER"], 4),
                   mkR=round(info["mkR"], 1),
               mk_joint=round(info["mk_joint"], 1), delay=round(info["delay"], 1),
                   unc=round(info["unc"], 1),
               hovers=[dict(lon=h[0], lat=h[1], alt=h[2], agl=h[3]) for h in HOVERS],
                    shifts=shifts,
               sorties=rows, relays=relays, relay_rows=relay_rows)
    json.dump(out, open(path, "w", encoding="utf-8"), ensure_ascii=False, indent=1)
    return out

if __name__ == "__main__":
    seed = int(sys.argv[1]) if len(sys.argv) > 1 else 0
    iters = int(sys.argv[2]) if len(sys.argv) > 2 else 20000
    tag = sys.argv[3] if len(sys.argv) > 3 else "joint"
    init_path = sys.argv[4] if len(sys.argv) > 4 else str(ROOT / "results" / "q2" / "
        final.json")
    w = dict(W3)
    if len(sys.argv) > 5:
        w.update(json.loads(sys.argv[5]))
    init, init_sh = load_init(init_path)
    log = open(OUT / f"log_{tag}_s{seed}.txt", "w")
    if not init_sh:
        init_sh = structured_shifts(init, seed) if seed % 7 else initial_shifts(init)
    J0, i0 = evaluate3(init, init_sh, w)
    if i0["infeasible"]:
        print(f"初始解有 {i0['infeasible']} 个不可行架次，重新修复", file=log, flush=
            True)
```

```
        q2.evaluate = lambda s, ww=w: evaluate3(s, init_sh, ww)
        init = q2.repair([], list(range(NB)), random.Random(seed), w)
        J0, i0 = evaluate3(init, init_sh, w)
    print(f"init J={J0:.2f} shifts={len(init_sh)} NR={i0['NR']} ER={i0['ER']:.2f}
        delay={i0['delay']:.0f} unc={i0['unc']:.0f} mk_joint={i0['mk_joint']:.0f}",
        file=log, flush=True)
    (best_s, best_r), Jb = alns3(init, init_sh, seed, iters, w, log)
    best_r = trim_shifts(best_s, best_r, w)
    out = dump3(best_s, best_r, w, OUT / f"sol_{tag}_s{seed}.json")
    print(json.dumps({k: v for k, v in out.items() if k not in ("sorties", "relays",
        "relay_rows", "hovers", "shifts")}, ensure_ascii=False))
```

q3_relay.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
    深度求索，版本发布日期 2026 年。

import sys
import json
import math
from pathlib import Path
from multiprocessing import Pool
sys.path.insert(0, str(Path(__file__).resolve().parent))
import numpy as np
import pandas as pd
from common.q2model import NODES, types, T, depot, t_charge
from common.data import ROOT, load_relay_type
from common.comm import (trajectory, direct_status, available, GW, THR_ACCESS,
    THR_BACKHAUL)
from common.flight import G, J2KWH

OUT = ROOT / "results" / "q3"
OUT.mkdir(parents=True, exist_ok=True)
R = load_relay_type()
DT = 5.0
GRID = 400.0
HEIGHTS = (100.0, 200.0, 300.0)
GAP_SPLIT = 900.0
THIN = 4

def relay_flight(hover):

    zc, d = T.cruise_alt((depot.lon, depot.lat), (hover[0], hover[1]))
    zc = max(zc, hover[2])

    hu = zc - depot.ground
    hd = zc - hover[2]
    t_out = hu / R.v_up + d / R.v_cruise + hd / R.v_down
    E_out = R.P_cruise * (d / R.v_cruise) / 3600 + R.m_takeoff * G * hu / R.eta_up *
        J2KWH

    hu2 = zc - hover[2]
    hd2 = zc - depot.ground
    t_back = hu2 / R.v_up + d / R.v_cruise + hd2 / R.v_down
    E_back = R.P_cruise * (d / R.v_cruise) / 3600 + R.m_takeoff * G * hu2 / R.eta_up
        * J2KWH
    return t_out, E_out, t_back, E_back

def sortie_energy(hover, t_service):
    t_out, E_out, t_back, E_back = relay_flight(hover)
    E = E_out + E_back + (R.P_hover + R.P_comm) * (t_service + R.t_link) / 3600
    return E, t_out, t_back

def candidates():
    lons = [n.lon for n in NODES]
```

```
    lats = [n.lat for n in NODES]
    x0, y0 = T.xy(min(lons), min(lats))
    x1, y1 = T.xy(max(lons), max(lats))
    pad = 1000.0
    xs = np.arange(x0 - pad, x1 + pad + 1, GRID)
    ys = np.arange(y0 - pad, y1 + pad + 1, GRID)
    out = []
    for x in xs:
        for y in ys:
            lon, lat = T.ll(x, y)
            if not T.inside(lon, lat):
                continue
            g = float(T.elev(lon, lat))
            for h in HEIGHTS:
                out.append((float(lon), float(lat), g + h, h))
    return out

def _bh(c):
    ok, L, b = available(c[:3], GW, THR_BACKHAUL)
    return ok

def _cover(args):
    c, samples = args
    return [available(c[:3], s[1:4], THR_ACCESS)[0] for s in samples]

def plan(sol_path, tag):
    sol = json.load(open(sol_path, encoding="utf-8"))

    status = {}
    fails = []
    for s in sol["sorties"]:
        stops = tuple((a, tuple(bx)) for a, bx in zip(s["route"], s["boxes"]))
        st = direct_status(trajectory(s["type"], stops, s["start"], DT))
        status[s["id"]] = st
        for x in st:
            if not x[6]:
                fails.append((x[0], x[2], x[3], x[4], s["id"], x[1]))
    print(f"采样 {sum(len(v) for v in status.values())} 失联样本 {len(fails)}", flush
        =True)
    relay_rows, comm_rows = [], []
    metrics = dict(n_relay=0, E_relay=0.0, mk_relay=0.0, hover_s=0.0, fails=len(fails
        ))
    if not fails:
        return finish(sol, status, {}, relay_rows, comm_rows, metrics, tag)
    fails.sort()

    cands = candidates()
    with Pool(16) as p:
        bh = p.map(_bh, cands, chunksize=32)
    cands = [c for c, ok in zip(cands, bh) if ok]
    print("回传可用候选", len(cands), flush=True)
    thin = fails[::THIN]
    with Pool(16) as p:
        cov = p.map(_cover, [(c, thin) for c in cands], chunksize=8)
    cov = np.array(cov, dtype=bool)

    uncovered = np.ones(len(thin), dtype=bool)
    chosen = []
    while uncovered.any():
        gain = (cov & uncovered).sum(axis=1)
        if gain.max() == 0:
            print("有失联样本无候选点可覆盖", uncovered.sum())
            break
        best = int(np.argmax(gain - 1e-6 * np.array([T.horiz_dist((c[0], c[1]), (
            depot.lon, depot.lat)) for c in cands]) / 1000))
```

```
        chosen.append(best)
        uncovered &= ~cov[best]
    print("选中悬停点", [(round(cands[i][0], 5), round(cands[i][1], 5), cands[i][3]) for i in chosen], flush=True)

    hovers = [cands[i] for i in chosen]
    assign = {}
    for k, f in enumerate(fails):
        for hi, h in enumerate(hovers):
            if available(h[:3], f[1:4], THR_ACCESS)[0]:
                assign[k] = hi
                break
        else:
            assign[k] = None
    miss = [k for k, v in assign.items() if v is None]
    if miss:
        print("精确判定后仍有未覆盖样本", len(miss))

    windows = []
    for hi, h in enumerate(hovers):
        ts = sorted(fails[k][0] for k, v in assign.items() if v == hi)
        if not ts:
            continue
        cur = [ts[0], ts[0]]
        for t in ts[1:]:
            if t - cur[1] > GAP_SPLIT:
                windows.append((hi, cur[0], cur[1] + DT))
                cur = [t, t]
            else:
                cur[1] = t
        windows.append((hi, cur[0], cur[1] + DT))
    windows.sort(key=lambda w: w[1])

    avail = {d: 0.0 for d in R.drones}
    ready = {f"PR{k+1:02d}": 0.0 for k in range(R.n_pack)}
    budget = (1 - R.rho) * R.E_use
    relays = []
    for hi, t0, t1 in windows:
        h = hovers[hi]
        E, t_out, t_back = sortie_energy(h, t1 - t0)
        assert E <= budget, f"中继架次能量超限 {E:.2f} 服务 {t1-t0:.0f}s"
        d = min(avail, key=avail.get)
        p = min(ready, key=ready.get)
        start_need = t0 - R.t_prep - t_out - R.t_link
        start = max(start_need, avail[d], ready[p])
        if start > start_need + 1e-6:
            print(f"中继资源不足, 窗口 {t0:.0f}-{t1:.0f} 需推迟 {start-start_need:.0f}s")
        t_link_done = start + R.t_prep + t_out + R.t_link
        t_end = t1
        t_ret = t_end + t_back
        soc = 1 - E / R.E_use
        avail[d] = t_ret + R.t_turn
        ready[p] = t_ret + t_charge(soc, R.T_full)
        rid = f"R-{len(relays)+1:03d}"
        relays.append(dict(id=rid, drone=d, pack=p, start=round(start, 1), lon=h[0], lat=h[1], alt=h[2], agl=h[3],
                           t_link=round(t_link_done, 1), t_end=round(t_end, 1), t_ret=round(t_ret, 1), E=round(E, 4),
                           service=round(t1 - t0, 1), hi=hi, t0=t0, t1=t1, ok_from=t_link_done))
        relay_rows.append([rid, d, p, round(start, 1), round(h[0], 6), round(h[1], 6), round(h[2], 1),
                           round(t_link_done, 1), round(t_end, 1), round(t_ret, 1), round(E, 4)])
```

```
    metrics.update(n_relay=len(relays), E_relay=round(sum(r["E"] for r in relays), 4),
                   mk_relay=round(max((r["t_ret"] for r in relays), default=0), 1),
                   hover_s=round(sum(r["service"] for r in relays), 1))
    return finish(sol, status, {i: h for i, h in enumerate(hovers)}, relay_rows, comm_rows, metrics, tag, relays)

def finish(sol, status, hovers, relay_rows, comm_rows, metrics, tag, relays=()):

    bad = 0
    for s in sol["sorties"]:
        st = status[s["id"]]
        seg = None
        for x in st:
            t, ph = x[0], x[1]
            if ph == "ground" or x[6]:
                mode, rid = "直连", ""
            else:
                mode, rid = "中断", ""
                for r in relays:
                    if r["ok_from"] - 1e-6 <= t <= r["t_end"] + 1e-6 and available(hovers[r["hi"]][:3], (x[2], x[3], x[4]), THR_ACCESS)[0]:
                        mode, rid = "中继", r["id"]
                        break
                if mode == "中断":
                    bad += 1
            key = (ph, mode, rid)
            if seg is None or seg[0] != key:
                if seg is not None:
                    comm_rows.append([s["id"], seg[0][0], round(seg[1], 1), round(t, 1), seg[0][1], seg[0][2]])
                seg = [key, t]
        comm_rows.append([s["id"], seg[0][0], round(seg[1], 1), round(st[-1][0], 1), seg[0][1], seg[0][2]])
    metrics["uncovered_samples"] = bad
    mk_joint = max(sol["mk"], metrics["mk_relay"])
    out = dict(source=str(sol_path_global), tag=tag, q2=dict(N=sol["N"], E=sol["E"], mk=sol["mk"], late=sol["late"], hard=sol["hard"]),
               relay=metrics, mk_joint=mk_joint, E_total=round(sol["E"] + metrics["E_relay"], 4),
               hovers={i: dict(lon=h[0], lat=h[1], alt=h[2], agl=h[3]) for i, h in hovers.items()},
               relays=list(relays), relay_rows=relay_rows, comm_rows=comm_rows, sorties=sol["sorties"])
    json.dump(out, open(OUT / f"{tag}.json", "w", encoding="utf-8"), ensure_ascii=False, indent=1)
    print(json.dumps(dict(tag=tag, q2=out["q2"], relay=metrics, mk_joint=mk_joint, E_total=out["E_total"]), ensure_ascii=False))
    return out

if __name__ == "__main__":
    sol_path_global = sys.argv[1]
    tag = sys.argv[2] if len(sys.argv) > 2 else Path(sol_path_global).stem
    plan(sol_path_global, tag)
```

q3_run.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import sys
import json
import subprocess
from pathlib import Path
```

```
from concurrent.futures import ThreadPoolExecutor

HERE = Path(__file__).resolve().parent
ITERS = int(sys.argv[1]) if len(sys.argv) > 1 else 20000
INIT = sys.argv[2] if len(sys.argv) > 2 else str(HERE.parent / "results" / "q2" / "final.json")
CONFIGS = {
    "joint": {},
    "jtime": {"late": 20.0, "hard": 200.0},
    "jmk": {"mk": 12.0, "mkR": 12.0},
    "jenergy": {"E": 4.0, "ER": 4.0},
    "jsortie": {"N": 6.0, "NR": 6.0},
}
SEEDS = {"joint": range(1, 8), "jtime": (1, 2, 3), "jmk": (1, 2, 3), "jenergy": (1, 2), "jsortie": (1, 2)}
jobs = [(tag, s, w) for tag, w in CONFIGS.items() for s in SEEDS[tag]]

def run(job):
    tag, s, w = job
    cmd = [sys.executable, str(HERE / "q3_joint.py"), str(s), str(ITERS), tag, INIT, json.dumps(w)]
    r = subprocess.run(cmd, capture_output=True, text=True)
    return tag, s, r.stdout.strip().splitlines()[-1] if r.stdout.strip() else r.stderr[-800:]

with ThreadPoolExecutor(max_workers=len(jobs)) as ex:
    for tag, s, line in ex.map(run, jobs):
        print(tag, s, line, flush=True)
```

q3_pick.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import sys
import json
import glob
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
import pandas as pd
from common.data import ROOT
from q3_joint import load_init, evaluate3, trim_shifts, dump3, W3
import q3_verify

D = ROOT / "results" / "q3"
rows = []
cands = {}
for f in sorted(glob.glob(str(D / "sol_*_s*.json"))):
    name = Path(f).name
    if name.startswith("sol_pipe"):
        continue
    tag = name.split("_")[1]
    try:
        sorties, shifts = load_init(f)
        shifts = trim_shifts(sorties, shifts, W3)
        J, info = evaluate3(sorties, shifts, W3)
    except Exception as e:
        print("skip", name, e)
        continue
    cands[name] = (sorties, shifts)
    rows.append(dict(file=name, tag=tag, J=round(J, 2), N=info["N"], E=round(info["E"], 2), mk=round(info["mk"]),
                     NR=info["NR"], ER=round(info["ER"], 2), mkR=round(info["mkR"]), mk_joint=round(info["mk_joint"]),
                     delay=round(info["delay"]), unc=round(info["unc"]), late=round(
```

```
                            info["late"], 2), hard=round(info["hard"], 2)))
df = pd.DataFrame(rows).sort_values("J")
pd.set_option("display.width", 220)
print(df.to_string(index=False))
df.to_csv(D / "summary.csv", index=False, encoding="utf-8-sig")
best_tag = df.sort_values("J").groupby("tag").head(1)
best_tag.to_csv(D / "tradeoff.csv", index=False, encoding="utf-8-sig")
print("\n各权重最优: ")
print(best_tag.to_string(index=False))

ok = df[(df.hard == 0) & (df.unc == 0)]
pick = (ok if len(ok) else df).iloc[0]["file"]
sorties, shifts = cands[pick]
out = dump3(sorties, shifts, W3, D / "best.json")
print("pick", pick, {k: out[k] for k in ("N", "E", "mk", "NR", "ER", "mk_joint", "
    late", "hard", "unc")})
bad = q3_verify.main(str(D / "best.json"))
print("verify uncovered", bad)
```

q3_analyze.py

```
# 本程序及代码是在人工智能工具辅助下完成的: 工具 DeepSeek, 模型 DeepSeek-V3, 开发机构
    深度求索, 版本发布日期 2026 年。

import sys
import json
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
import pandas as pd
from common.q2model import NODES, boxes
from common.comm import trajectory, direct_status, THR_DIRECT, THR_ACCESS,
    THR_BACKHAUL, max_range_km
from common.data import ROOT

OUT = ROOT / "results" / "q3"
OUT.mkdir(parents=True, exist_ok=True)
sol = json.load(open(sys.argv[1], encoding="utf-8"))
print("门限", THR_DIRECT, THR_ACCESS, THR_BACKHAUL)
rows = []
segs = []
for s in sol["sorties"]:
    stops = tuple((a, tuple(bx)) for a, bx in zip(s["route"], s["boxes"]))
    pts = trajectory(s["type"], stops, s["start"])
    st = direct_status(pts)
    n_bad = sum(1 for x in st if not x[6])
    rows.append(dict(id=s["id"], type=s["type"], route=s["route"], n=len(st), n_fail=
        n_bad,
                     fail_frac=round(n_bad / len(st), 3)))

    cur = None
    for x in st:
        if not x[6]:
            if cur is None:
                cur = [x[0], x[0], x[1], x[5], 1]
            else:
                cur[1] = x[0]
                cur[4] += 1
        elif cur is not None:
            segs.append(dict(id=s["id"], t0=cur[0], t1=cur[1], phase=cur[2], node=cur
                [3], n=cur[4]))
            cur = None
    if cur is not None:
        segs.append(dict(id=s["id"], t0=cur[0], t1=cur[1], phase=cur[2], node=cur[3],
             n=cur[4]))
    pd.DataFrame(st, columns=["t", "phase", "lon", "lat", "alt", "node", "direct", "L
```

```
        ", "blocked"]).to_csv(
        OUT / f"traj_{s['id']}.csv", index=False)
df = pd.DataFrame(rows)
print(df.to_string(index=False))
sd = pd.DataFrame(segs)
if len(sd):
    sd["dur"] = sd.t1 - sd.t0 + 5
    print(sd.to_string(index=False))
    print("失联总时长 s", sd.dur.sum(), " 段数", len(sd))
    print("按节点汇总:", sd.groupby("node").dur.sum().to_dict())
sd.to_csv(OUT / "direct_fail_segments.csv", index=False)
df.to_csv(OUT / "direct_summary.csv", index=False)

from common.comm import available, GW
print("服务区作业点直连:")
for i, n in enumerate(NODES[1:], 1):
    ok, L, b = available((n.lon, n.lat, n.work_alt), GW, THR_DIRECT)
    print(n.id, "ok" if ok else "FAIL", round(L, 1), "blocked" if b else "los")
```

q3_verify.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import sys
import json
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
from common.comm import trajectory, direct_status, available, GW, THR_ACCESS,
    THR_BACKHAUL
from common.q2model import check
from common.data import ROOT, load_relay_type

OUT = ROOT / "results" / "q3"
R = load_relay_type()

def main(path):
    sol = json.load(open(path, encoding="utf-8"))
    relays = sol["relays"]
    hov = [(h["lon"], h["lat"], h["alt"]) for h in sol["hovers"]]

    for r in relays:
        assert available(hov[r["hi"]], GW, THR_BACKHAUL)[0], f"{r['id']} 回传不可用"
        assert r["E"] <= (1 - R.rho) * R.E_use + 1e-6, f"{r['id']} 能量超限"
    busy = {}
    for r in relays:
        busy.setdefault(r["drone"], []).append((r["start"], r["t_ret"] + R.t_turn))
        busy.setdefault(r["pack"], []).append((r["start"], r["t_ret"]))
    for k, iv in busy.items():
        iv.sort()
        for (a, b), (c, d) in zip(iv, iv[1:]):
            assert c >= b - 1e-6, f"{k} 中继资源重叠 {b:.0f}>{c:.0f}"

    comm_rows = []
    bad = 0
    total = 0
    n_relay_samples = 0
    for s in sol["sorties"]:
        stops = tuple((a, tuple(bx)) for a, bx in zip(s["route"], s["boxes"]))
        st = direct_status(trajectory(s["type"], stops, s["start"]))
        seg = None
        for x in st:
            t, ph = x[0], x[1]
            total += 1
```

```
            if ph == "ground" or x[6]:
                mode, rid = "直连", ""
            else:
                mode, rid = "中断", ""
                for r in relays:
                    if r["t_link"] - 0.2 <= t <= r["t_end"] + 0.2 and available(hov[r["hi"]], (x[2], x[3], x[4]), THR_ACCESS)[0]:
                        mode, rid = "中继", r["id"]
                        break
                if mode == "中断":
                    bad += 1
                else:
                    n_relay_samples += 1
            key = (ph, mode, rid)
            if seg is None or seg[0] != key:
                if seg is not None:
                    comm_rows.append([s["id"], seg[0][0], round(seg[1], 1), round(t, 1), seg[0][1], seg[0][2]])
                seg = [key, t]
        comm_rows.append([s["id"], seg[0][0], round(seg[1], 1), round(st[-1][0], 1), seg[0][1], seg[0][2]])
    sorties_q2 = [(s["type"], tuple((a, tuple(bx)) for a, bx in zip(s["route"], s["boxes"]))) for s in sol["sorties"]]
    check(sorties_q2)
    sol["comm_rows"] = comm_rows
    sol["verify"] = dict(samples=total, uncovered=bad, relay_samples=n_relay_samples)
    print(json.dumps(dict(N=sol["N"], E=sol["E"], mk=sol["mk"], NR=sol["NR"], ER=sol["ER"], mk_joint=sol["mk_joint"],
                          delay=sol["delay"], verify=sol["verify"]), ensure_ascii=False))
    json.dump(sol, open(OUT / "final.json", "w", encoding="utf-8"), ensure_ascii=False, indent=1)
    return bad

if __name__ == "__main__":
    sys.exit(1 if main(sys.argv[1]) else 0)
```

q4.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构 深度求索，版本发布日期 2026 年。

import sys
import json
import itertools
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
import numpy as np
import pandas as pd
from common.q2model import NODES, types, sortie_eval, t_charge, boxes
from common.data import ROOT, load_relay_type

OUT = ROOT / "results" / "q4"
OUT.mkdir(parents=True, exist_ok=True)
R = load_relay_type()
INV = dict(A=len(types["A"].drones), B=len(types["B"].drones), C=len(types["C"].drones),
           bA=types["A"].n_batt, bB=types["B"].n_batt, bC=types["C"].n_batt, R=len(R.drones), P=R.n_pack)

def max_overlap(intervals):
    ev = []
    for s, e in intervals:
        ev.append((s, 1))
```

```
        ev.append((e - 0.5, -1))
    ev.sort(key=lambda x: (x[0], x[1]))
    cur = best = 0
    for _, d in ev:
        cur += d
        best = max(best, cur)
    return best

def group_resources(sorties, relays, relay_of_sortie):

    res = {}
    for g in types:
        S = [s for s in sorties if s["type"] == g]
        res[g] = max_overlap([(s["start"], s["ret"]) for s in S])
        res["b" + g] = max_overlap([(s["start"], s["ret"] + t_charge(s["soc"] / 100,
            types[g].T_full)) for s in S])
    rids = set()
    for s in sorties:
        rids |= relay_of_sortie.get(s["id"], set())
    RS = [r for r in relays if r["id"] in rids]
    res["R"] = max_overlap([(r["start"], r["t_ret"] + R.t_turn) for r in RS])
    res["P"] = max_overlap([(r["start"], r["t_ret"] + t_charge(1 - r["E"] / R.E_use,
        R.T_full)) for r in RS])
    res["n_sortie"] = len(sorties)
    res["n_relay"] = len(RS)
    res["E"] = round(sum(s["E"] for s in sorties) + sum(r["E"] for r in RS), 3)
    res["work_s"] = round(sum(s["ret"] - s["start"] for s in sorties), 1)
    res["mk"] = round(max([s["ret"] for s in sorties] + [r["t_ret"] for r in RS]), 1)
    res["n_box"] = sum(len(bx) for s in sorties for bx in s["boxes"])
    return res

def main(path):
    sol = json.load(open(path, encoding="utf-8"))
    sorties = sol["sorties"]
    relays = sol.get("relays", [])
    relay_of_sortie = {}
    for row in sol.get("comm_rows", []):
        sid, ph, t0, t1, mode, rid = row
        if mode == "中继":
            relay_of_sortie.setdefault(sid, set()).add(rid)

    parent = {i: i for i in range(1, 16)}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for s in sorties:
        for a in s["route"][1:]:
            parent[find(a)] = find(s["route"][0])
    blocks = {}
    for a in range(1, 16):
        blocks.setdefault(find(a), []).append(a)
    blocks = list(blocks.values())
    print("原子块", [[NODES[a].id for a in b] for b in blocks])
    nb = len(blocks)
    results = {}
    for K in (2, 3):
        rows = []
        seen = set()
        for labels in itertools.product(range(K), repeat=nb):
            if len(set(labels)) < K:
                continue
```

```
86
87              order = {}
88              canon = tuple(order.setdefault(l, len(order)) for l in labels)
89              if canon in seen:
90                  continue
91              seen.add(canon)
92              groups = [[] for _ in range(K)]
93              for l, b in zip(canon, blocks):
94                  groups[l] += b
95              gres = []
96              for gareas in groups:
97                  gs = [s for s in sorties if s["route"][0] in gareas]
98                  gres.append(group_resources(gs, relays, relay_of_sortie))
99              tot = {k: sum(r[k] for r in gres) for k in ("A", "B", "C", "bA", "bB", "
                    bC", "R", "P")}
100             total_units = sum(tot.values())
101             gap = {k: max(0, tot[k] - INV[k]) for k in tot}
102             work = [r["work_s"] for r in gres]
103             imb = (max(work) - min(work)) / max(1, sum(work)) * K
104             rows.append(dict(K=K, groups=[[NODES[a].id for a in g] for g in groups],
                    res=gres, tot=tot,
105                              total_units=total_units, gap=gap, gap_units=sum(gap.
                                 values()), imb=round(imb, 4)))
106         rows.sort(key=lambda r: (r["gap_units"], r["total_units"], r["imb"]))
107         results[K] = rows
108         if not rows:
109             print(f"K={K} 原子块只有 {nb} 个，无法分成 {K} 组")
110             continue
111         print(f"K={K} 共 {len(rows)} 种分区，最优前 5：")
112         for r in rows[:5]:
113             print("  ", r["groups"], "总资源", r["total_units"], "缺口", r["gap"], "失
                    衡", r["imb"])
114
115     whole = group_resources(sorties, relays, relay_of_sortie)
116     best = {K: results[K][0] for K in (2, 3)}
117     out_rows = []
118     for K in (2, 3):
119         for gi, (gareas, r) in enumerate(zip(best[K]["groups"], best[K]["res"])):
120             out_rows.append([K, f"G{K}-{gi+1}", ";".join(gareas), r["A"], r["B"], r["
                    C"], r["bA"], r["bB"], r["bC"], r["R"], r["P"]])
121     json.dump(dict(inventory=INV, whole=whole, best={K: best[K] for K in (2, 3)},
122                    top={K: results[K][:10] for K in (2, 3)}, rows=out_rows,
                           n_partitions={K: len(results[K]) for K in (2, 3)}),
123               open(OUT / "final.json", "w", encoding="utf-8"), ensure_ascii=False,
                      indent=1)
124     pd.DataFrame(out_rows, columns=["K", "任务组", "服务区", "A机", "B机", "C机", "A
            电池", "B电池", "C电池", "中继机", "能源组件"]).to_csv(
125         OUT / "Q4_分区配置.csv", index=False, encoding="utf-8-sig")
126     print("整体需求", whole)
127     print("库存", INV)
128
129 if __name__ == "__main__":
130     main(sys.argv[1] if len(sys.argv) > 1 else str(ROOT / "results" / "q3" / "final.
            json"))
```

export.py

```
1 # 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
      深度求索，版本发布日期 2026 年。
2
3 import sys
4 import json
5 from pathlib import Path
6 sys.path.insert(0, str(Path(__file__).resolve().parent))
```

```
import pandas as pd
from openpyxl import load_workbook
from common.data import ROOT
from common import paths as P

TPL = Path(P.SAITI) / "结果提交模板.xlsx"
RES = ROOT / "results"
OUTX = RES / "结果文件.xlsx"

def fill(ws, rows):
    for r in range(ws.max_row, 1, -1):
        ws.delete_rows(r)
    for row in rows:
        ws.append(list(row))

def main():
    wb = load_workbook(TPL)

    f = RES / "q1" / "Q1_单点组批.csv"
    if f.exists():
        df = pd.read_csv(f)
        fill(wb["Q1_单点组批"], df.itertuples(index=False))
        print("Q1", len(df))

    f = RES / "q2" / "final.json"
    if f.exists():
        sol = json.load(open(f, encoding="utf-8"))
        from common.q2model import NODES
        rows, drows = [], []
        for s in sol["sorties"]:
            rows.append((s["id"], s["drone"], s["type"], s["battery"], s["start"],
                         "→".join(NODES[a].id for a in s["route"]), s["ret"], s["E"])
                        )
            for bid, t in s["deliv"].items():
                drows.append((bid, s["id"], bid[:4], t))
        drows.sort()
        fill(wb["Q2_运输架次"], rows)
        fill(wb["Q2_逐箱交付"], drows)
        print("Q2", len(rows), len(drows))

    f = RES / "q3" / "final.json"
    if f.exists():
        sol = json.load(open(f, encoding="utf-8"))
        fill(wb["Q3_中继架次"], [tuple(r) for r in sol["relay_rows"]])

        PH = {"ground": "地面准备", "climb": "爬升", "cruise": "巡航",
              "descent": "下降", "handover": "投送"}
        rows = [(r[0], PH.get(r[1], r[1])) + tuple(r[2:]) for r in sol["comm_rows"]]
        fill(wb["Q3_通信保障"], rows)
        print("Q3", len(sol["relay_rows"]), len(sol["comm_rows"]))

    f = RES / "q4" / "final.json"
    if f.exists():
        sol = json.load(open(f, encoding="utf-8"))
        fill(wb["Q4_分区配置"], [tuple(r) for r in sol["rows"]])
        print("Q4", len(sol["rows"]))
    wb.save(OUTX)
    print("saved", OUTX)

if __name__ == "__main__":
    main()
```

srv.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构深度求索，版本发布日期 2026 年。

import os
import sys
import stat

HOST, USER = "10.16.13.145", "liutianrui"
REMOTE = "/data1/liutianrui/yjs2026D"
PY = REMOTE + "/venv/bin/python"
LOCAL = os.path.dirname(os.path.abspath(__file__))
VPN_SKILL = r"C:\Users\34752\.claude\skills\campus-vpn-server"

def connect():

    if VPN_SKILL not in sys.path:
        sys.path.insert(0, VPN_SKILL)
    from connect import ssh
    c = ssh(route=os.environ.get("SRV_ROUTE") or "auto")
    print("通路:", getattr(c, "_route", "?"))
    return c

def disconnect(c):

    try:
        from connect import close
        close(c)
    except Exception:
        c.close()

def _mkdir(s, p):
    try:
        s.stat(p)
    except IOError:
        _mkdir(s, os.path.dirname(p))
        s.mkdir(p)

def push(c, sub="code"):
    s = c.open_sftp()
    src = os.path.join(LOCAL, sub)
    n = 0
    for root, dirs, files in os.walk(src):
        dirs[:] = [d for d in dirs if d != "__pycache__"]
        rel = os.path.relpath(root, LOCAL).replace(os.sep, "/")
        d = REMOTE + "/" + rel
        _mkdir(s, d)
        for f in files:
            if f.endswith(".pyc"):
                continue
            s.put(os.path.join(root, f), d + "/" + f)
            n += 1
    s.close()
    print(f"pushed {n} files")

def pull(c, sub="results"):
    s = c.open_sftp()
    n = 0

    def walk(rp, lp):
        nonlocal n
        os.makedirs(lp, exist_ok=True)
        for e in s.listdir_attr(rp):
            r = rp + "/" + e.filename
            l = os.path.join(lp, e.filename)
            if stat.S_ISDIR(e.st_mode):
```

```
                walk(r, l)
            else:
                s.get(r, l)
                n += 1

    walk(REMOTE + "/" + sub, os.path.join(LOCAL, sub))
    s.close()
    print(f"pulled {n} files")

def run(c, cmd, timeout=3600):
    full = f"cd {REMOTE} && export PYTHONIOENCODING=utf-8 PATH={REMOTE}/venv/bin:
        $PATH && {cmd}"
    _, o, e = c.exec_command(full, timeout=timeout)
    out = o.read().decode("utf-8", "replace")
    err = e.read().decode("utf-8", "replace")
    print(out)
    if err.strip():
        print("[stderr]\n" + err[-4000:])
    return o.channel.recv_exit_status()

if __name__ == "__main__":
    c = connect()
    a = sys.argv[1] if len(sys.argv) > 1 else "push"
    if a == "push":
        push(c)
    elif a == "run":
        push(c)
        rc = run(c, " ".join(sys.argv[2:]))
        sys.exit(rc)
    elif a == "pull":
        pull(c, sys.argv[2] if len(sys.argv) > 2 else "results")
    disconnect(c)
```

common/__init__.py

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
    深度求索，版本发布日期 2026 年。
```

q2_ablation.sh

```
#!/bin/bash
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
    深度求索，版本发布日期 2026 年。
# 问题二多算法对比：贪心构造、纯局部扰动模拟退火、只接受改进的大邻域搜索，各 6 个种
    子。
# 本文的混合算法沿用 base 权重下已有的 6 个种子结果。
cd /data1/liutianrui/yjs2026D
mkdir -p results/q2_abl
for s in 0 1 2 3 4 5; do
  Q2_MODE=greedy venv/bin/python code/q2.py $s 1 greedy > /dev/null 2>&1 &
  Q2_MODE=sa  nohup venv/bin/python code/q2.py $s 40000 sa  > results/q2_abl/sa_$s.
      out 2>&1 < /dev/null &
  Q2_MODE=lns nohup venv/bin/python code/q2.py $s 40000 lns > results/q2_abl/lns_$s.
      out 2>&1 < /dev/null &
done
sleep 2
echo "started: $(ps -eo args | grep '[q]2.py' | wc -l)"
```

q2_ga_run.sh

```
# 本程序及代码是在人工智能工具辅助下完成的：工具 DeepSeek，模型 DeepSeek-V3，开发机构
    深度求索，版本发布日期 2026 年。
# 并行跑遗传算法的多个种子。用法：bash code/q2_ga_run.sh [代数] [种子数]
```

```
GENS=${1:-200}
NSEED=${2:-12}
HERE="$(cd "$(dirname "$0")" && pwd)"
OUT="$HERE/../results/q2"
mkdir -p "$OUT"
cd "$HERE" || exit 1
for s in $(seq 1 "$NSEED"); do
  python q2_ga.py "$s" "$GENS" 24 > "$OUT/ga_out_s${s}.log" 2>&1 &
done
wait
echo "全部结束"
```